

情報エレクトロニクス実験III (4年前期)

SQL、データベース

2025/6/27, 7/4, 7/11, 7/25, 8/1

シラバス

到達目標

- ・ システムプログラミングを理解してシェルの開発ができ、データベースの概要の理解、SQL コマンドでの簡単なデータベース操作ができ、文書検索のプログラムを作成し、基本的な検索エンジンの仕組みが理解できること。
- ・ これらの内容を満足することで、学習・教育目標のD-1及びD-2の達成とする。

D-2

- ・ 基盤となる工学分野において、論理展開に必要な基礎問題を解くことができる。

D-3

- ・ 基盤となる工学分野以外の工学分野の基礎的な知識を身につける。

スケジュール

データベースの概要

- 6/27

MySQL

- 7/4、7/11

課題演習

- 7/25、8/1

評価

- レポート: 70%
- 毎回の取り組み: 20%
 - Formsからその日にやった内容を文章で提出
- 期限内提出: 10%

本実習の目的

- データベースを使ってデータを保存する、抽出する、整理する方法を学ぶ
- SQL (データベースとやり取りする言語) を使って、データベースを使う方法を学ぶ
- 正規化の種類や適用方法を把握し、任意のデータを抽出する方法を学ぶ

MySQL

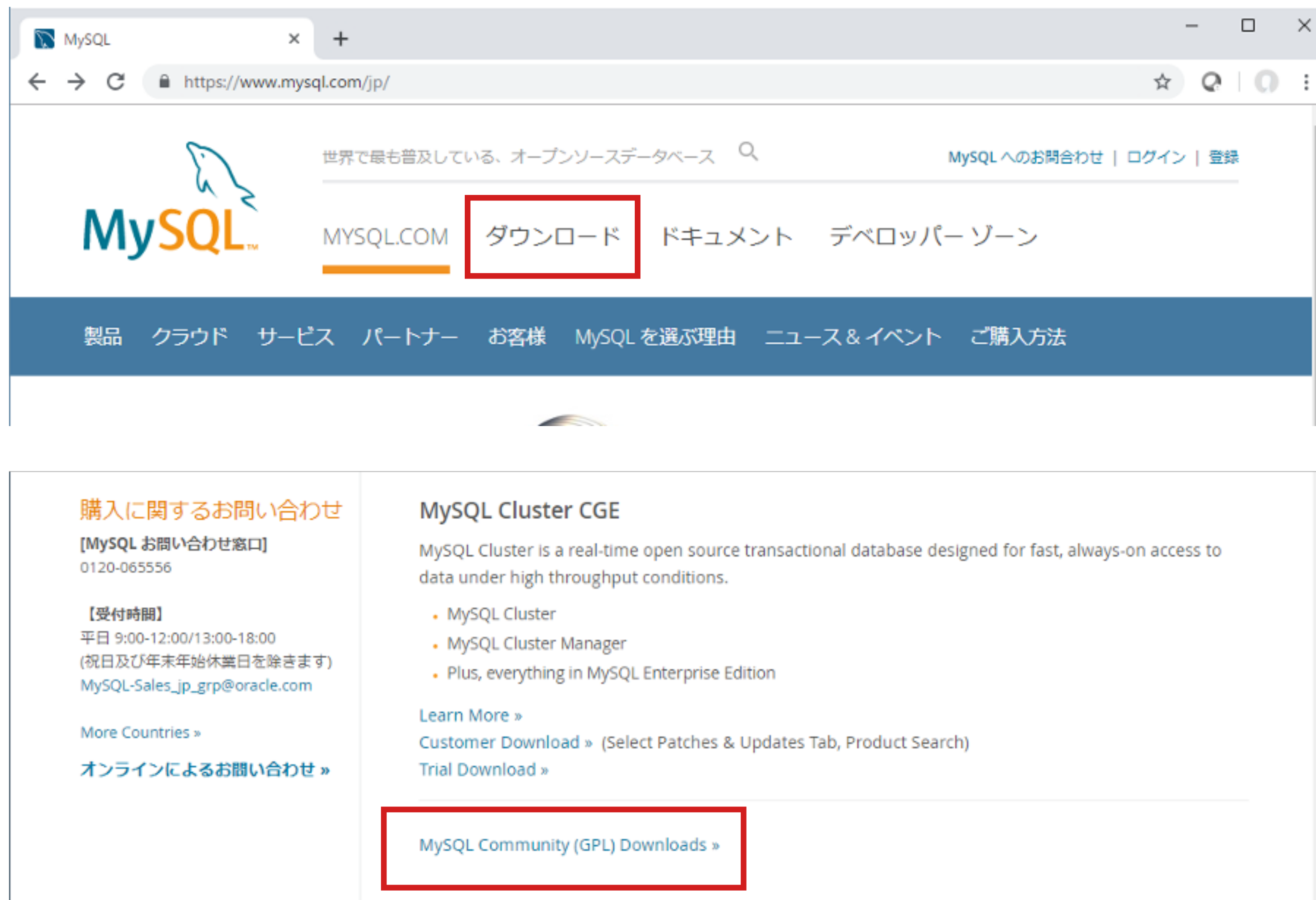
ダウンロード

- 以下のアドレスをブラウザで開く

<https://www.mysql.com/jp/>

MySQL

ダウンロード



The screenshot shows the MySQL website interface. The browser address bar displays `https://www.mysql.com/jp/`. The main navigation bar includes the MySQL logo, a search bar, and links for "MySQL へのお問い合わせ | ログイン | 登録". Below this, a secondary navigation bar features "MYSQL.COM", "ダウンロード" (highlighted with a red box), "ドキュメント", and "デベロッパーゾーン". A dark blue horizontal bar contains links for "製品", "クラウド", "サービス", "パートナー", "お客様", "MySQL を選ぶ理由", "ニュース & イベント", and "ご購入方法". The footer area is divided into two columns. The left column, titled "購入に関するお問い合わせ", provides contact information for MySQL sales, including a phone number (0120-065556), reception hours (9:00-12:00/13:00-18:00), and an email address (MySQL-Sales_jp_grp@oracle.com). It also includes a link for "More Countries »" and a link for "オンラインによるお問い合わせ »". The right column, titled "MySQL Cluster CGE", describes the product as a real-time open source transactional database. It lists features: "MySQL Cluster", "MySQL Cluster Manager", and "Plus, everything in MySQL Enterprise Edition". Below this, there are links for "Learn More »", "Customer Download » (Select Patches & Updates Tab, Product Search)", and "Trial Download »". At the bottom of the right column, a link for "MySQL Community (GPL) Downloads »" is highlighted with a red box.

MySQL

世界で最も普及している、オープンソースデータベース

MySQL へのお問い合わせ | ログイン | 登録

MYSQL.COM **ダウンロード** ドキュメント デベロッパーゾーン

製品 クラウド サービス パートナー お客様 MySQL を選ぶ理由 ニュース & イベント ご購入方法

購入に関するお問い合わせ
[MySQL お問い合わせ窓口]
0120-065556

【受付時間】
平日 9:00-12:00/13:00-18:00
(祝日及び年末年始休業日を除きます)
MySQL-Sales_jp_grp@oracle.com

More Countries »

オンラインによるお問い合わせ »

MySQL Cluster CGE
MySQL Cluster is a real-time open source transactional database designed for fast, always-on access to data under high throughput conditions.

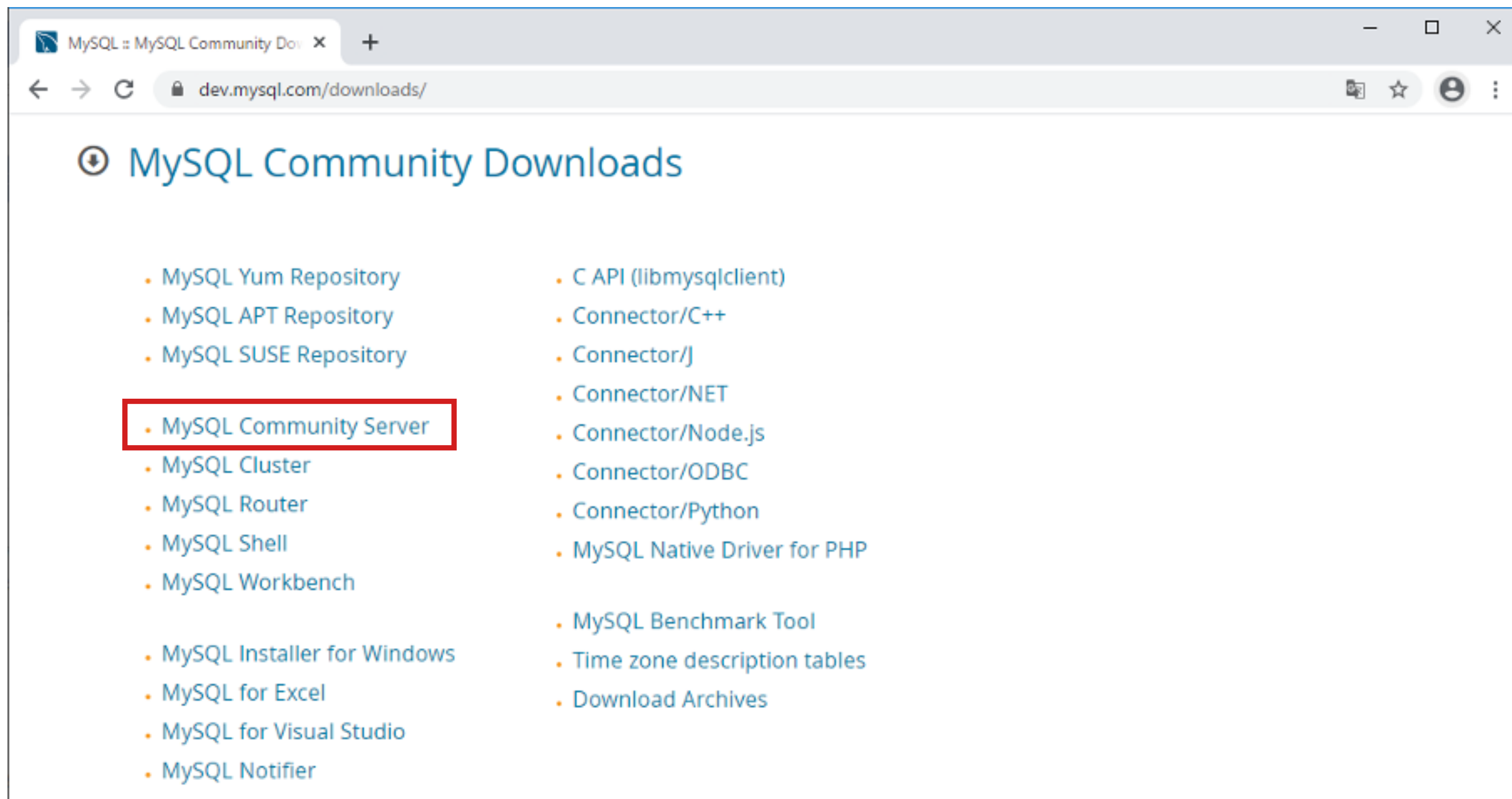
- MySQL Cluster
- MySQL Cluster Manager
- Plus, everything in MySQL Enterprise Edition

Learn More »
Customer Download » (Select Patches & Updates Tab, Product Search)
Trial Download »

MySQL Community (GPL) Downloads »

MySQL

ダウンロード



MySQL

ダウンロード

General Availability (GA) Releases Archives ⓘ

MySQL Community Server 8.4.5 LTS

Select Version:

8.4.5 LTS

Select Operating System:

Microsoft Windows

Windows (x86, 64-bit), MSI Installer (mysql-8.4.5-winx64.msi)	8.4.5	128.6M	Download
MD5: d33a60083c590cf722b8ace012b90ed3 Signature			
Windows (x86, 64-bit), ZIP Archive (mysql-8.4.5-winx64.zip)	8.4.5	247.8M	Download
MD5: 6e177792f2ccbed67af977768ec7d1d5 Signature			
Windows (x86, 64-bit), ZIP Archive Debug Binaries & Test Suite (mysql-8.4.5-winx64-debug-test.zip)	8.4.5	755.1M	Download
MD5: d43be30ecf372faaabcc67c70e4f53b1 Signature			

MySQL

ダウンロード

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »

using my Oracle Web account

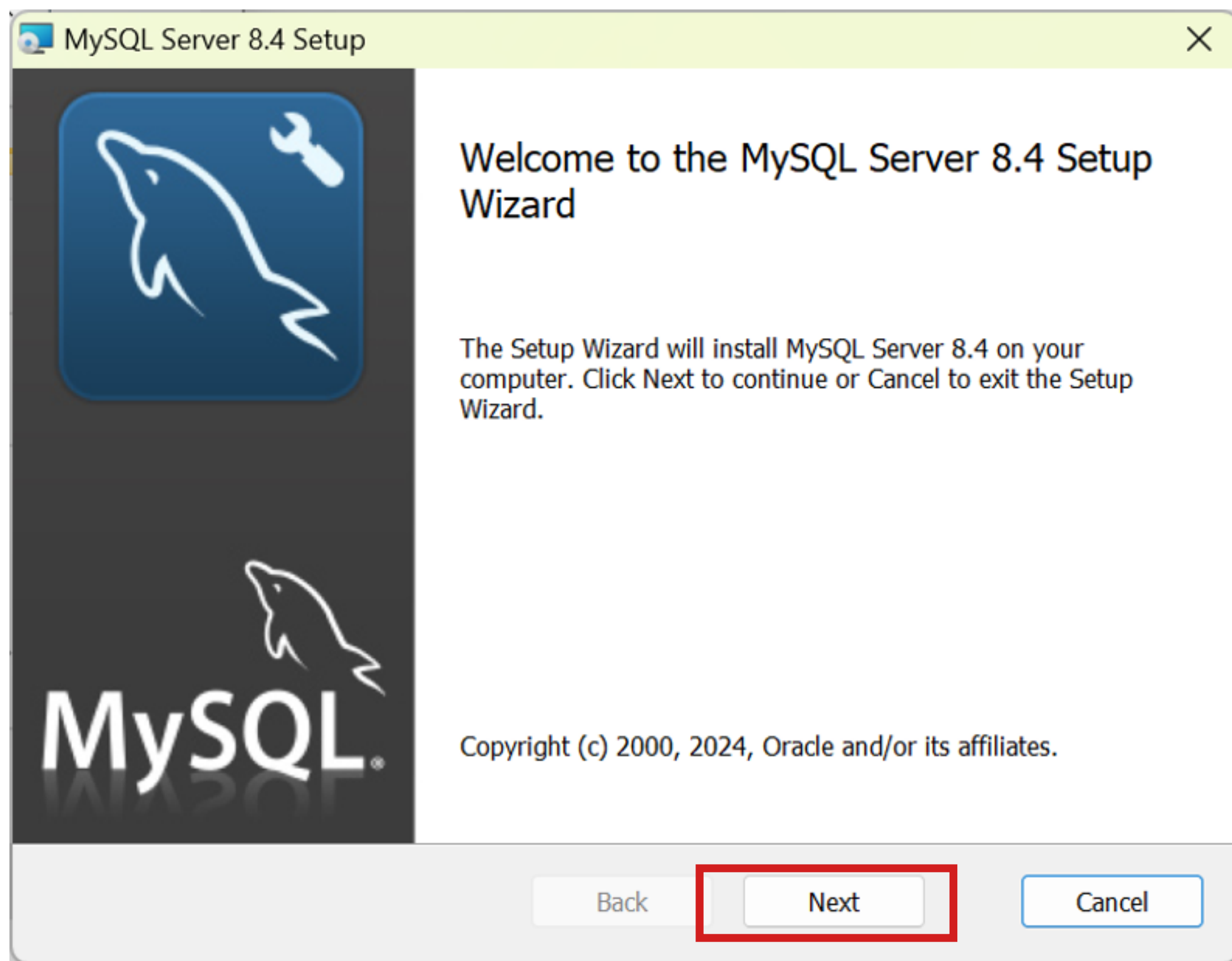
Sign Up »

for an Oracle Web account

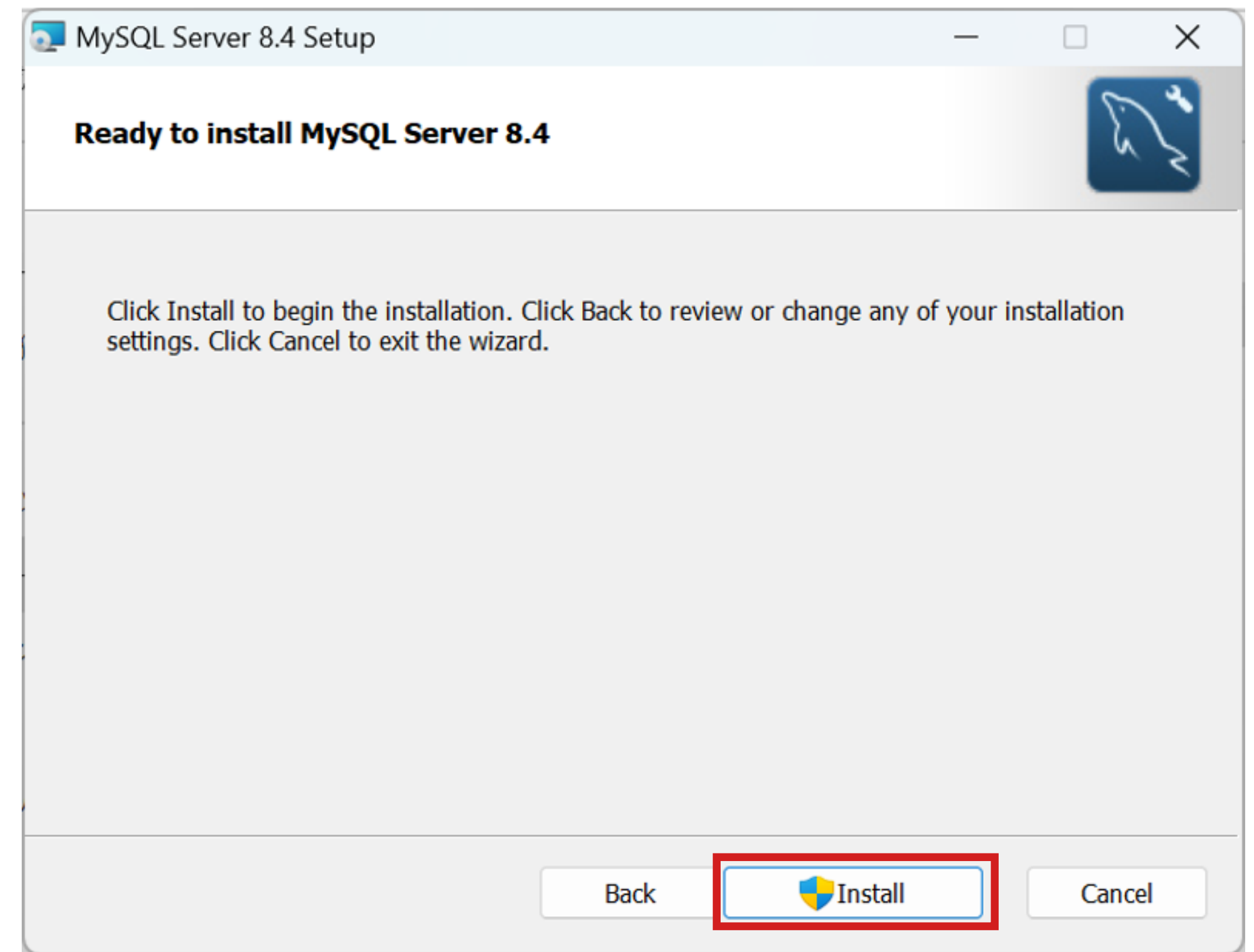
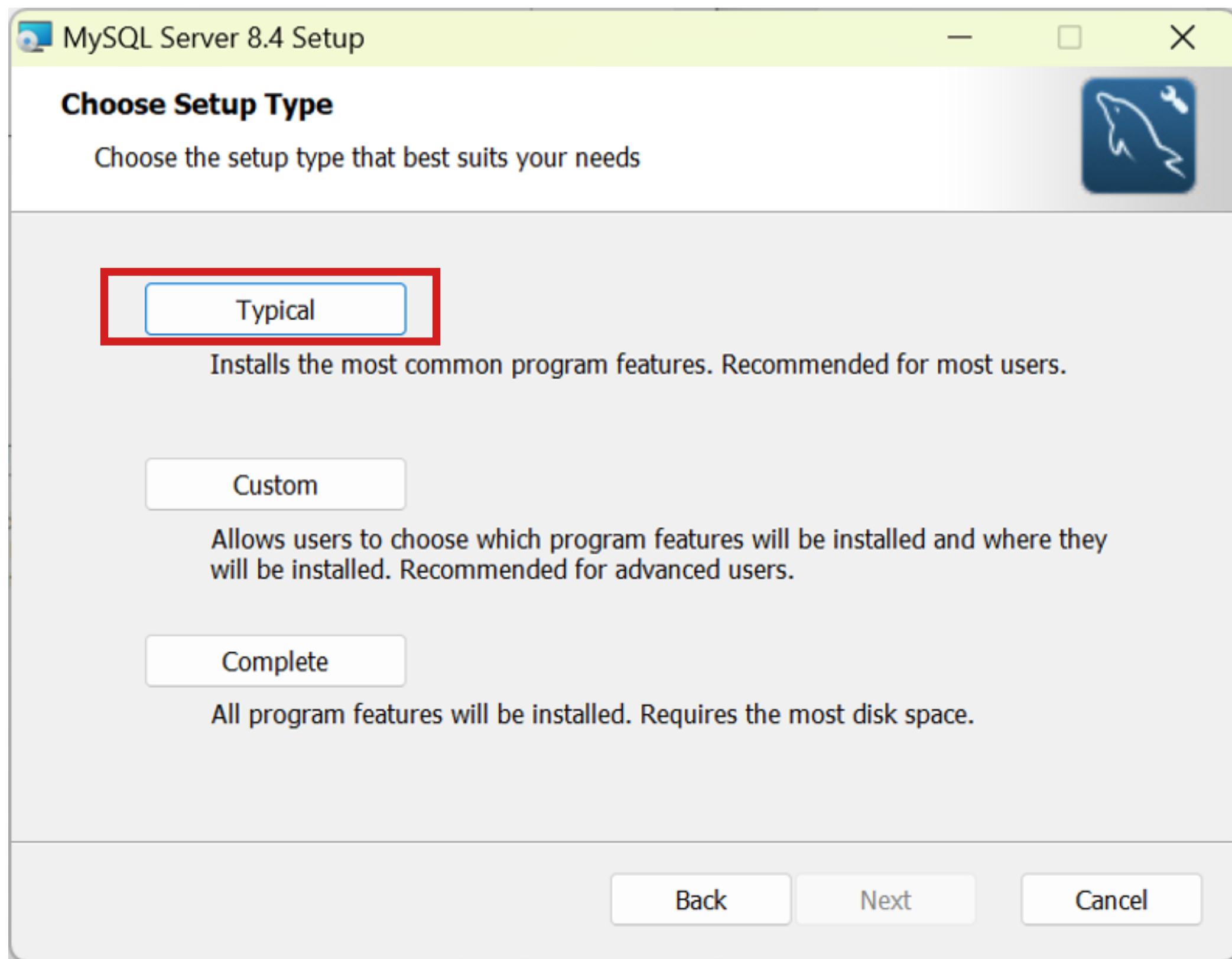
MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can signup for a free account by clicking the Sign Up link and following the instructions.

No thanks, just start my download.

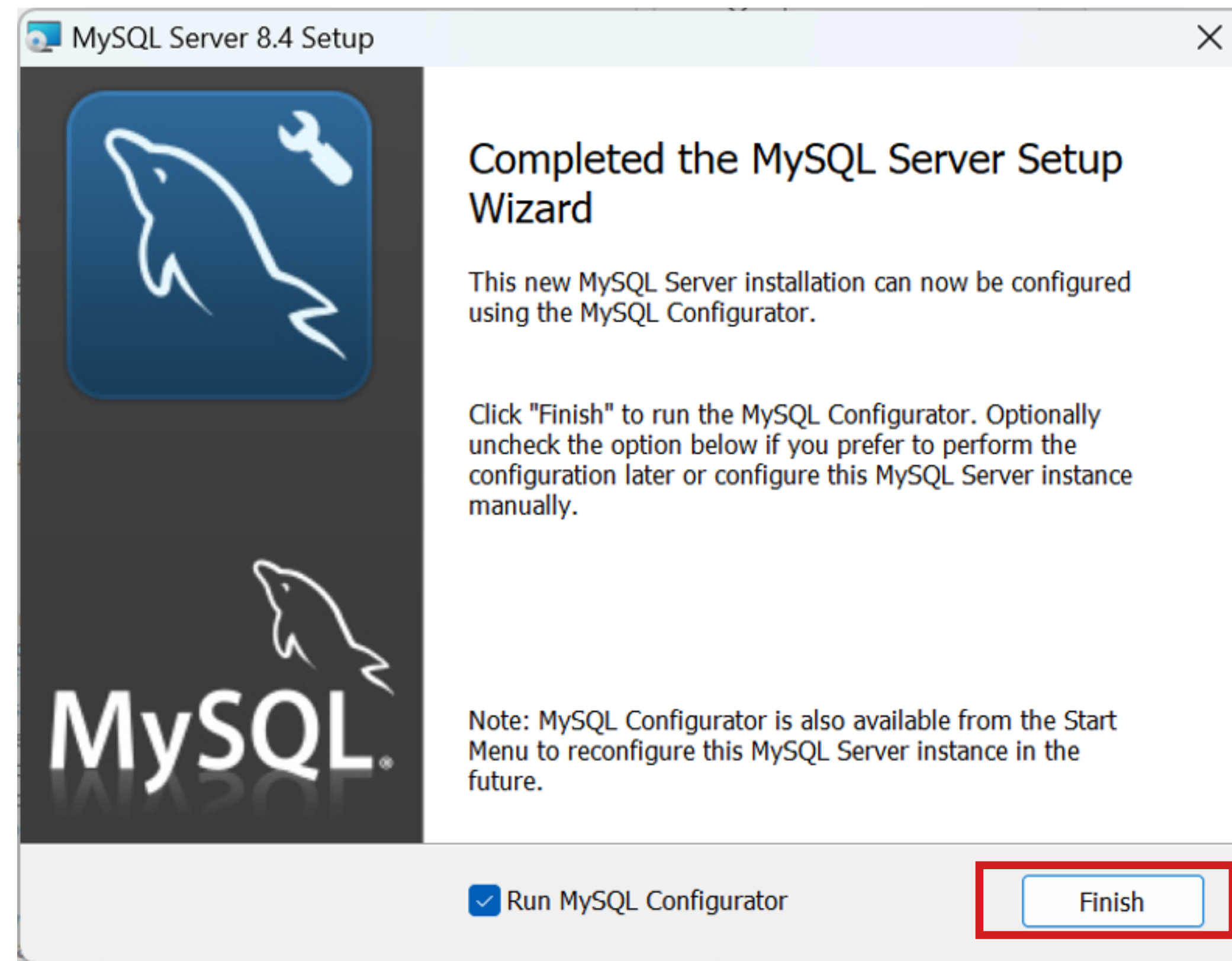
MySQL インストール



MySQL インストール

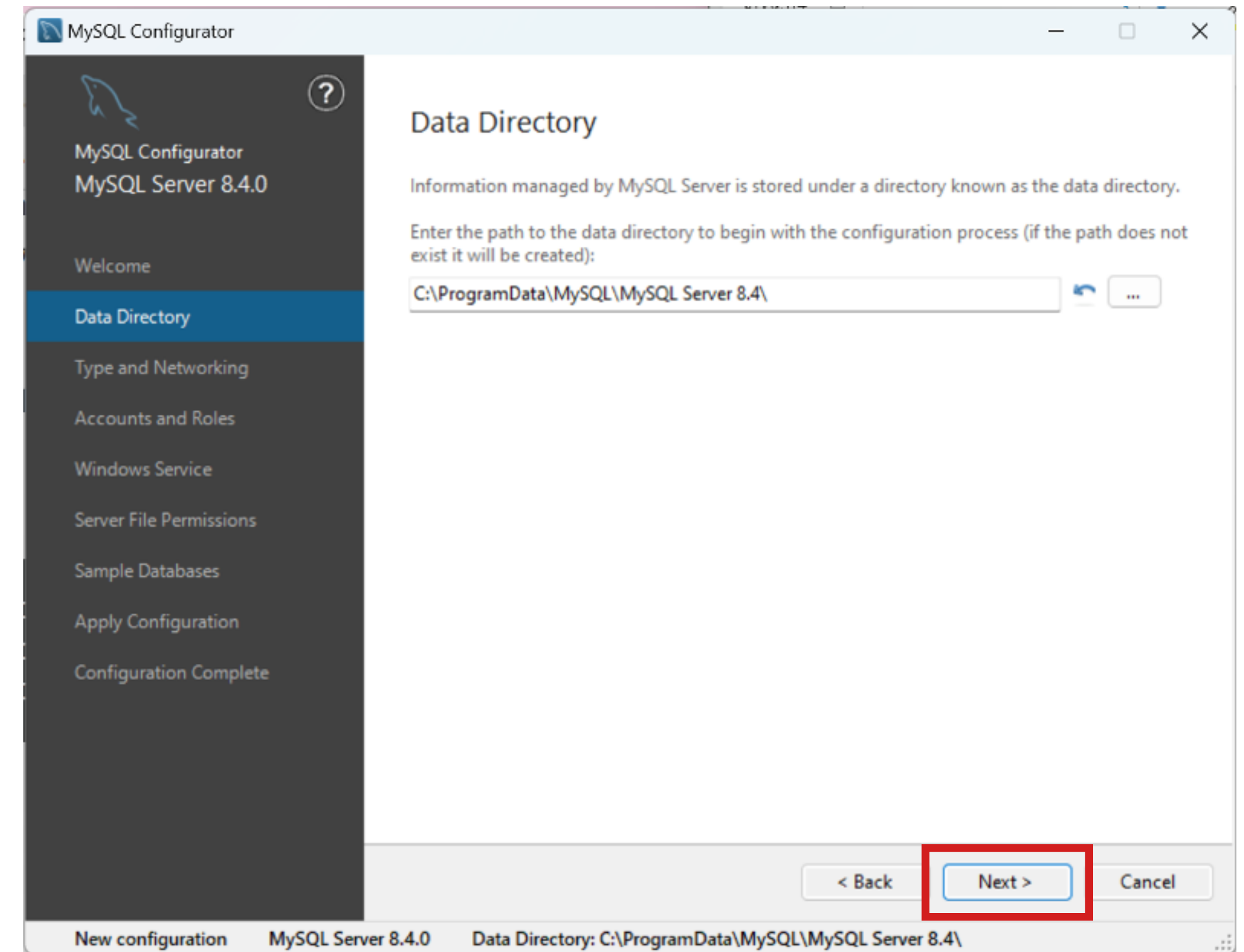
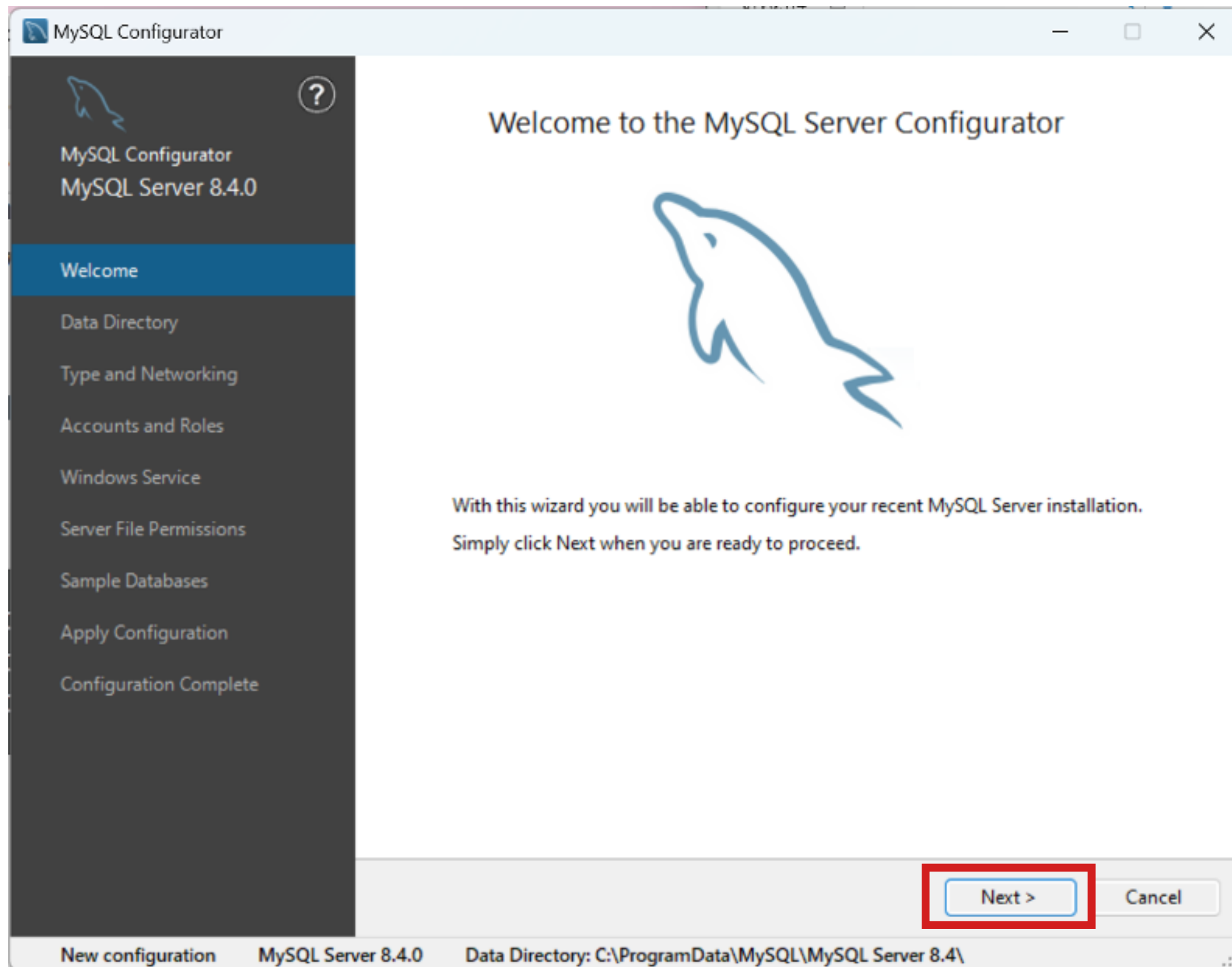


MySQL インストール



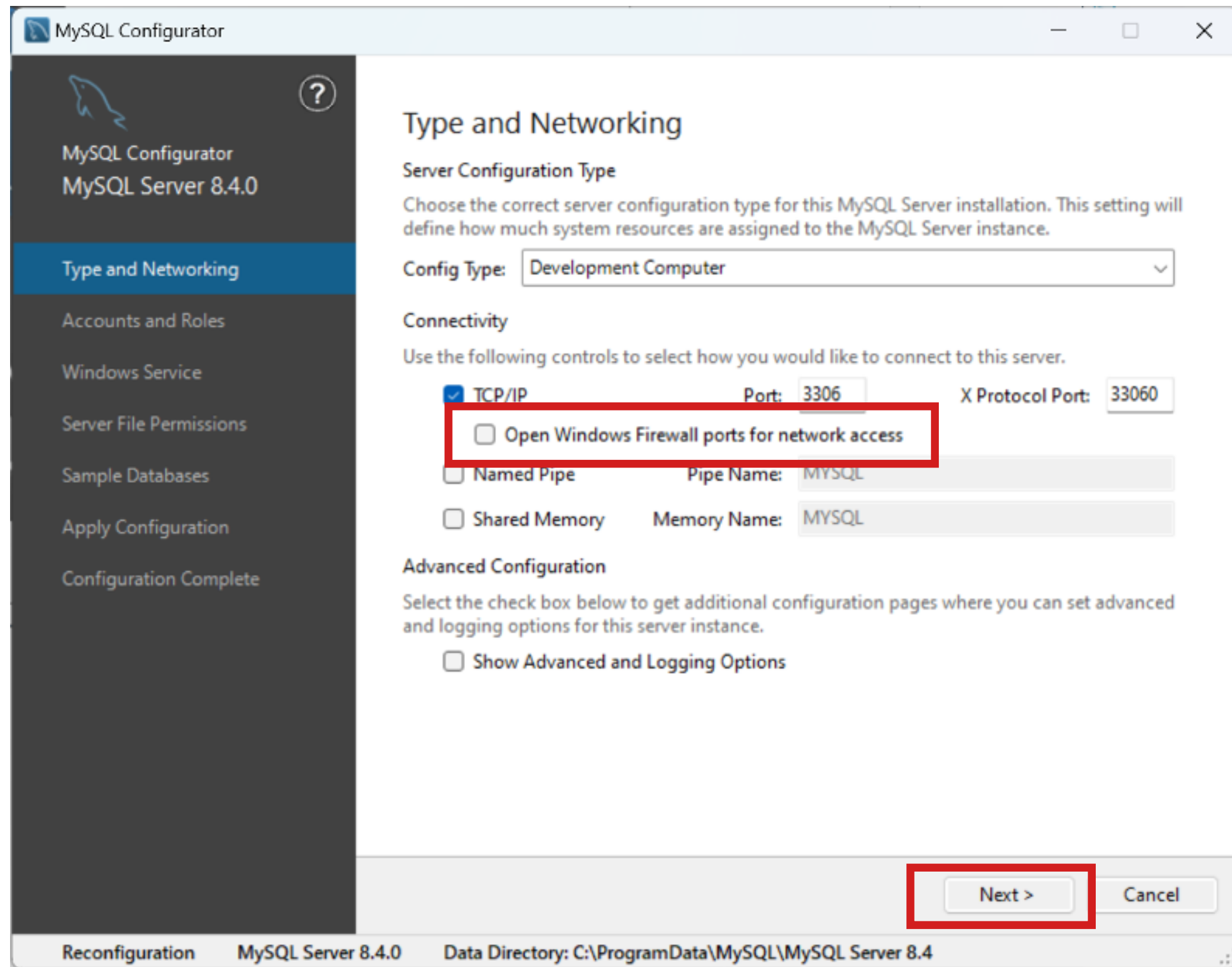
MySQL

インストール: MySQL Configurator

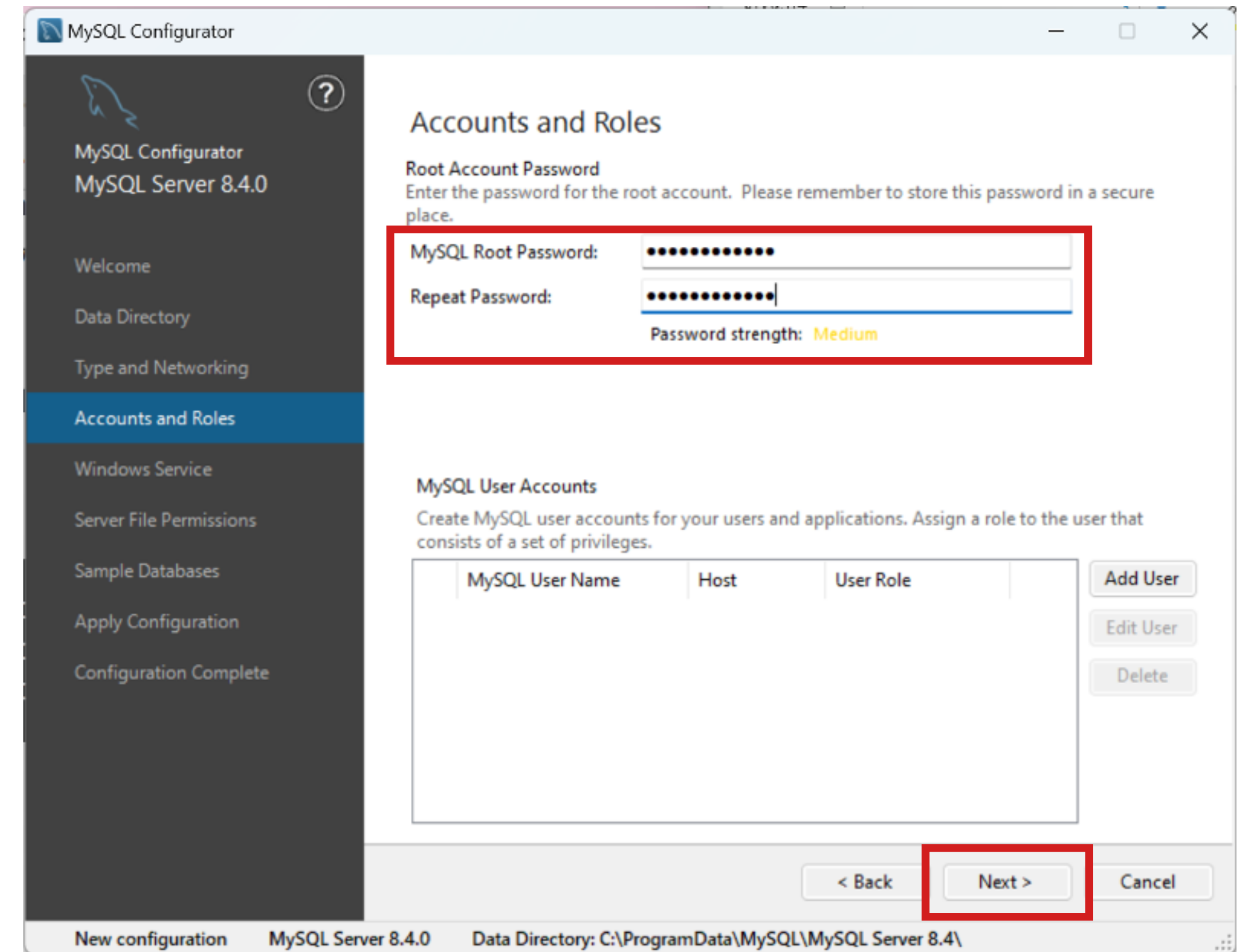


MySQL

インストール: MySQL Configurator



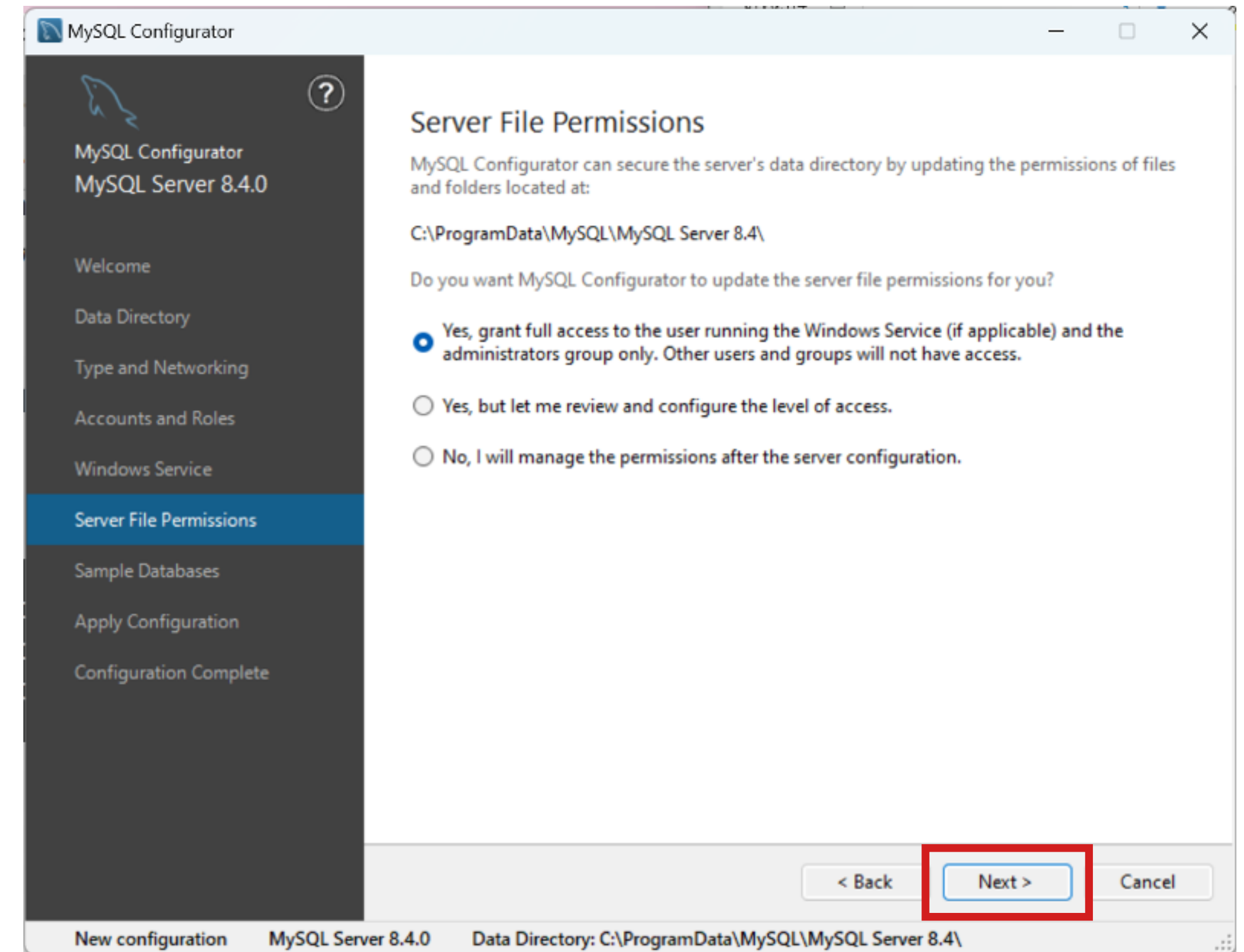
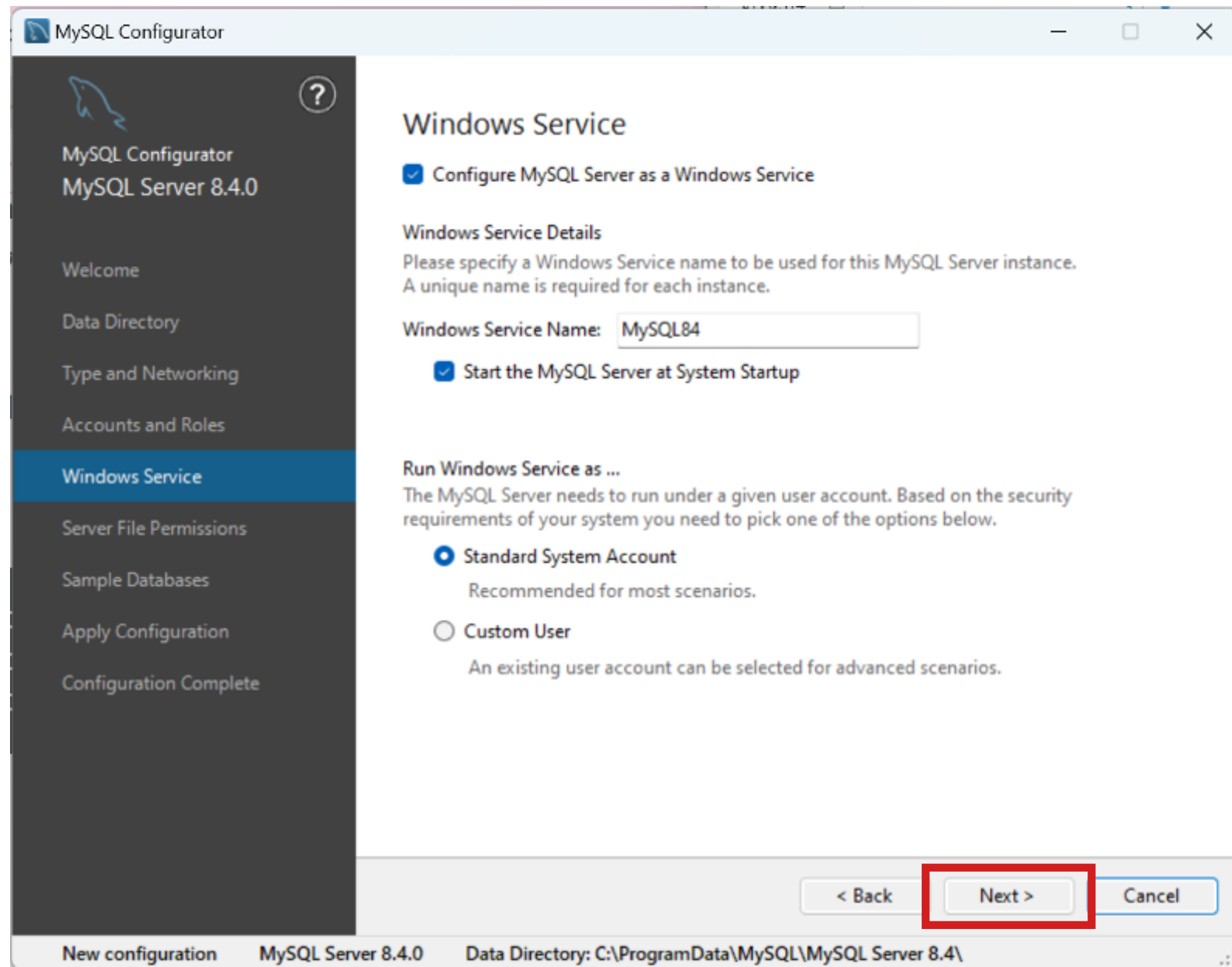
「Open Windows Firewall ports for network access」のチェックを外す



MySQL用のパスワードを決める

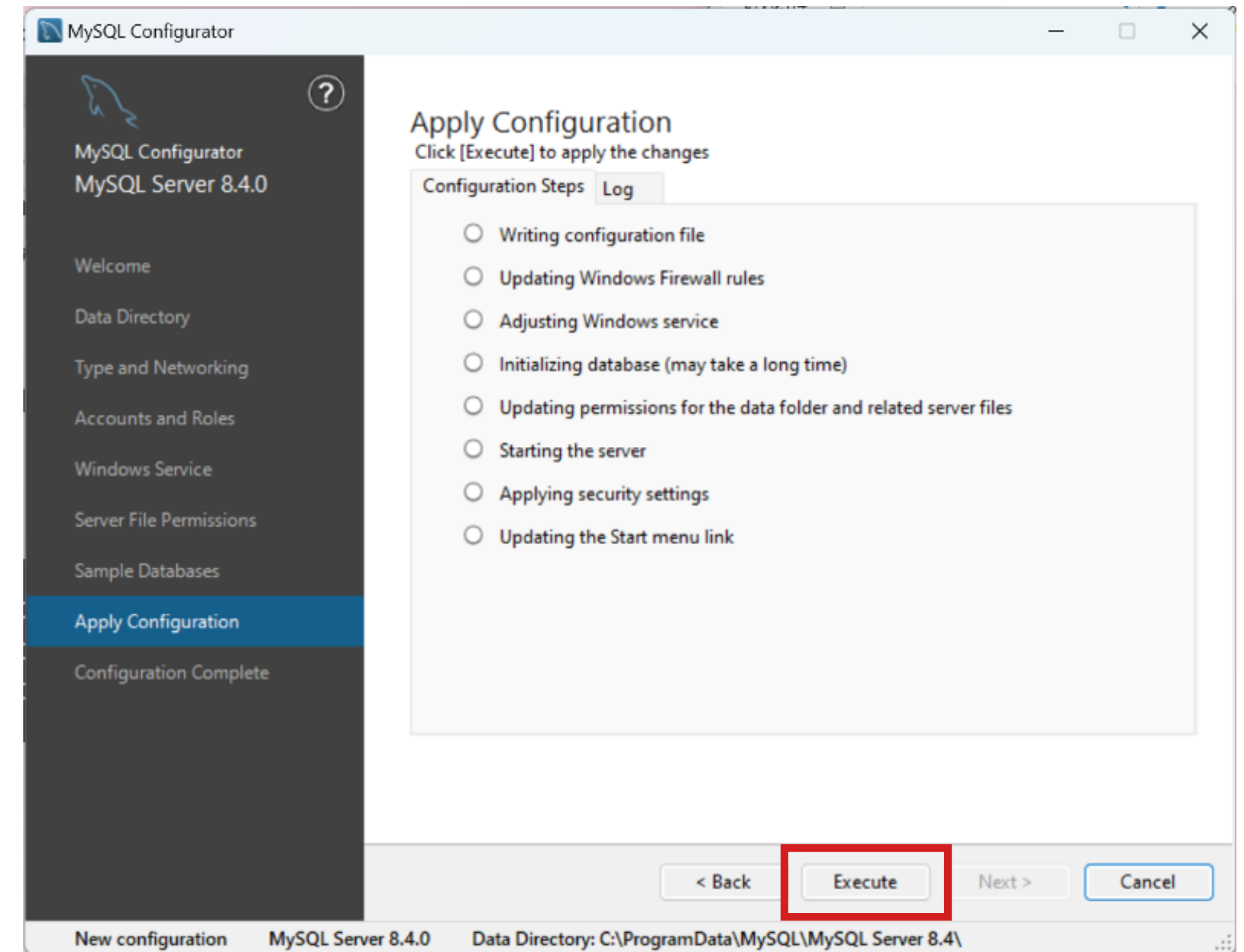
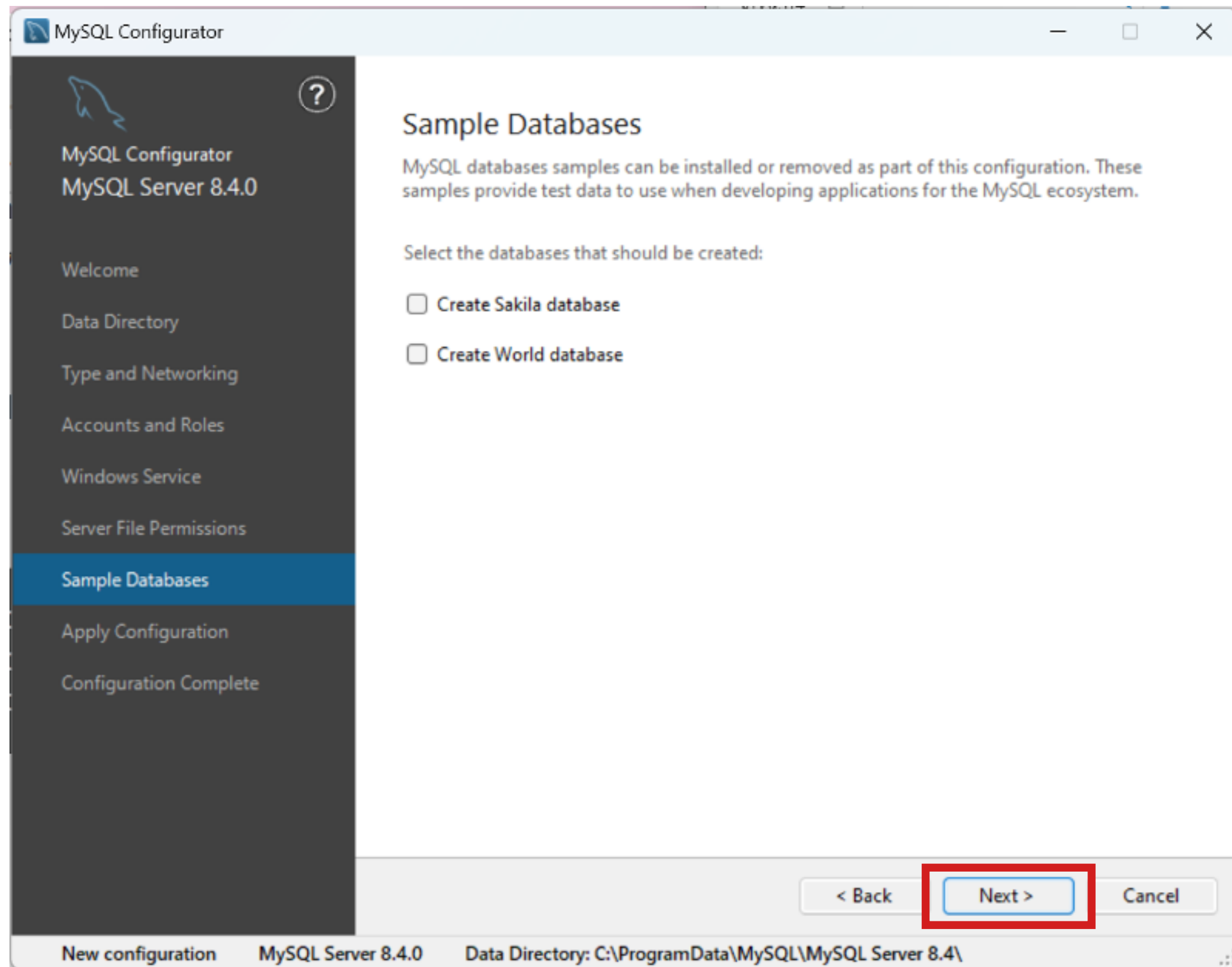
MySQL

インストール: MySQL Configurator



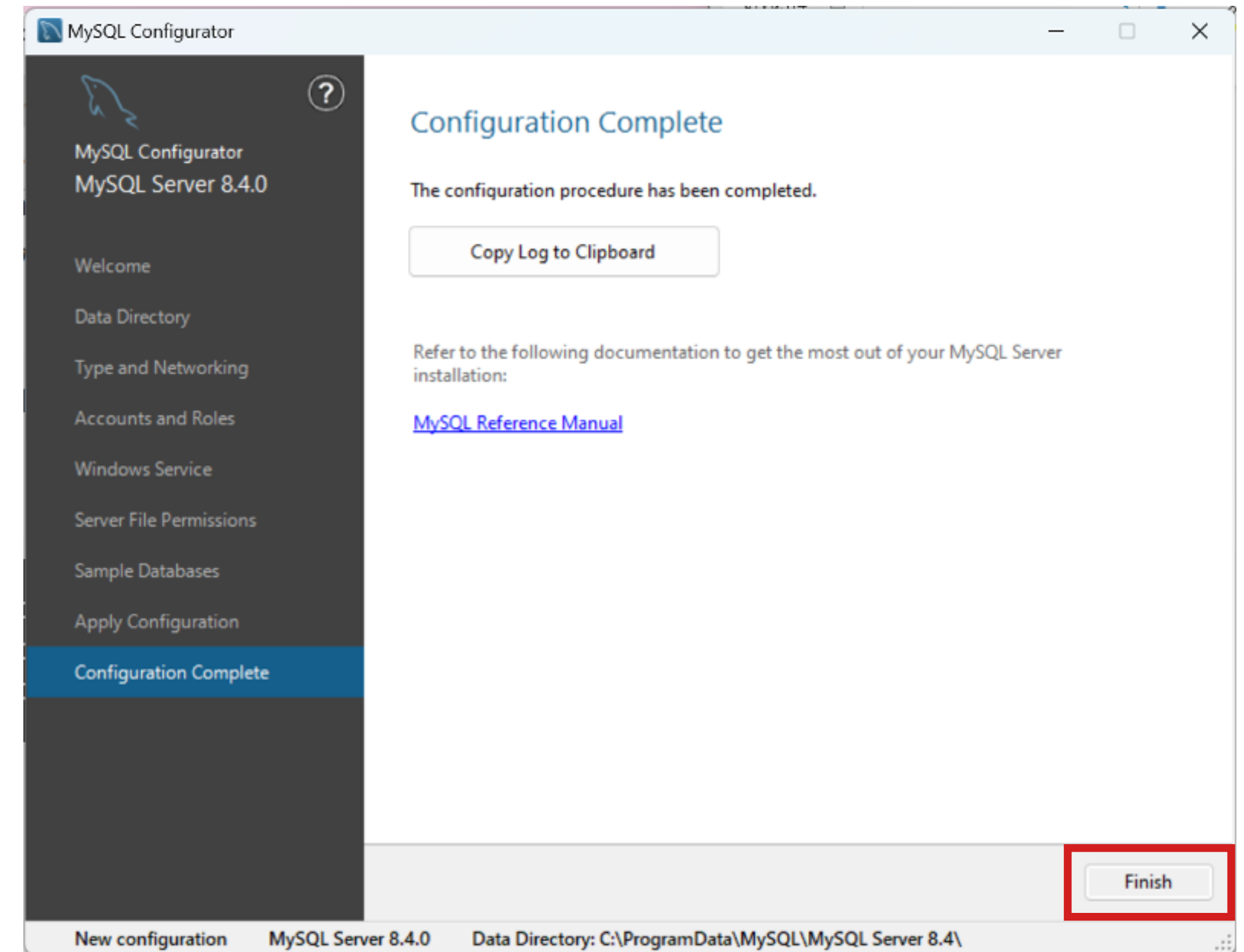
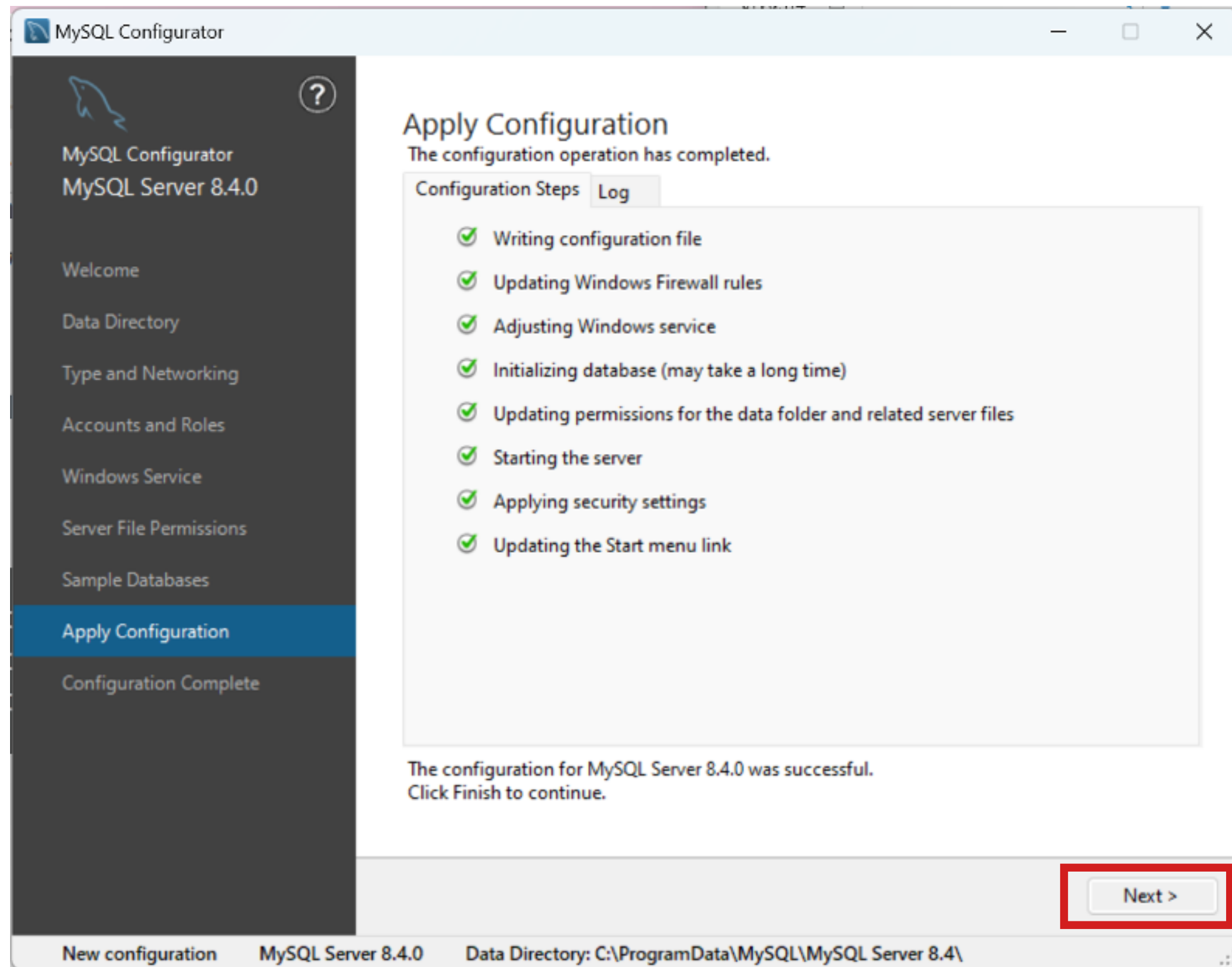
MySQL

インストール: MySQL Configurator



MySQL

インストール: MySQL Configurator



MySQL

起動確認

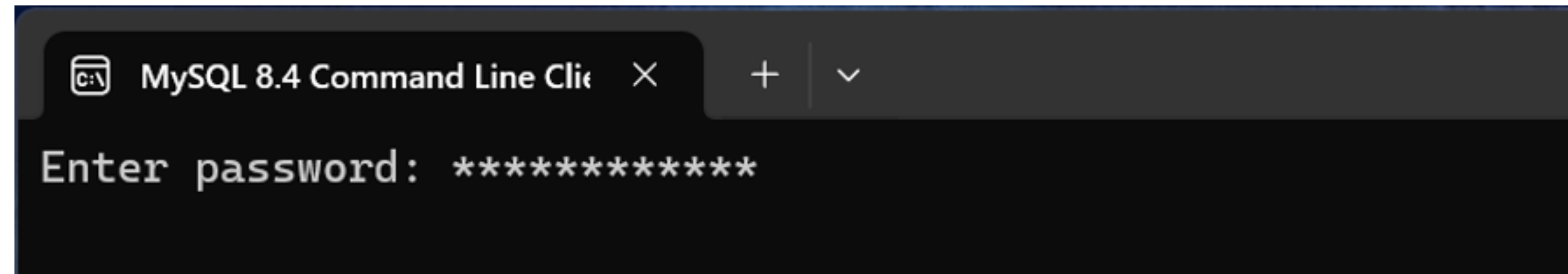
- Command Line Client Unicodeを起動する



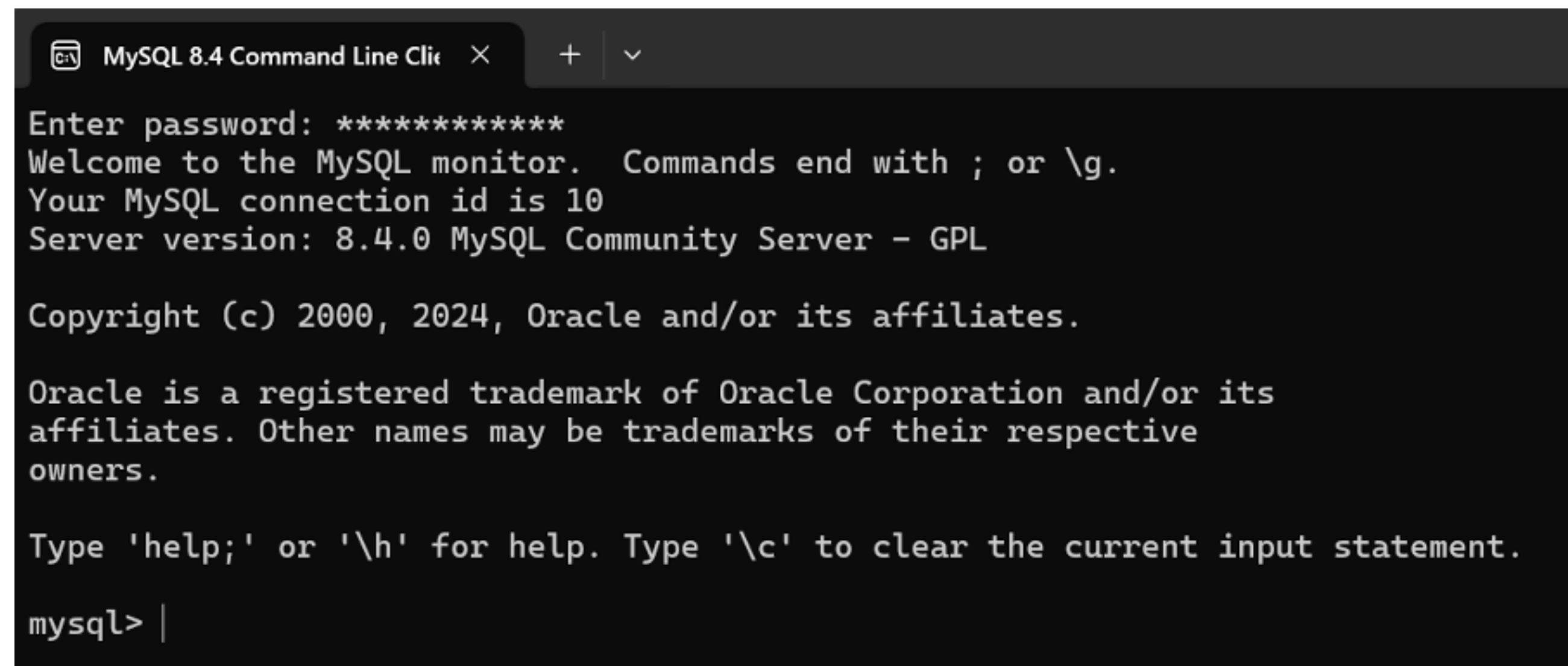
MySQL

起動確認

- ・ インストール時に設定したパスワードを入力する



```
MySQL 8.4 Command Line Client
Enter password: *****
```



```
MySQL 8.4 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.4.0 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

astah

ダウンロード

- 以下のアドレスをブラウザで開く

[https://astah.change-vision.com/
ja/download.html](https://astah.change-vision.com/ja/download.html)

astah ダウンロード

最新版をダウンロード

64bit Windows

(画像はMac版)

ライセンスをお持ちの方へ

ライセンスのご購入ありがとうございます。製品を選択してください。 [トライアルはこちらから →](#)

✓ [astah* professional](#)

[astah* UML](#)

[astah* System Safety](#)

[astah* SysML](#)

[astah* think!](#)

astah professional

ダウンロード

最新版をダウンロード
Mac

[OS別インストーラー一覧 →](#)

[システム要件 →](#)

[最新版リリースノート →](#)

[旧バージョンのダウンロード →](#)

ライセンス登録・設定

購入済みのライセンスを登録・設定します。

ライセンス登録ガイド

astah* Viewer

閲覧専用ツール [astah* Viewer](#) を
ダウンロード(無料) →

ラーニング・ヘルプ

[スタートガイド →](#)

[機能ガイド →](#)

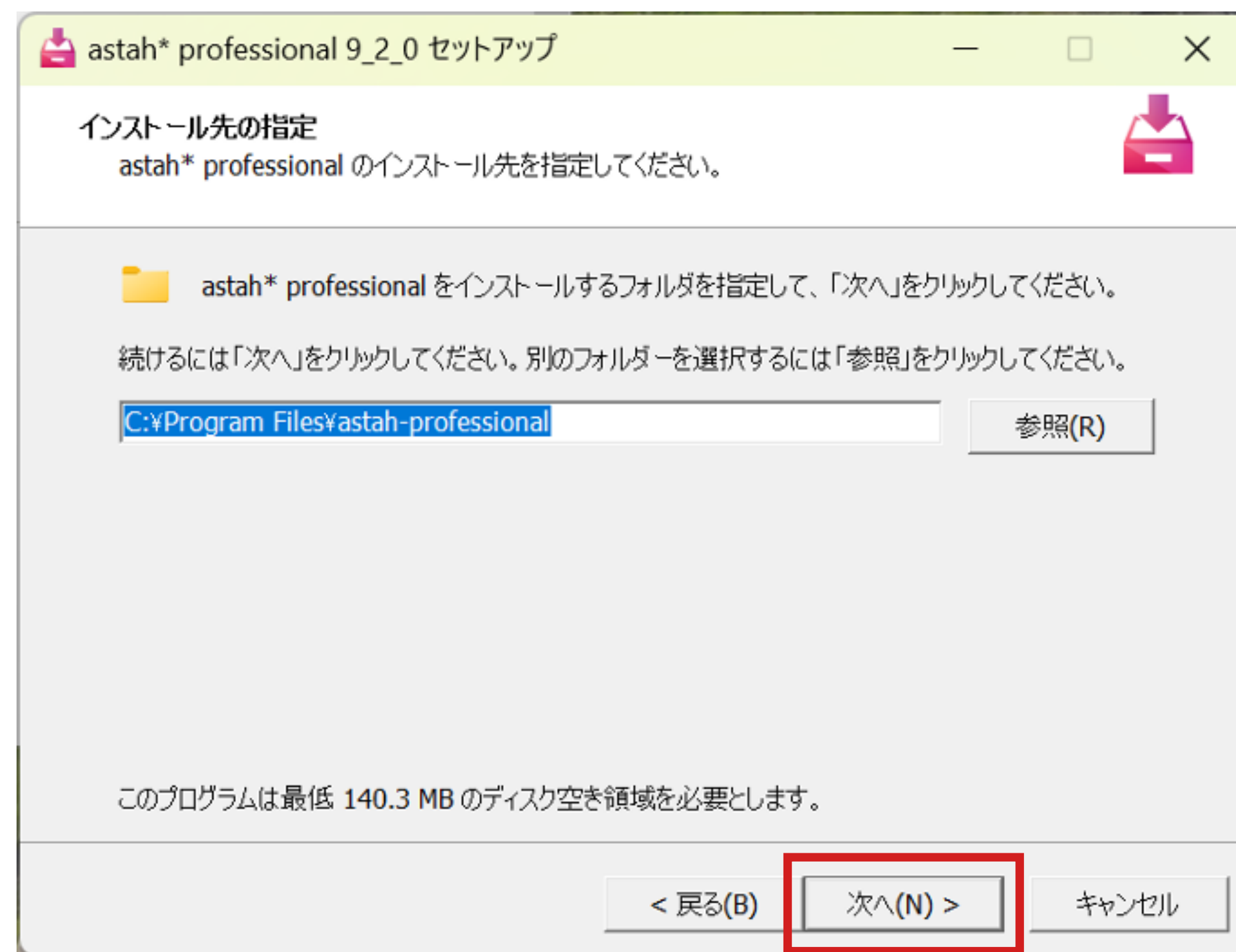
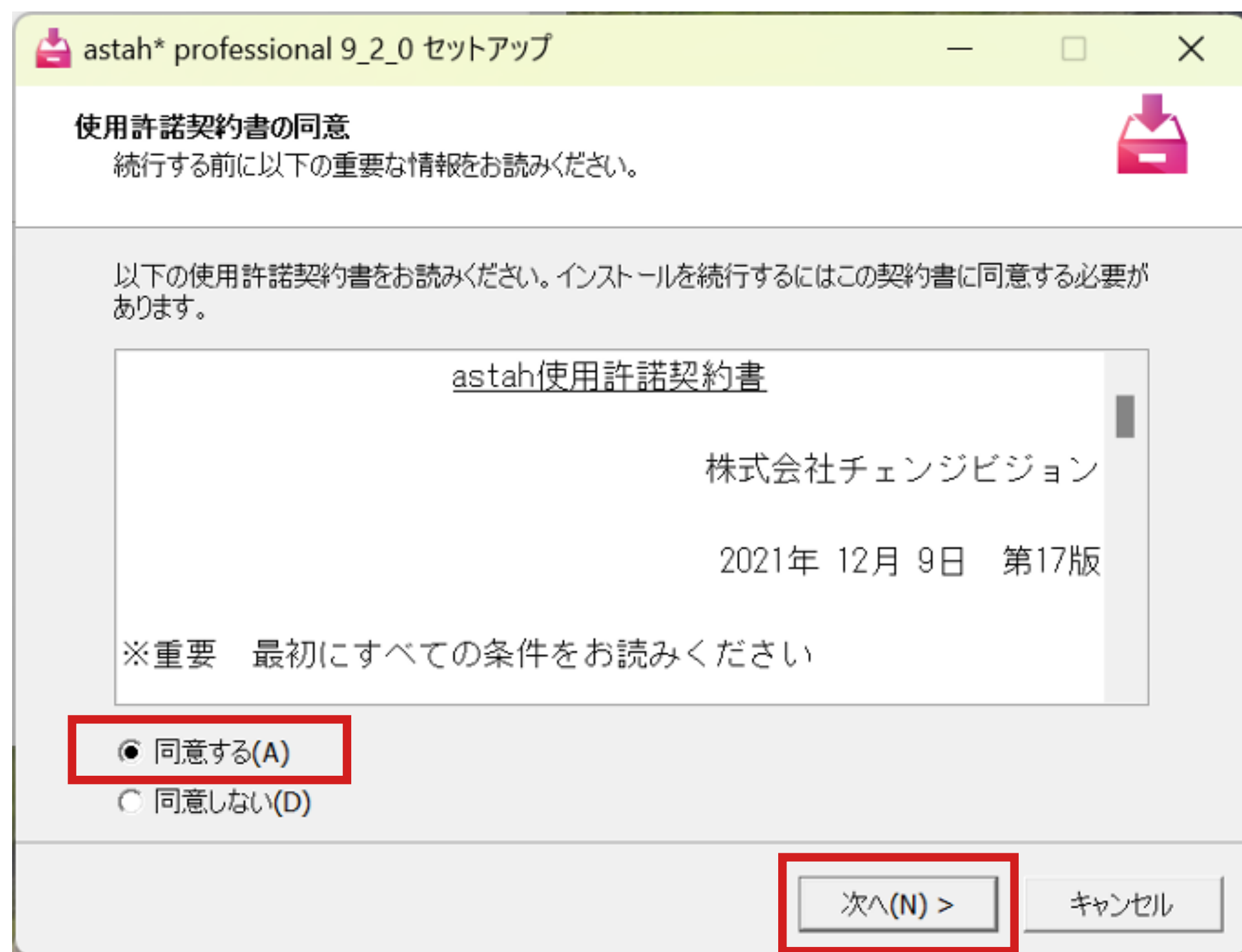
[チュートリアル →](#)

[キーボードショートカット →](#)

[よくあるご質問 →](#)

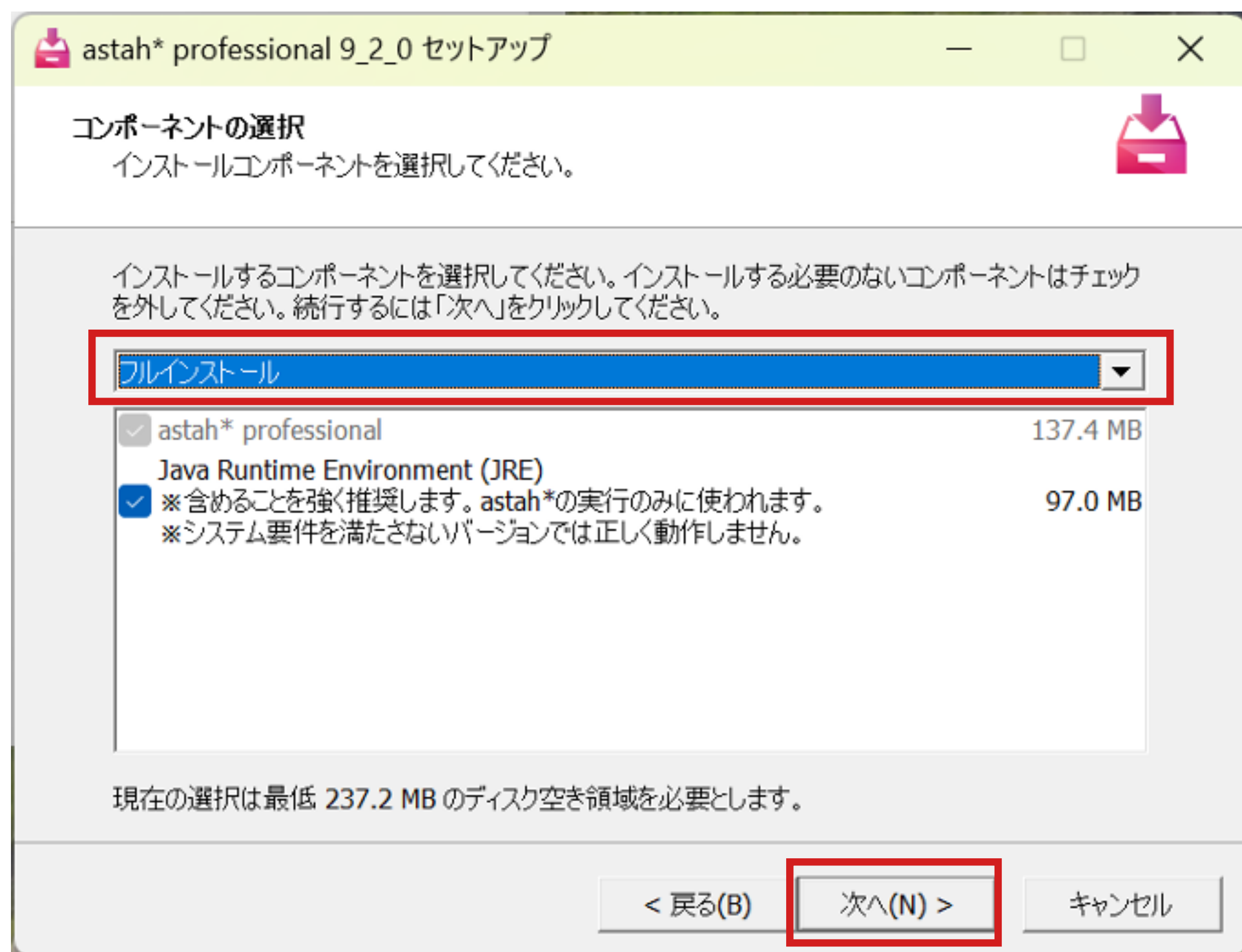
astah インストール

- ・ダウンロードしたexeファイルを実行
 - ・ astah-professional-9_2_0-0248cd-jre-64bit-setup.exe
みたいな名前

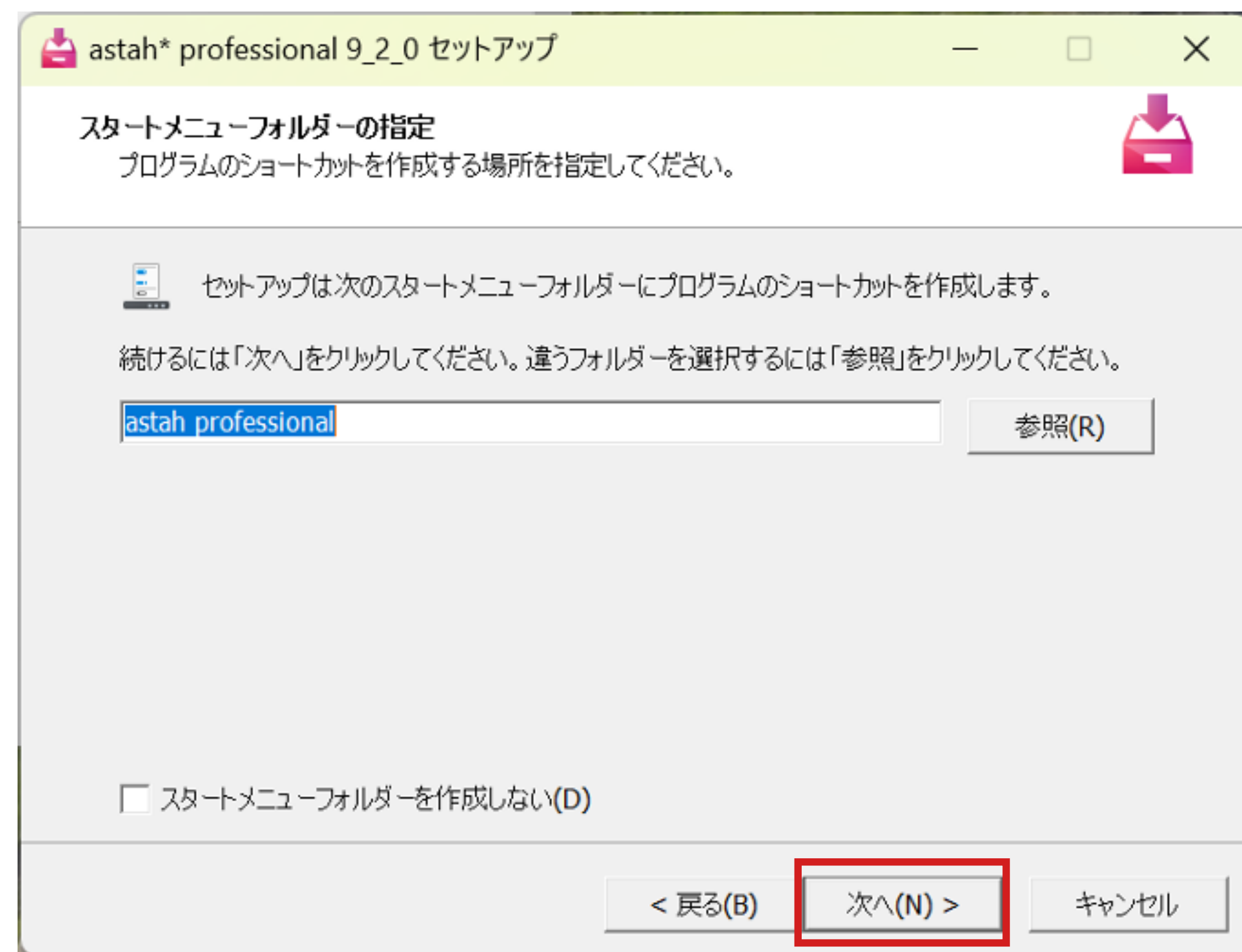


astah

インストール



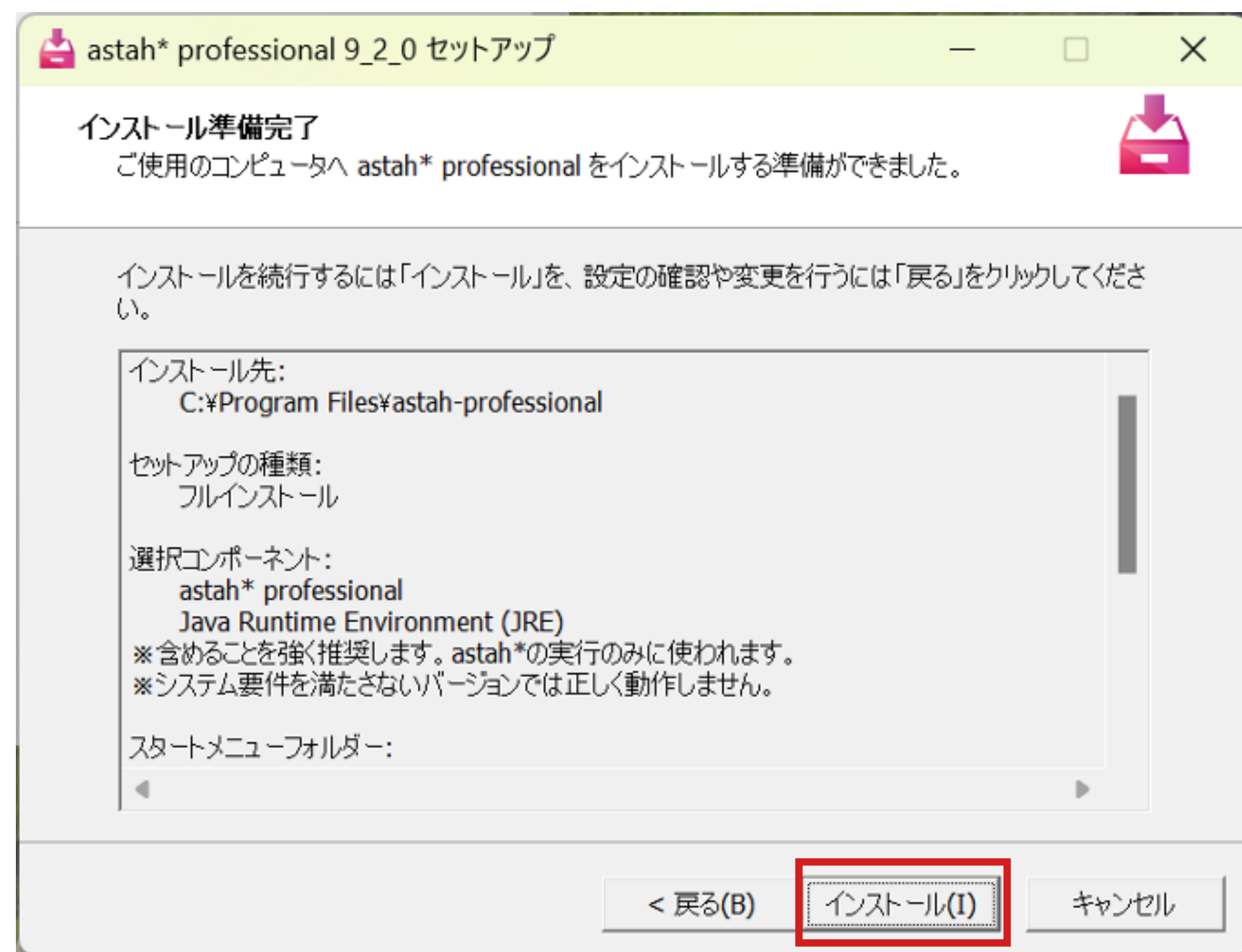
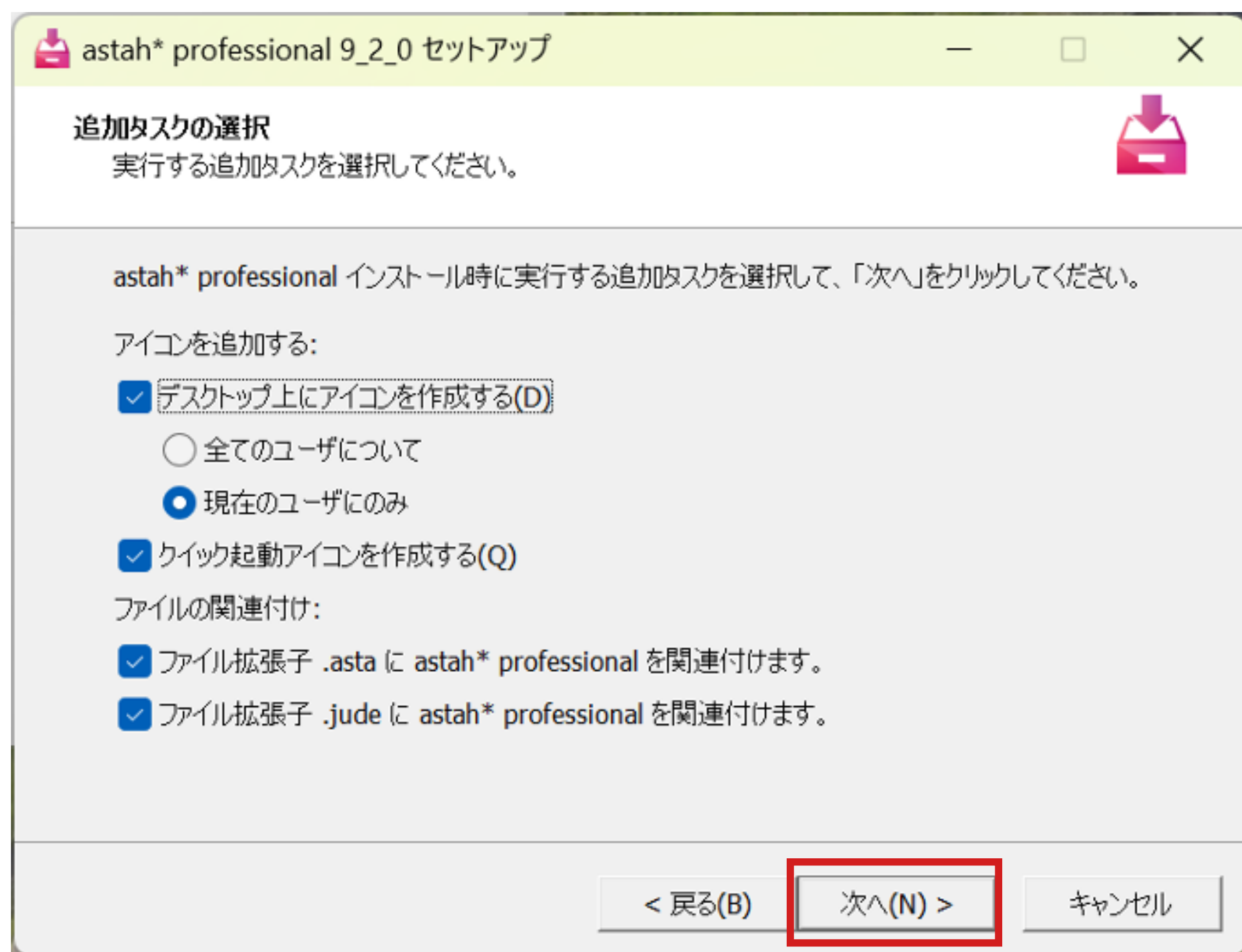
フルインストールを選択



ショートカットを作るかどうか (任意)

astah

インストール



ショートカットを作るかどうか (任意)

astah

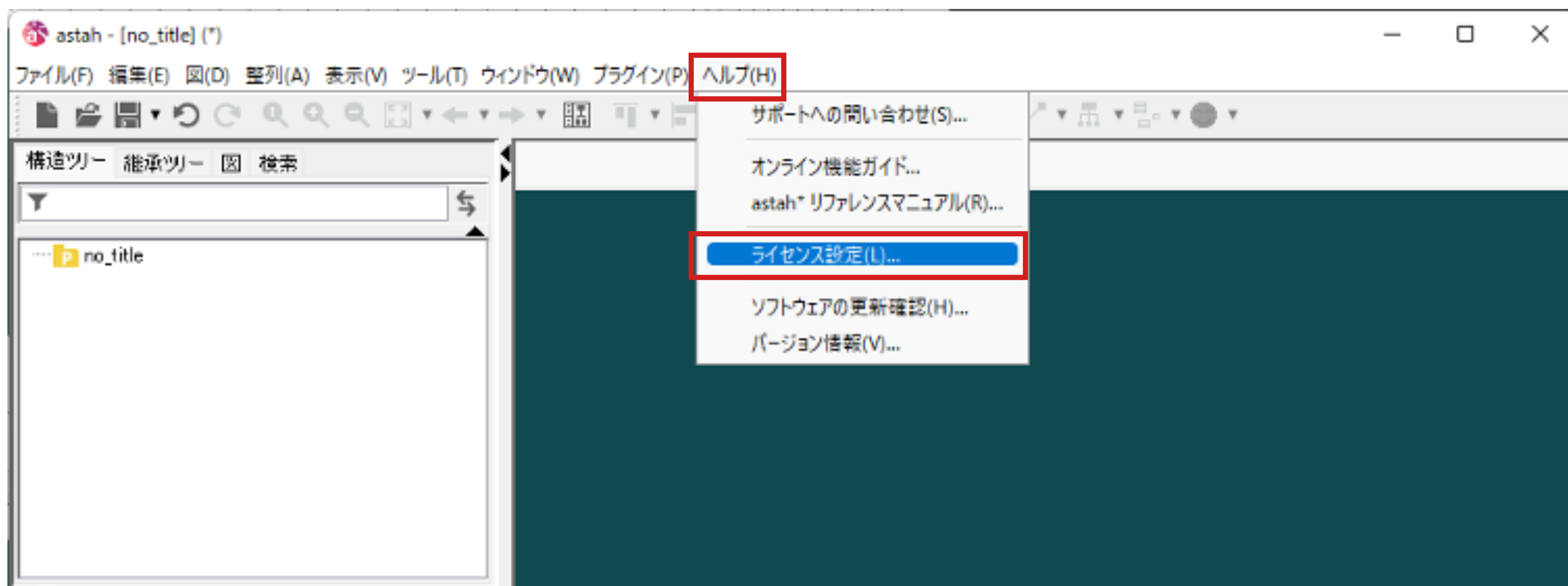
インストール



astah

ライセンス認証

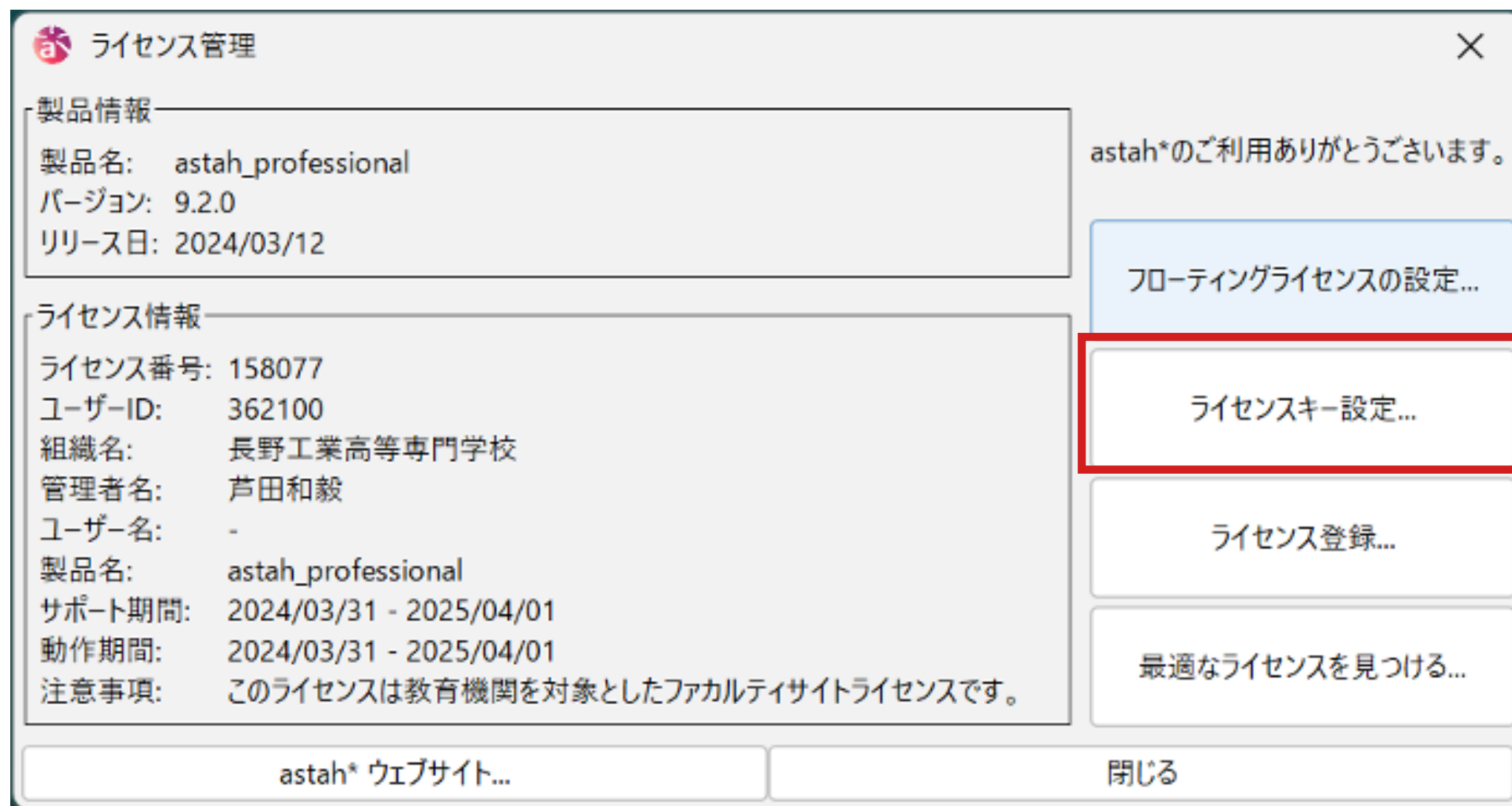
- astahを管理者として実行
- ヘルプ → ライセンス設定



astah

ライセンス認証

- 下のような画面が出れば認証済み
- 出なければ、ライセンスキー設定をクリック
- Teams上からダウンロードした「astah_faculty_license.xml」を選択



2章

データベース 概要

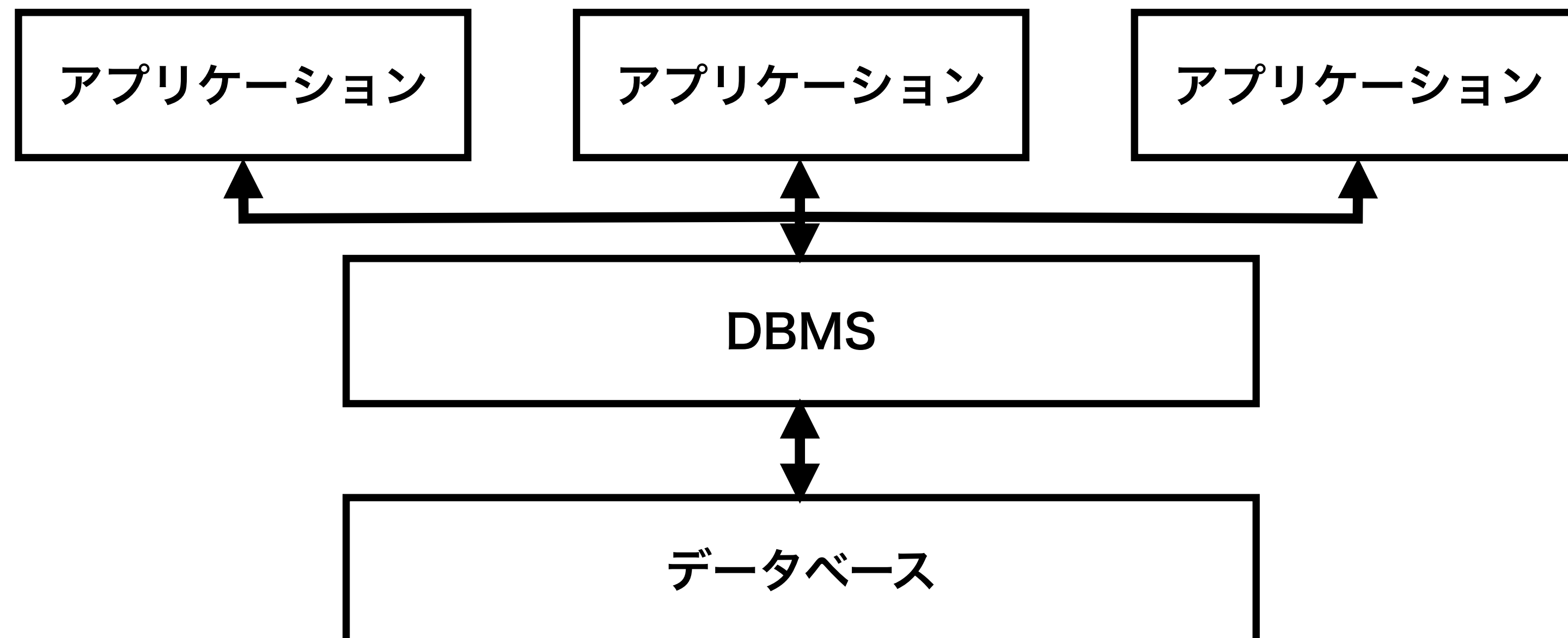
- ・ 一般に「データが整理されて保管されているもの」
- ・ 分析、活用、抽出するための「システム」



データベース

DBMS: データベースマネジメントシステム

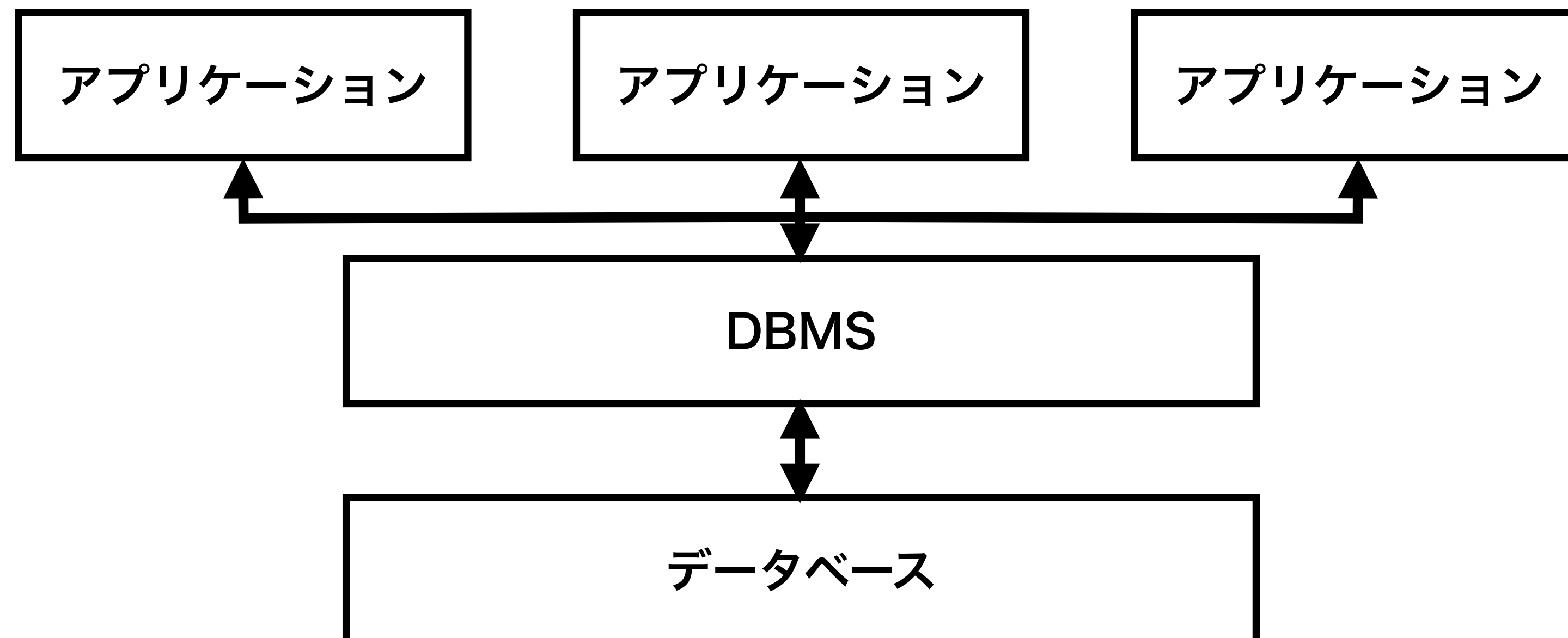
- コンピュータシステムにおけるデータベースはDBMSを指すことが多い
- 格納されたデータから、任意の条件に当てはまるものを抽出、並べ替え
- 必要なデータを取り出して提供するのが役割
- DBMSを介してデータにアクセス



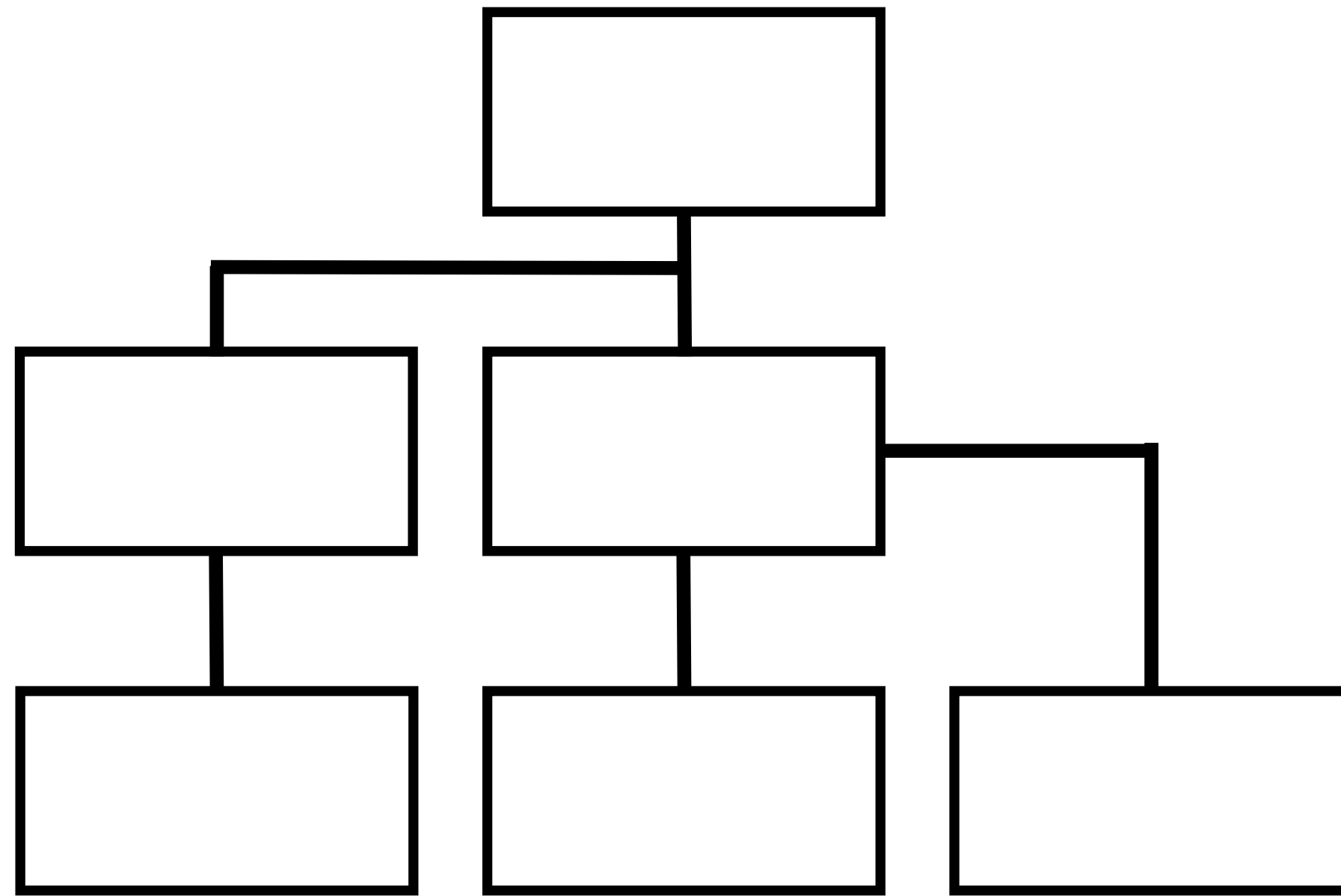
データベース

DBMS: データベースマネジメントシステム

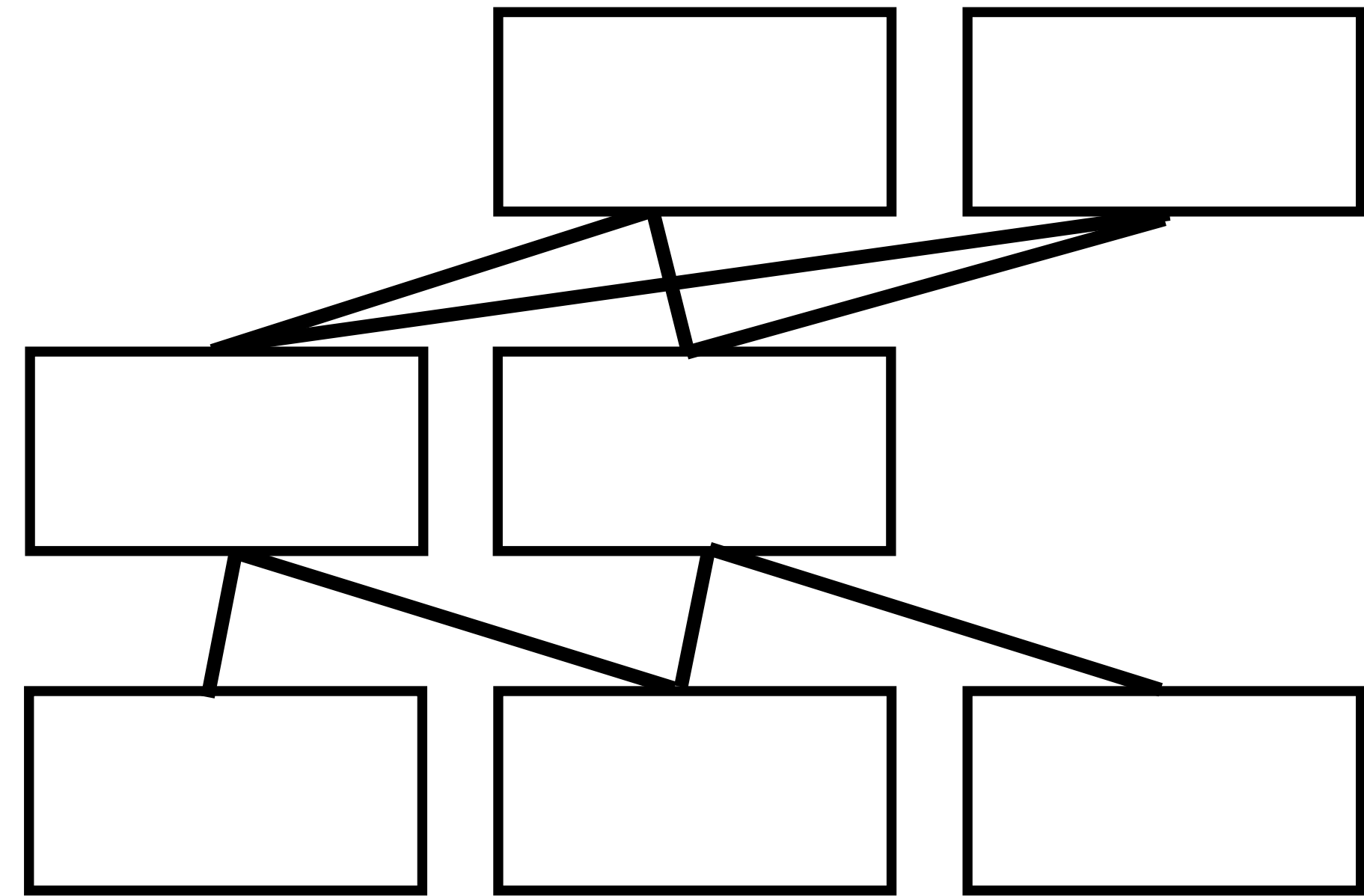
- 非常に堅牢である必要がある
 - 銀行のシステムとか
- Oracle Database、IBM DB2、MySQLなど



データベース 種類



階層型データベース

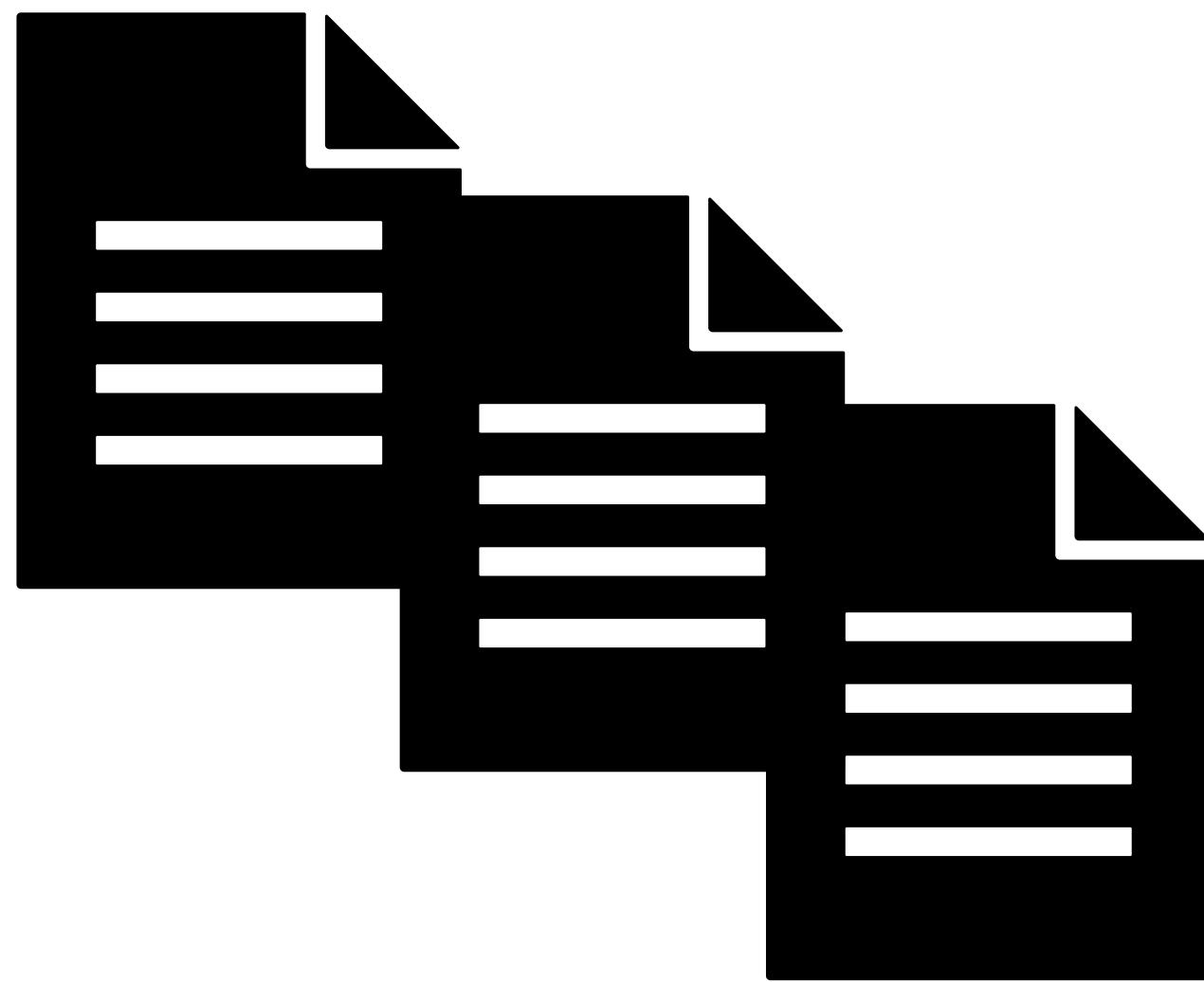


ネットワーク型データベース

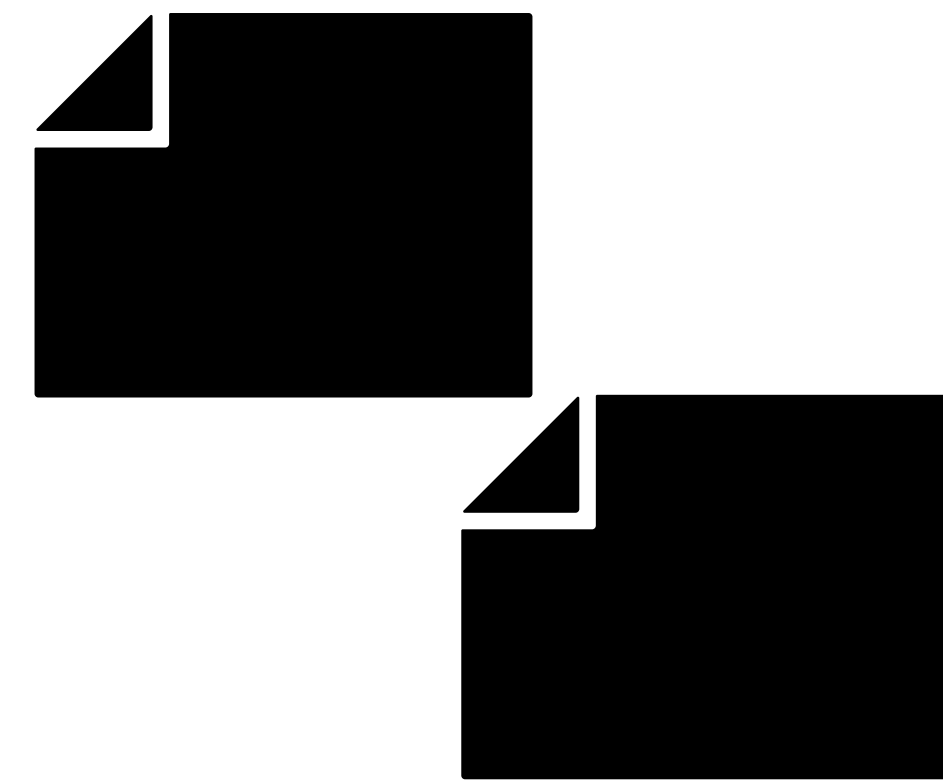
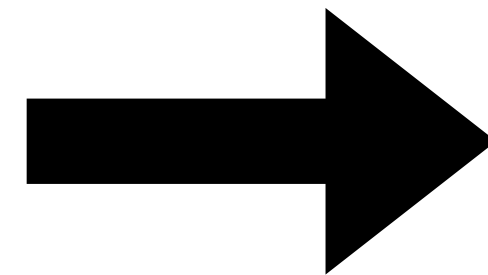
データベース

種類

- ドキュメント型
- NoSQL (Not only SQL): リレーショナルではないデータベース
- 入れ替わりが頻繁なデータを扱うので、定義を厳密にせず、柔軟に処理できる



JSON、XMLなどのデータドキュメント

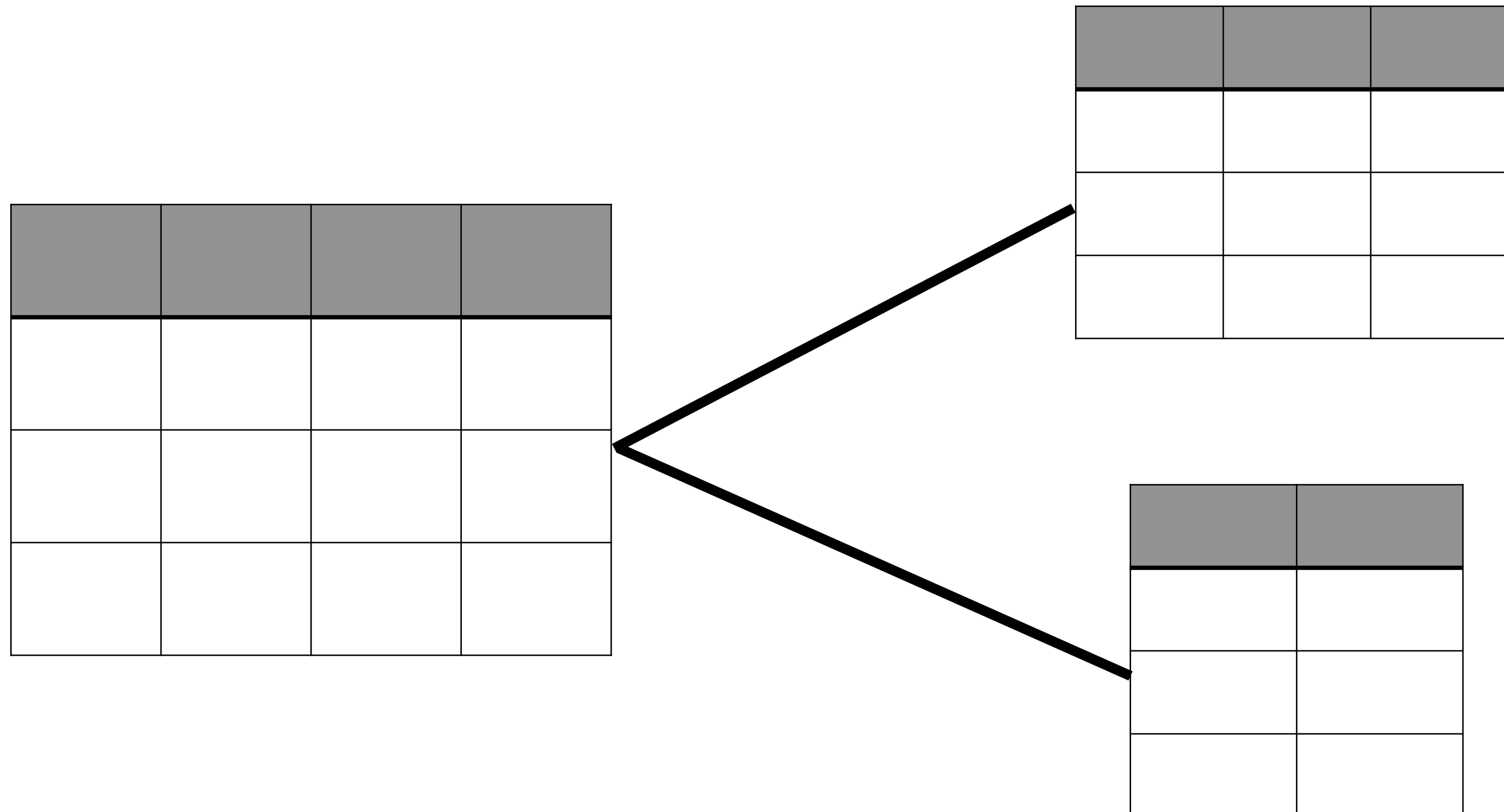


所望の抽出データドキュメント

データベース

種類

- ・ リレーショナルデータベース
- ・ 表のように定義され、各テーブルが相互に関係（リレーション）



データベース

リレーショナルデータベース: 基本機能

- 格納: たくさんのデータを格納できる
- 検索: 多くのデータから目的のデータを高速に検索できる
- 操作: データの追加、削除、更新ができる
- 定義: データ構造を定義し、関係づけができる

データベース

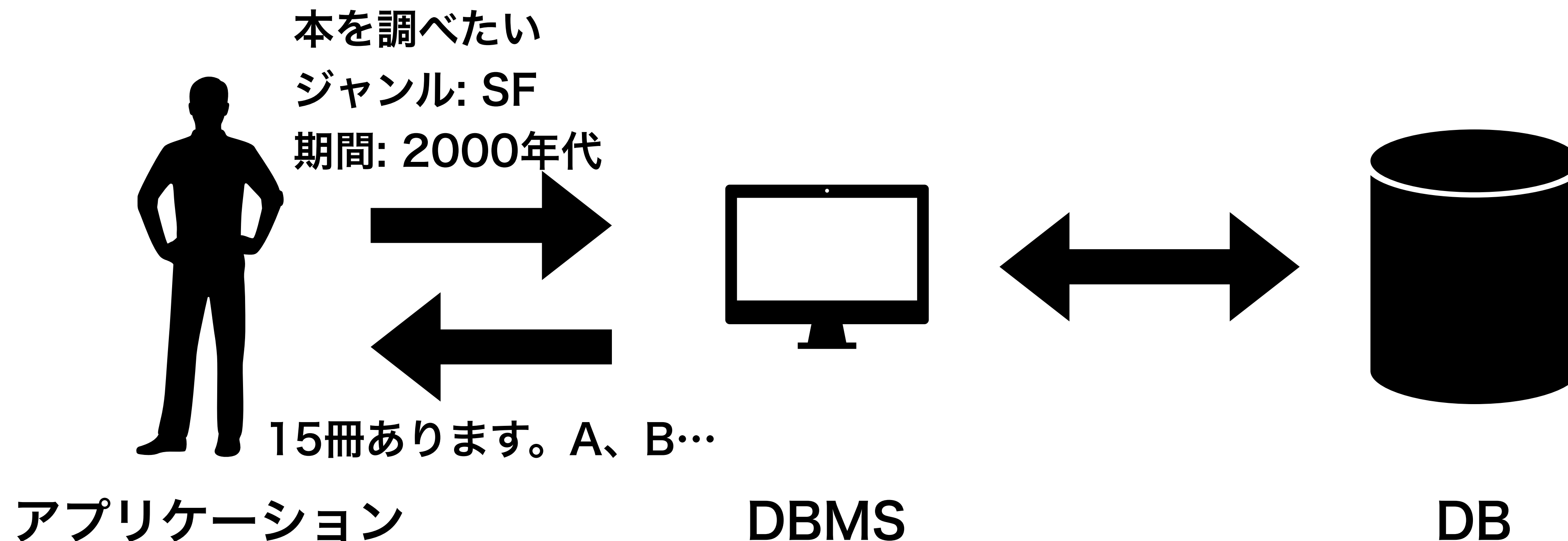
リレーショナルデータベース: 信頼性のための機能

- 同時制御: 複数ユーザがアクセスしたとき、不整合がでない
- トランザクション処理: データベースの一貫性を保つ
- 障害回復: バックアップ・リストア、ロールバック、ロールフォワード

SQL

Structured Query Language

- ・ リレーショナルデータベースへの問い合わせに使用
- ・ 所望のデータを取り出すために、問い合わせ (Query) をする言語



SQL

Structured Query Language

- ・ リレーショナルデータベースへの問い合わせに使用
- ・ 所望のデータを取り出すために、問い合わせ (Query) をする言語

```
SELECT 書名, 発行年 FROM 図書データ  
WHERE ジャンル=SF AND 発行年 BETWEEN 2000 AND 2025
```

書名	発行年
虐殺器官	2007
ハーモニー	2008
プロジェクト・ヘイル・メアリー	2021

MySQL

概要

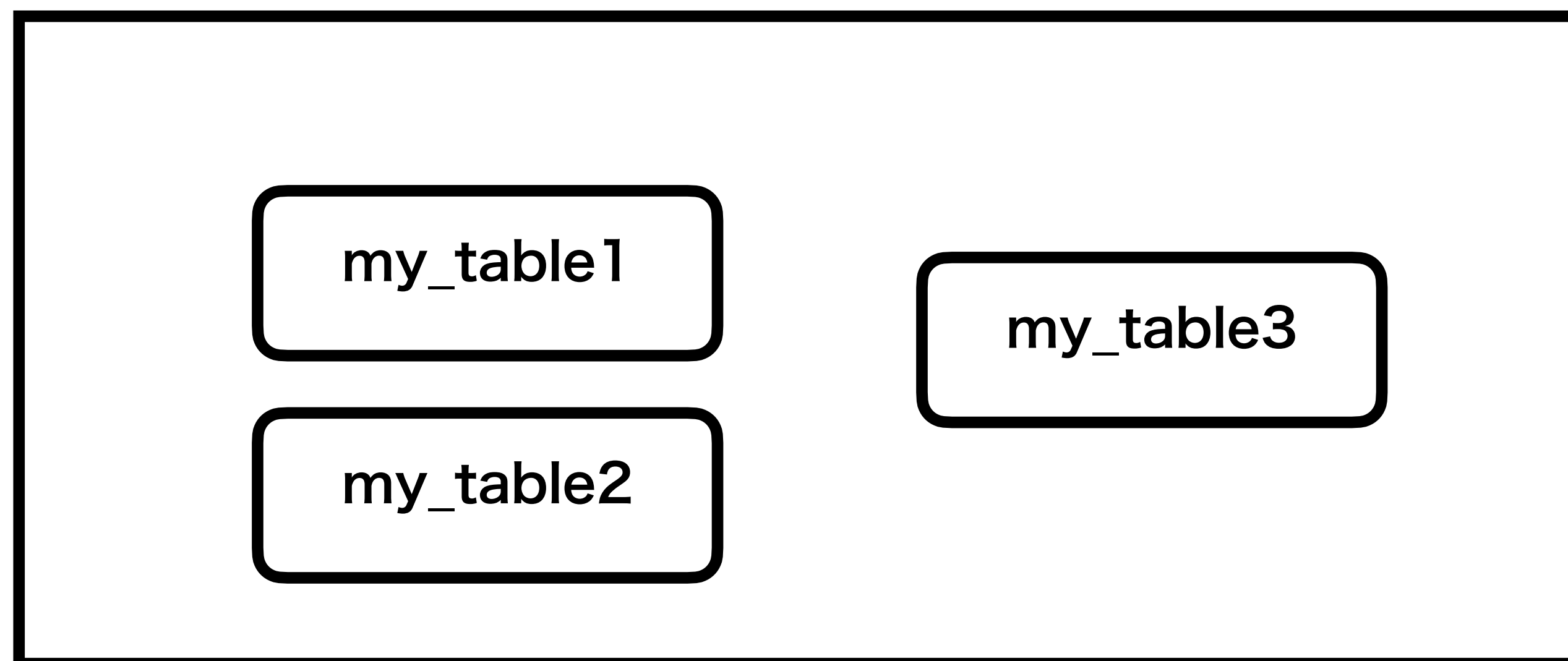
- DBMSの1つ
- 多様なプラットフォームで利用可能
- GPL (GNU General Public License) とコマーシャルライセンス

データベースの構造

データベース

- データを格納する各種テーブルを含む大きな枠
- 格納するテーブルをまとめるフォルダのようなもの
- 複数テーブル持つことができる

my_database



データベースの構造

データベース

- ストレージエンジン: テーブルへのデータ読み書きプログラム
- InnoDB: 信頼性が高い、MySQLの標準
- MyISAM: 検索が速い

テーブル型	トランザクション	外部キー	BLOB/TEXT
InnoDB	○	○	○
MyISAM	×	×	○

データベースの構造

テーブル

フィールド・カラム (データ型を定義)

学籍番号	名前	コース	科目	点数
22421	田中 太郎	情報コース	国語	80
22310	中村 花子	電気コース	物理	95

レコード

データベースの構造

テーブル: 数値型

型	ストレージ (バイト)	最小値 (符号付き)	最大値 (符号付き)
		最小値 (符号なし)	最大値 (符号なし)
TINYINT	1	-128	127
		0	255
SMALLINT	2	-32768	32767
		0	65535
MEDIUMINT	3	-8388608	8388607
		0	16777215
INT	4	-2147483648	2147483647
		0	4294967295
BIGINT	8	-9223372036854775808	9223372036854775807
		0	18446744073709551615

データベースの構造

テーブル: 日付型

型	内容	型 / 範囲
DATE	日付を含む 時間は含まない	YYYY-MM-DD' '1000-01-01'~'9999-12-31'
DATETIME	日付と時間の両方を含む	YYYY-MM-DD HH:MM:SS' '1000-01-01 00:00:00' ~'9999-12-31 23:59:59'
TIMESTAMP	日付と時間の両方を含む	1970-01-01 00:00:01' UTC ~'2038-01-19 03:14:07' UTC

データベースの構造

テーブル: 文字列型

型	内容	型 / 範囲
CHAR	固定文字列	0 から 255 までの任意の値
VARCHAR	可変長文字列	0 から 65,535 までの値
BLOB	バイナリ文字列 (バイトの文字列)	L + 2 バイト、ここで L < 2 ¹⁶
TEXT	非バイナリ文字列 (文字の文字列)	L + 2 バイト、ここで L < 2 ¹⁶

データベースの構造

テーブル: オプション

オプション	意味
NOT NULL	“NULL”の設定を禁止する
AUTO_INCREMENT	新しい行に一意の識別子をつける
DEFAULT	標準の値を設定する
PRIMARY_KEY(index_column_name)	主キー制約を設定する

データベースの構造

テーブル: 例

```
CREATE TABLE members (id VARCHAR(10) NOT NULL, name VARCHAR(10), gakka INT);
```

データベースの構造

レコード

- 設定されたカラムに合致したデータのこと
- 1行で表示される1件1件を指す
- 複数のテーブルで構成し、関連付ける
 - 重複を避ける、統一を取る、集計しやすくする

データベースの構造

レコード

単独テーブル

学籍番号 (id)	名前 (name)	コース (course)
22421	田中 太郎	情報コース
23315	中村 花子	電気コース

複数テーブル

学籍番号 (id)	名前 (name)	コース番号 (course_num)
22421	田中 太郎	2
23315	中村 花子	1

コース番号 (course_num)	コース名称 (course_name)
1	電気コース
2	情報コース

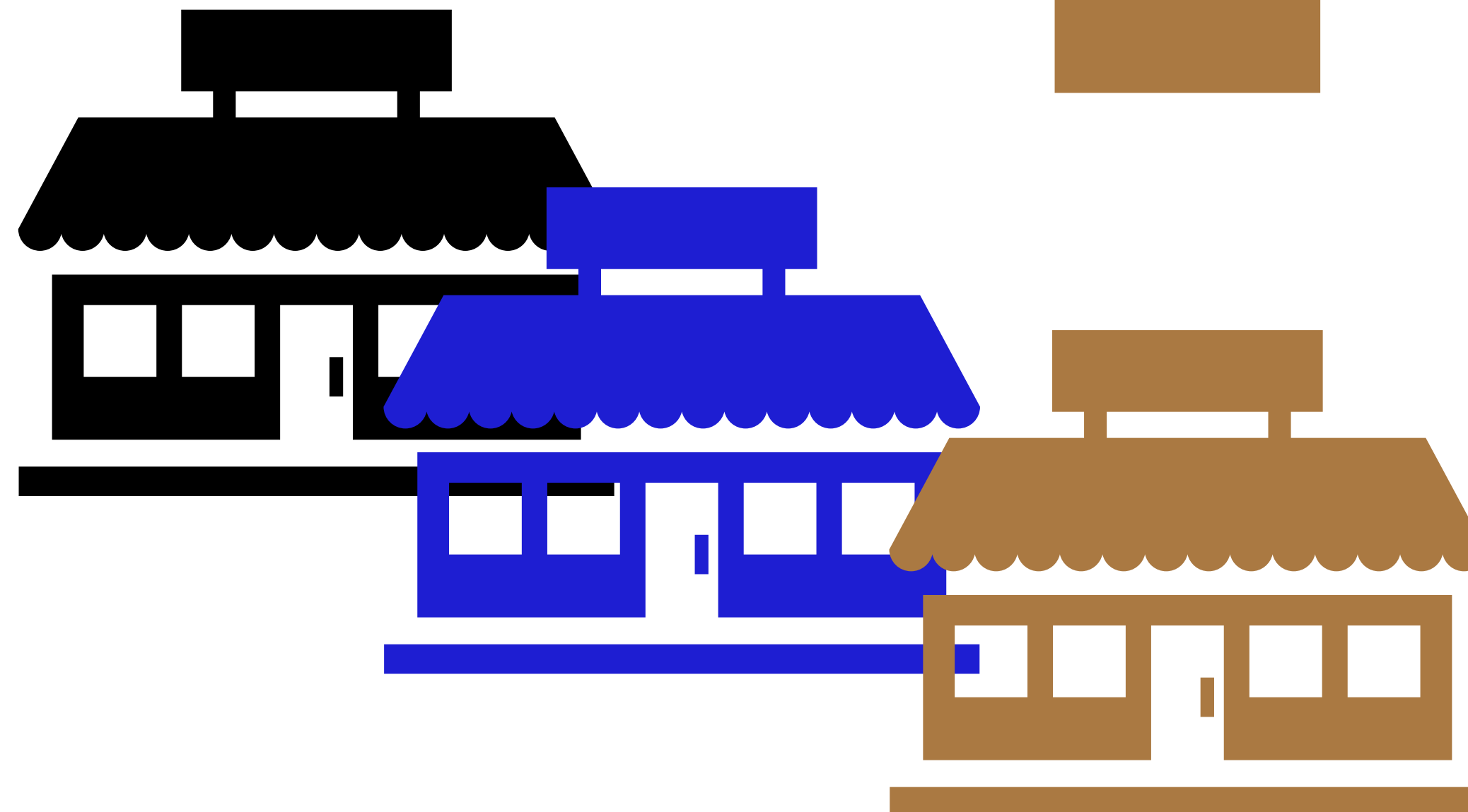
3章

データベースの構築

データの間係を考える



- ・商品名
- ・サイズ
- ・色
- ・単価
- ・在庫数



- ・名前
- ・郵便番号
- ・住所
- ・電話番号
- ・メールアドレス

データベースの構築

データのコード化

顧客台帳

顧客コード
名前
郵便番号
住所
電話番号
メールアドレス

商品台帳

商品コード
商品名
サイズ
色
単価
在庫数

売上台帳

売上年月日
<商品コード>
<顧客コード>
数量

データベースの構築

テーブルと関連の図式化

ER図 (Entity-Relationship図)

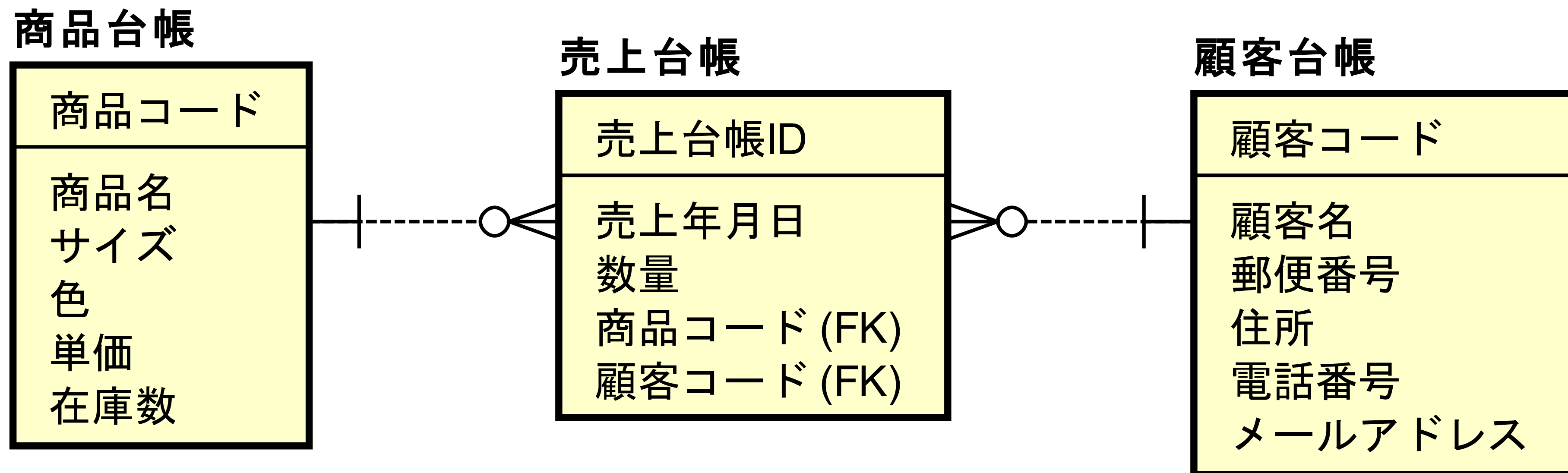
- テーブルの構造を定義し、テーブル間の関係も表す
- テーブルを作成するときのSQLコードの形で表すことができる

データベースの構築

テーブルと関連の図式化

論理モデル

- 登場する項目を現実のものそのまま用いて表したモデル

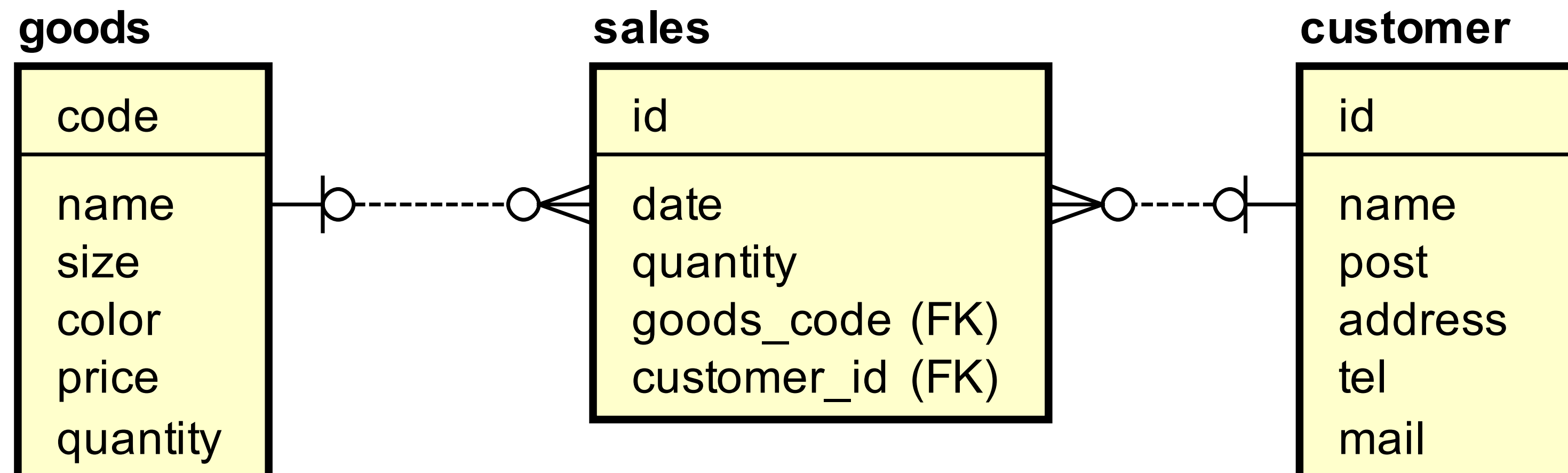


データベースの構築

テーブルと関連の図式化

物理モデル

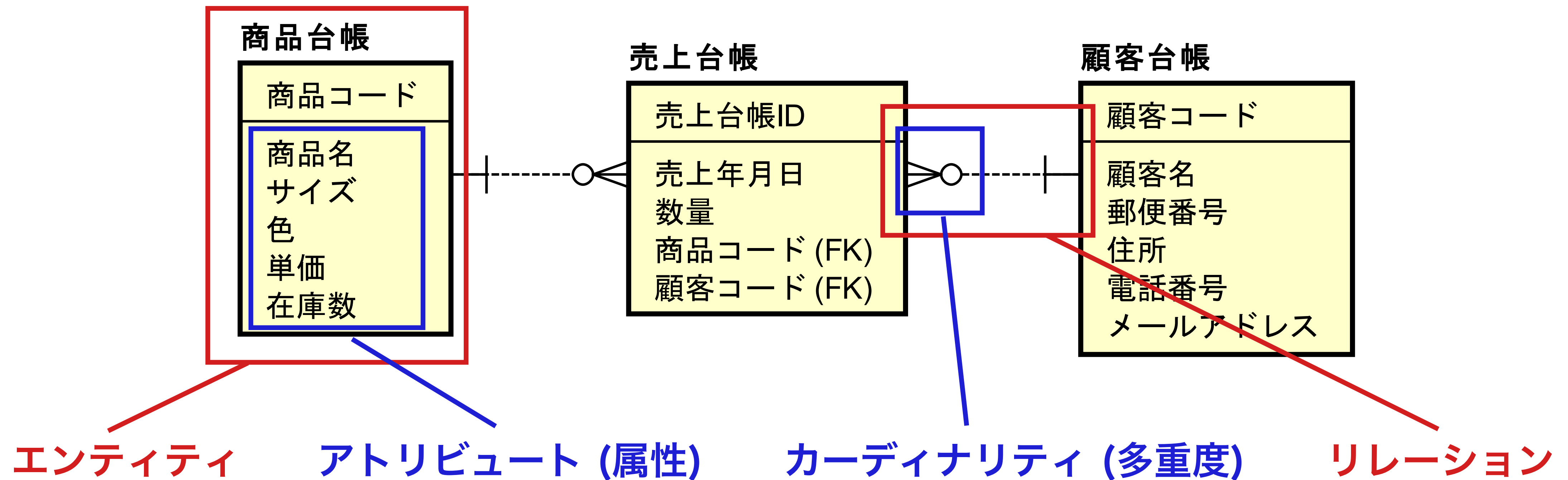
- 物理名で構成されているモデル
- データベースのテーブルに適用する名前のことを物理名と呼ぶ
- テーブルのカラム名が英数字で構成される形



データベースの構築

テーブルと関連の図式化

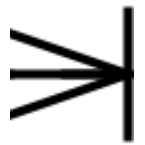
構成要素



データベースの構築

テーブルと関連の図式化

カーディナリティ

記号	意味
	0または1
	1以上
	0または1以上

データベースの構築

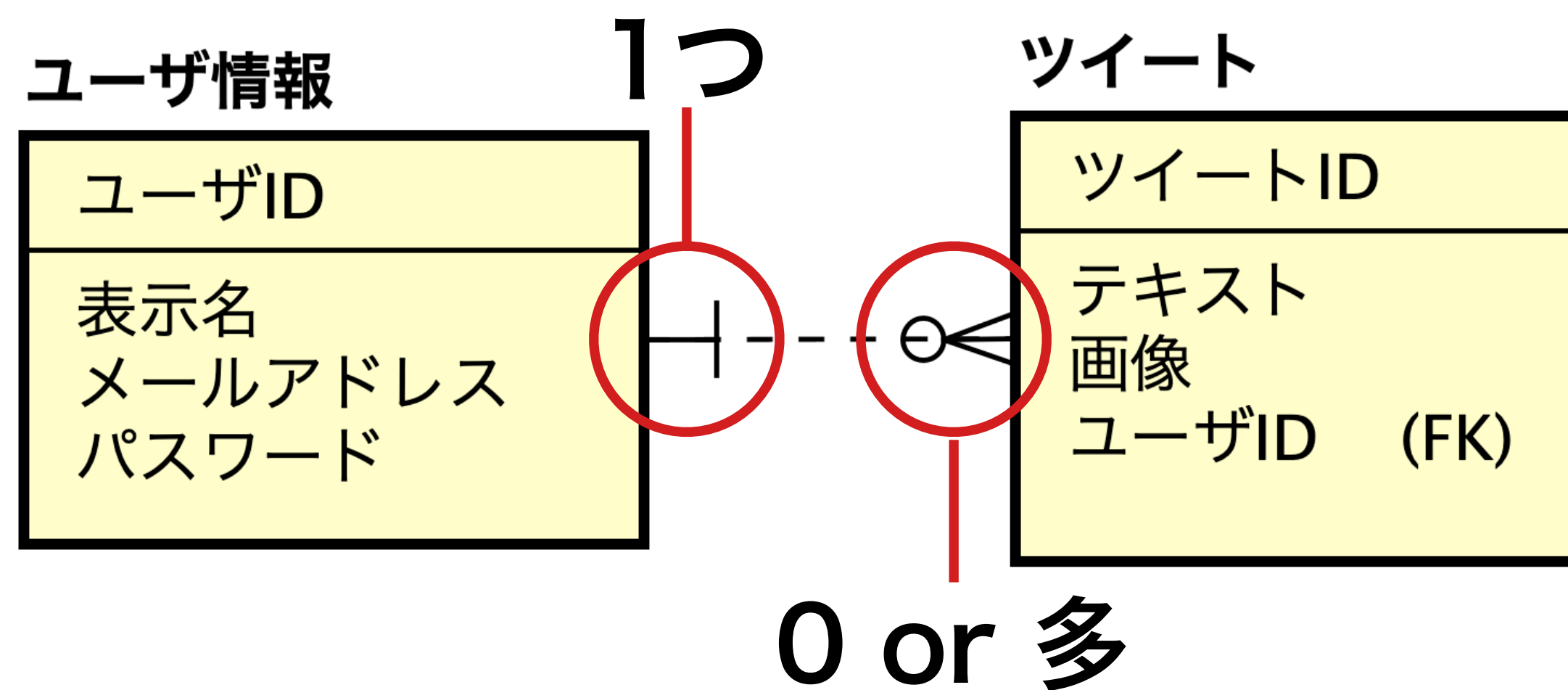
テーブルと関連の図式化

ユーザ → ツイート

- ・ 0個の人 (ROM専) もいればたくさんツイートする人もいる

ツイート → ユーザ

- ・ 必ず誰かがツイートしている



データベースの構築

テーブル設計

顧客台帳 (customer)

論理名	物理名	データ型	オプション
顧客コード	id	INT	NOT NULL, PRIMARY KEY
名前	name	VARCHAR (256)	
郵便番号	post	CHAR (10)	
住所	address	VARCHAR (256)	
電話番号	tel	CHAR (15)	
メールアドレス	mail	VARCHAR (50)	

データベースの構築

テーブル設計

商品台帳 (goods)

論理名	物理名	データ型	オプション
商品コード	code	CHAR (32)	NOT NULL, PRIMARY KEY
商品名	name	VARCHAR (40)	
サイズ	size	CHAR (4)	
色	color	CHAR (6)	
単価	price	INT	
在庫数	quantity	INT	

データベースの構築

テーブル設計

売上台帳 (sales)

論理名	物理名	データ型	オプション
売上台帳ID	id	CHAR (10)	NOT NULL, PRIMARY KEY
売上年月日	date	DATE	
<商品コード>	goods_code	CHAR (32)	FOREIGN KEY (goods.code)
<顧客コード>	customer_id	INT	FOREIGN KEY (customer.id)
数量	quantity	INT	

データベースの構築

データベース作成と接続

```
CREATE DATABASE tshirts_sales_db;  
USE tshirts_sales_db;
```

データベースの構築

テーブル作成

顧客台帳 (customer)

```
CREATE TABLE customer (  
  id INT NOT NULL PRIMARY KEY,  
  name VARCHAR(256),  
  post CHAR(10),  
  address VARCHAR(256),  
  tel CHAR(15),  
  mail VARCHAR(50)  
);
```

データベースの構築

テーブル作成

商品台帳 (goods)

```
CREATE TABLE goods (  
  code CHAR(32) NOT NULL PRIMARY KEY,  
  name VARCHAR(40) NOT NULL,  
  size CHAR(4),  
  color CHAR(6),  
  price INT,  
  quantity INT  
);
```

データベースの構築

テーブル作成

売上台帳 (sales)

```
CREATE TABLE sales (  
  id CHAR(10) NOT NULL PRIMARY KEY,  
  date DATE,  
  quantity INT,  
  goods_code CHAR(32),  
  customer_id INT,  
  
  FOREIGN KEY (goods_code) REFERENCES goods (code),  
  FOREIGN KEY (customer_id) REFERENCES customer (id)  
);
```

データベースの構築

テーブル作成: データ入力

顧客台帳

```
INSERT INTO customer(id,name,post,address,tel,mail) VALUES
(101,"青森商事","030-8570","青森県青森市長島一丁目1-1","017-722-1111","webmaster@pref.aomori.lg.jp"),
(102,"秋田産業","010-8570","秋田県秋田市米区102-1","018-860-1111","info@mail2.pref.akita.jp"),
(103,"岩手観光","020-8570","岩手県盛岡市内丸10-1","019-651-3111","kouhou@pref.iwate.jp"),
(104,"山形販売","990-8570","山形県山形市松波二丁目8-1","023-630-2211","t57W16uF@pref.yamagata.jp"),
(105,"福島ストア","960-8065","福島県福島市杉妻町2-16","024-521-1111","kouho@pref.fukushima.lg.jp");
```

データベースの構築

テーブル作成: データ入力

商品台帳

```
INSERT INTO goods (code, name, size, color, price, quantity) VALUES
("A341", "クールTシャツ", "L", "FF00FF", "1980", "10"),
("A342", "クールTシャツ", "M", "AA5588", "3450", "14"),
("B1001", "速暖シャツ", "S", "00FF00", "890", "3"),
("KK58", "高校指定シャツ", "M", "FA7F00", "2500", "50"),
("12A4561", "速乾Tシャツ", "LL", "FFFFFF", "1280", "250");
```

データベースの構築

テーブル作成: データ入力

売上台帳

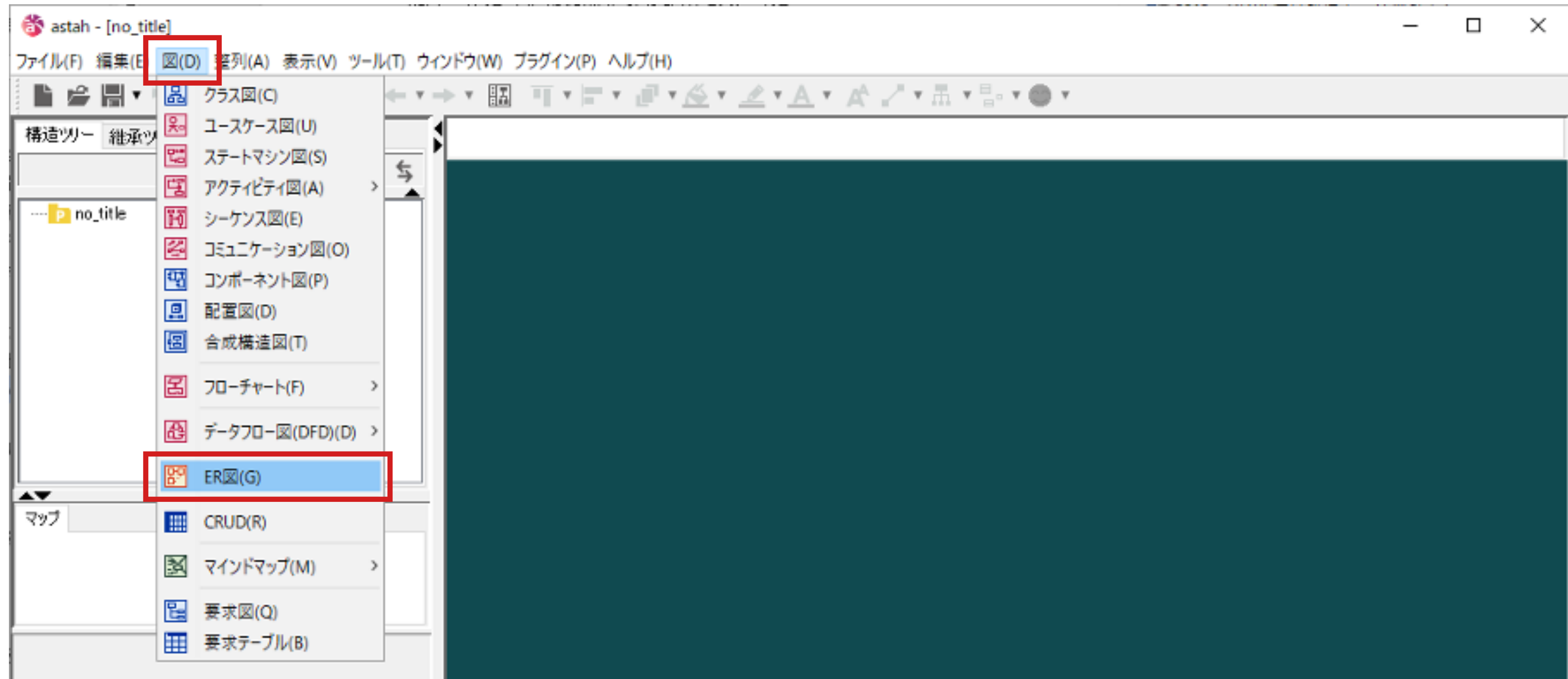
```
INSERT INTO sales(id, date, goods_code, customer_id, quantity) VALUES
("10001", "2010/2/13", "A341", "101", "120"),
("10002", "2011/4/12", "KK58", "103", "10"),
("10003", "2011/5/23", "KK58", "104", "45"),
("10004", "2012/10/19", "12A4561", "102", "22"),
("10005", "2018/12/24", "B1001", "103", "30");
```

4章

ER図の作成

ER図の新規作成

図 → ER図



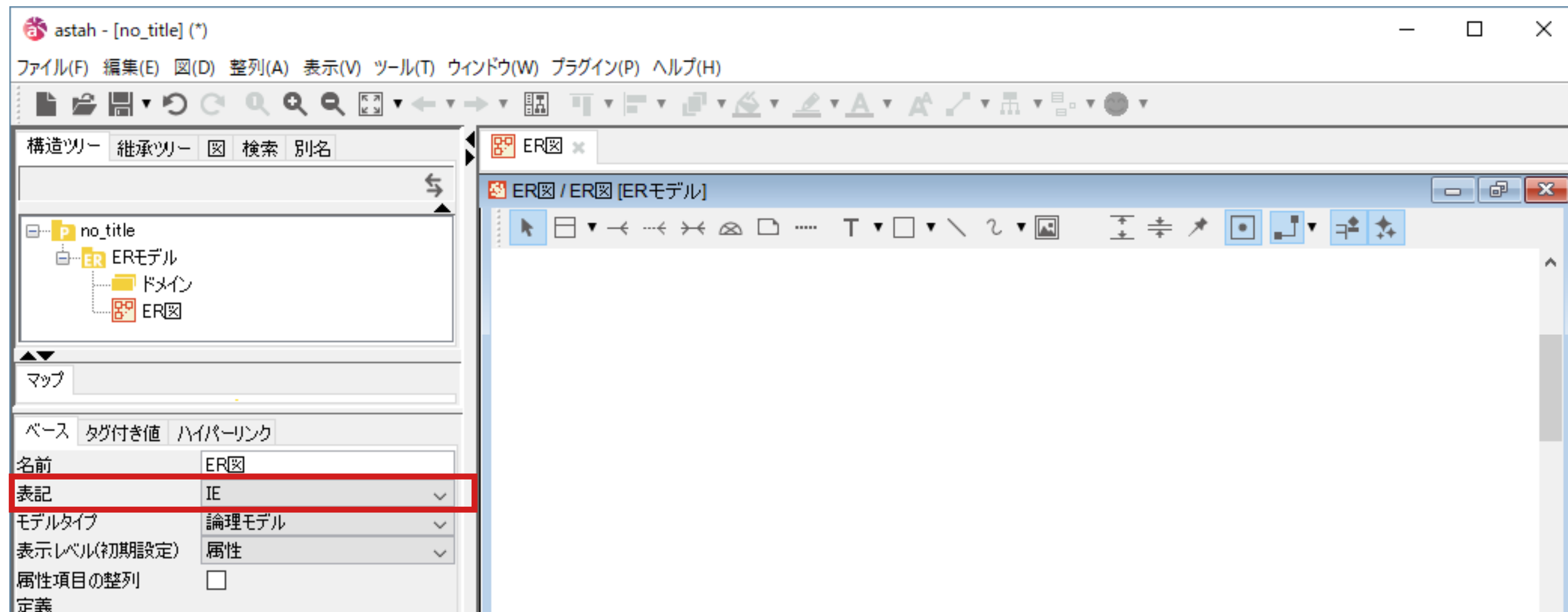
ER図の作成

ER図の新規作成

「表記」を「IE」に変更

書き方のお作法、記法のこと

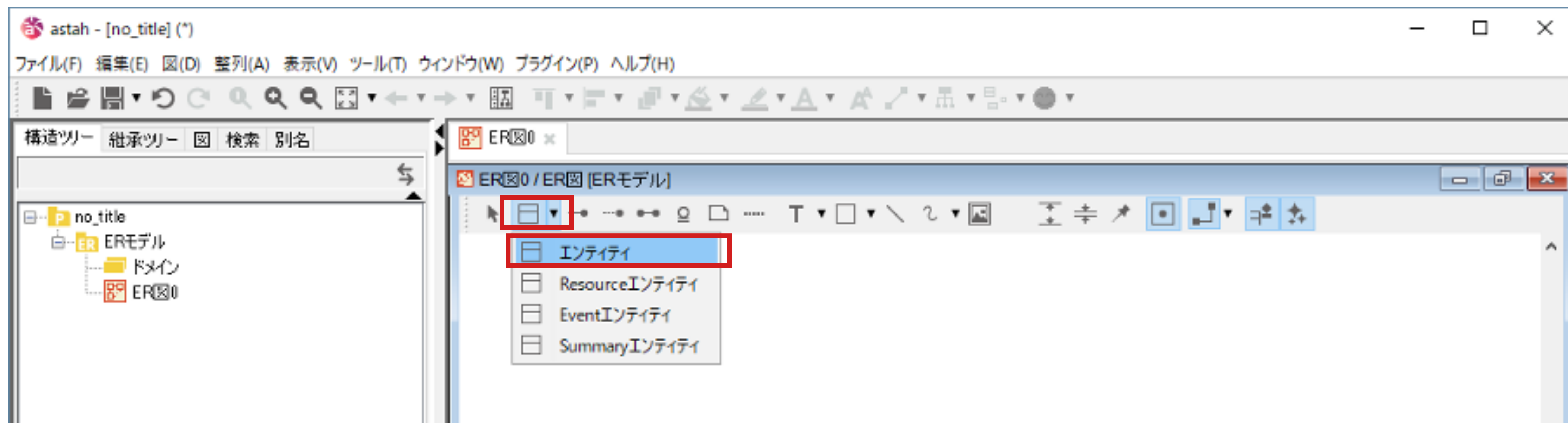
- ・ 「IE記法」と「IDEF1X記法」がある



ER図の作成

エンティティ

エンティティ追加

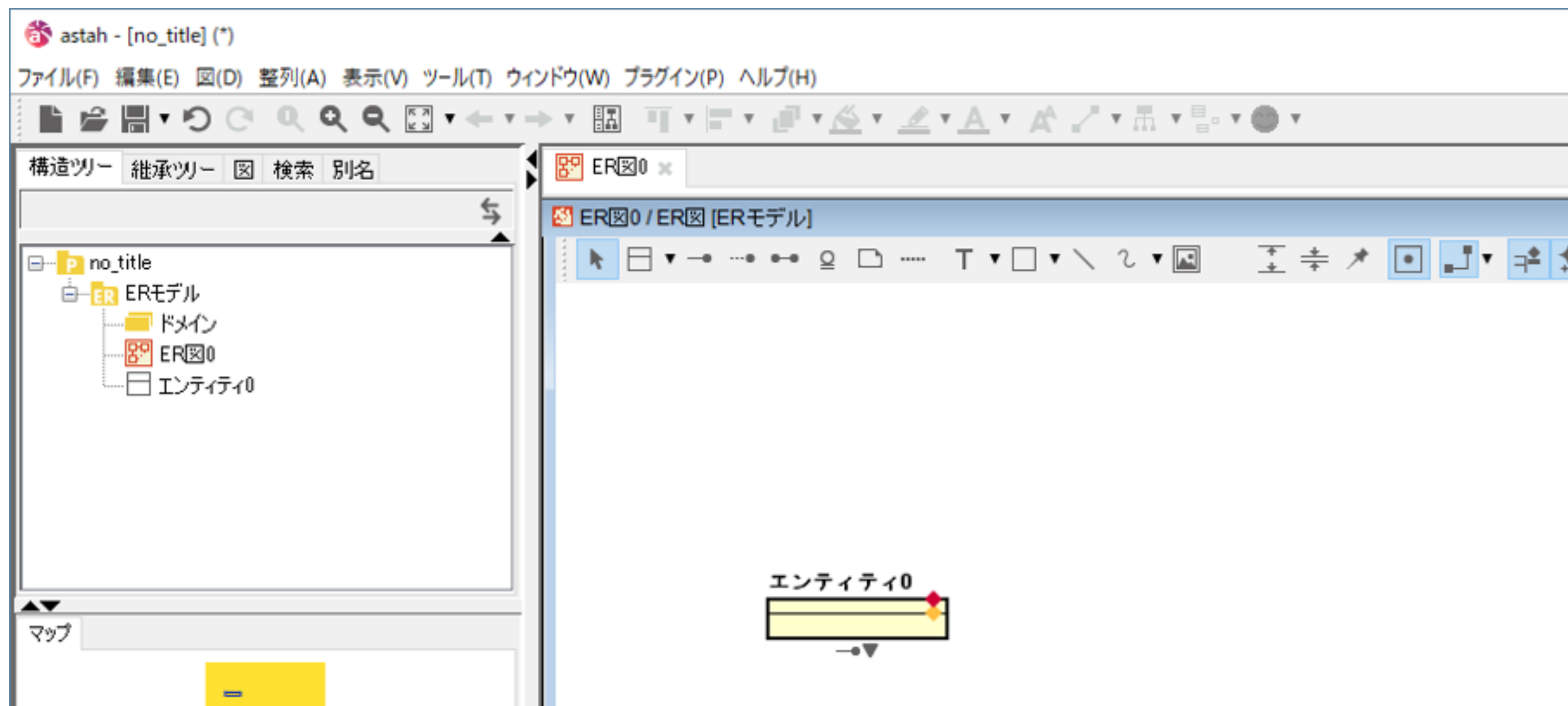


ER図の作成

エンティティ

◆: 主キーを追加するときのアイコン

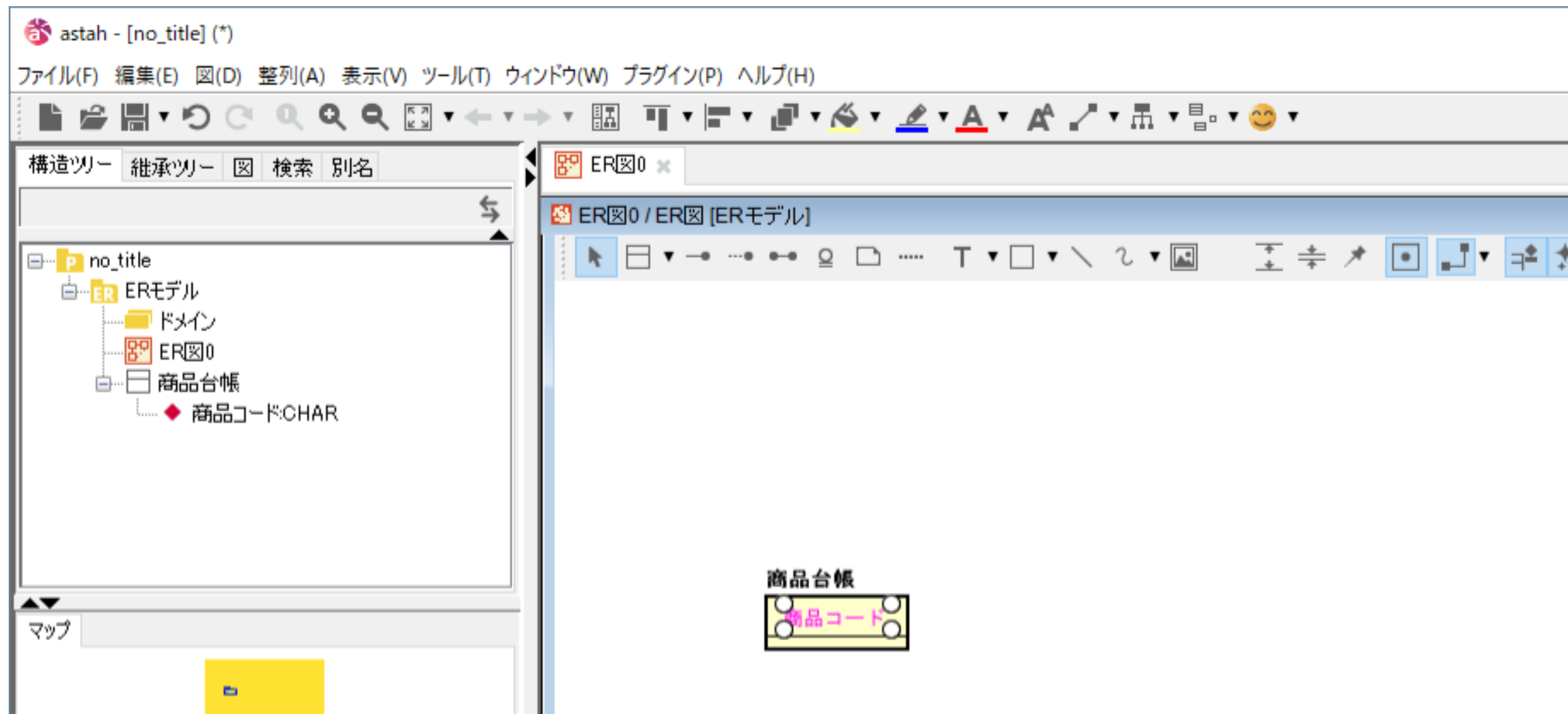
◆: 属性を追加するときのアイコン



ER図の作成

エンティティ

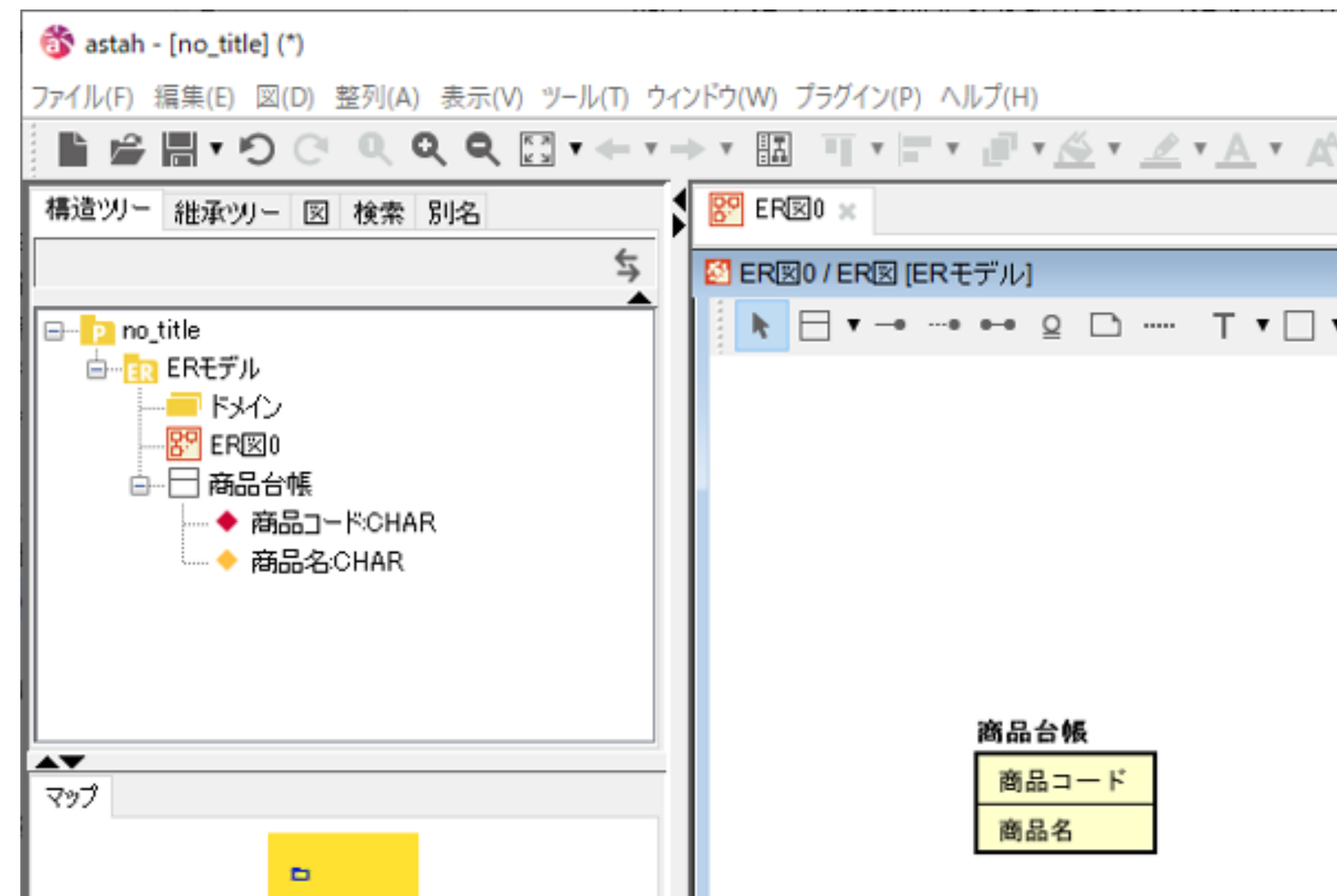
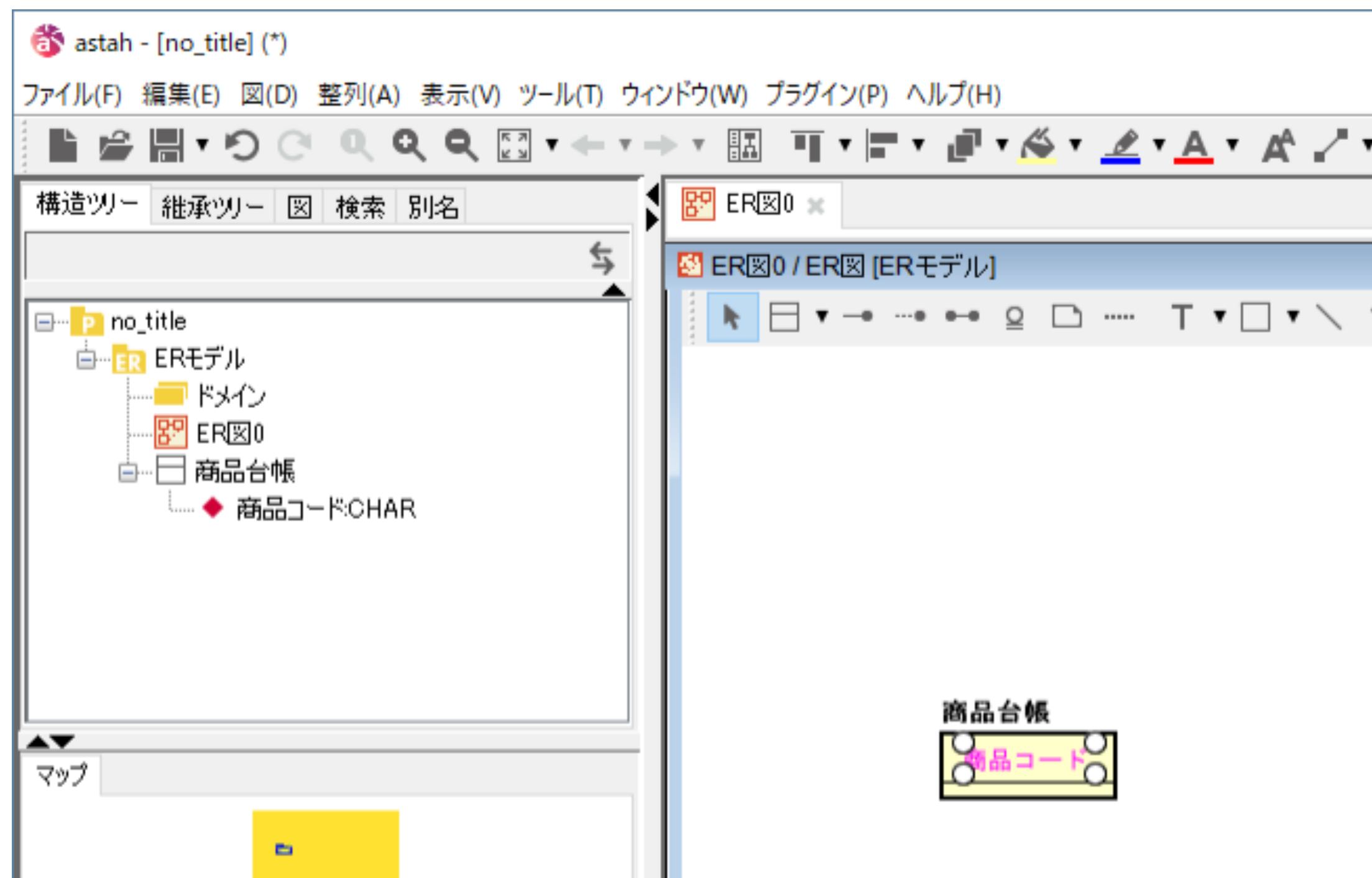
主キーを追加



ER図の作成

エンティティ

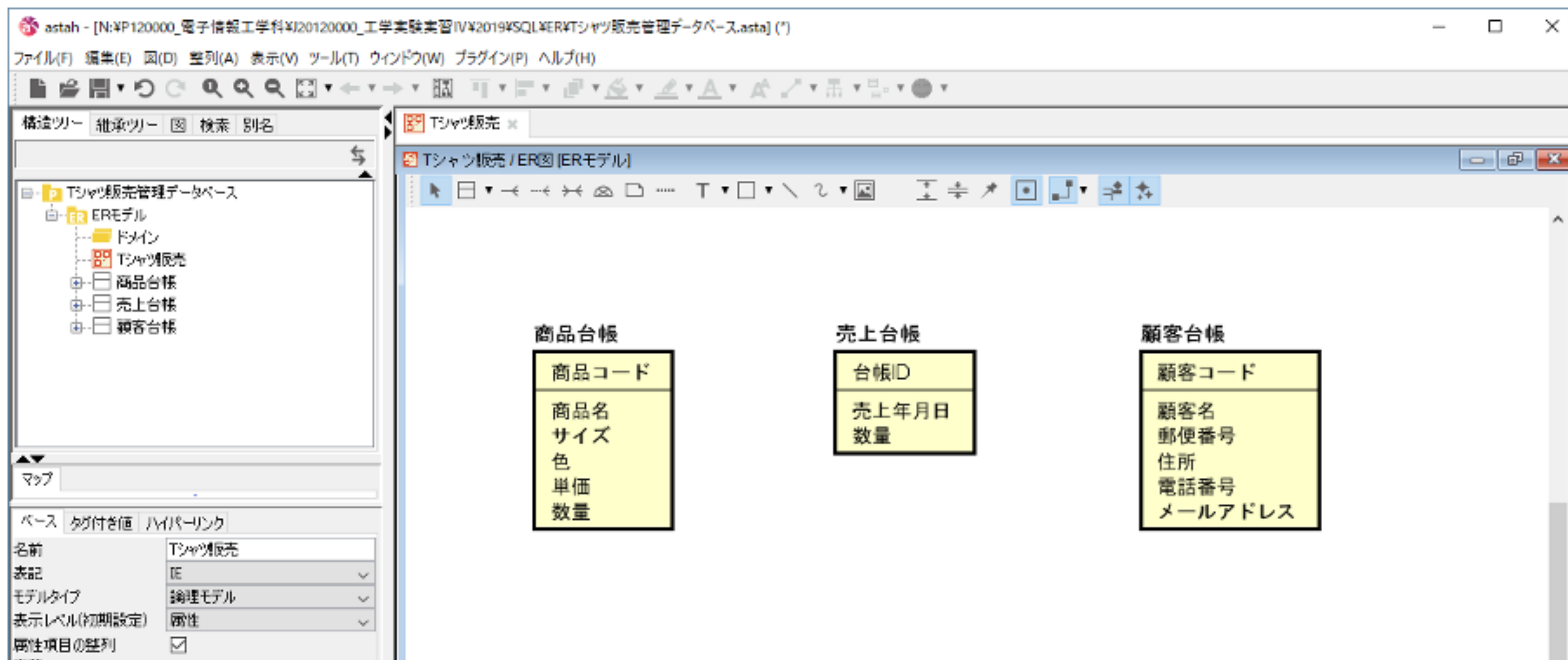
主キーと属性を追加



ER図の作成

エンティティ

他のテーブル、論理名を追加

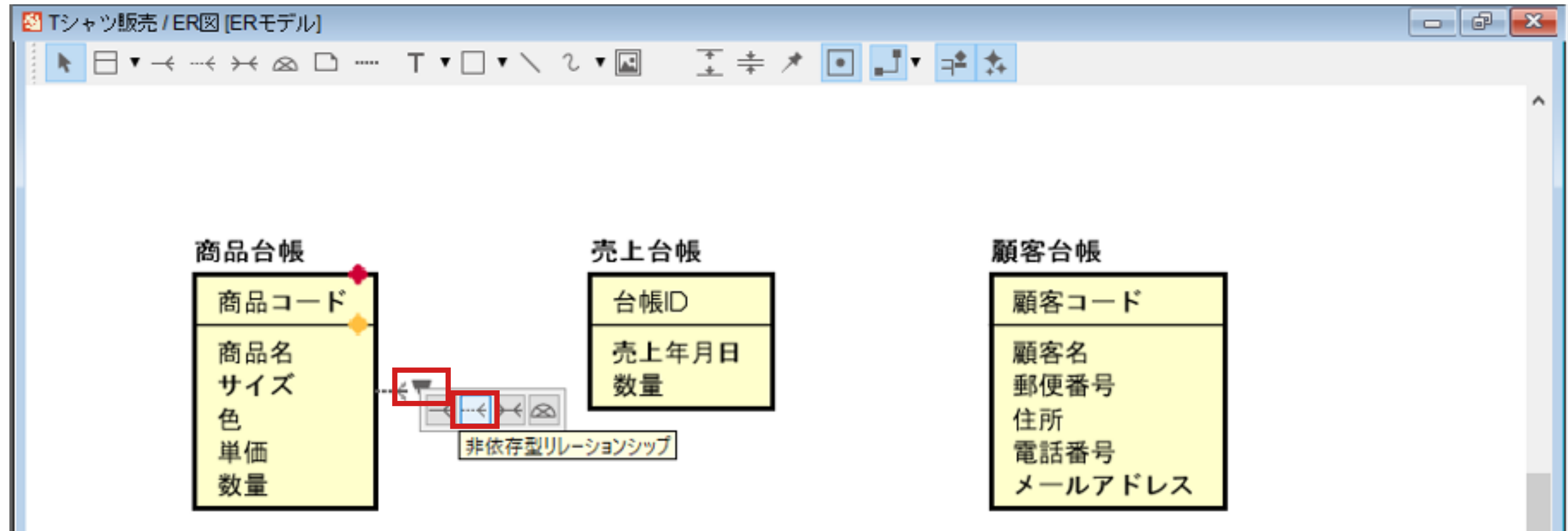


ER図の作成

リレーション

リレーションの接続元

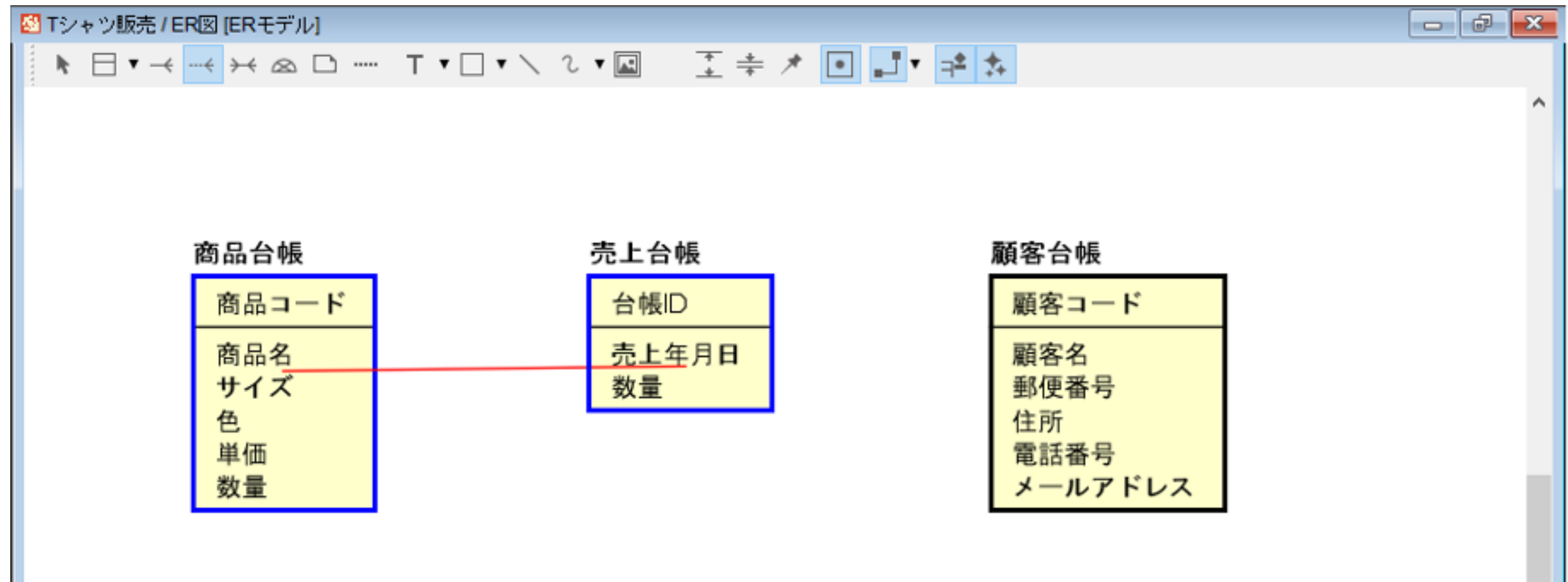
今回は「非依存型リレーション」を選択



ER図の作成

リレーション

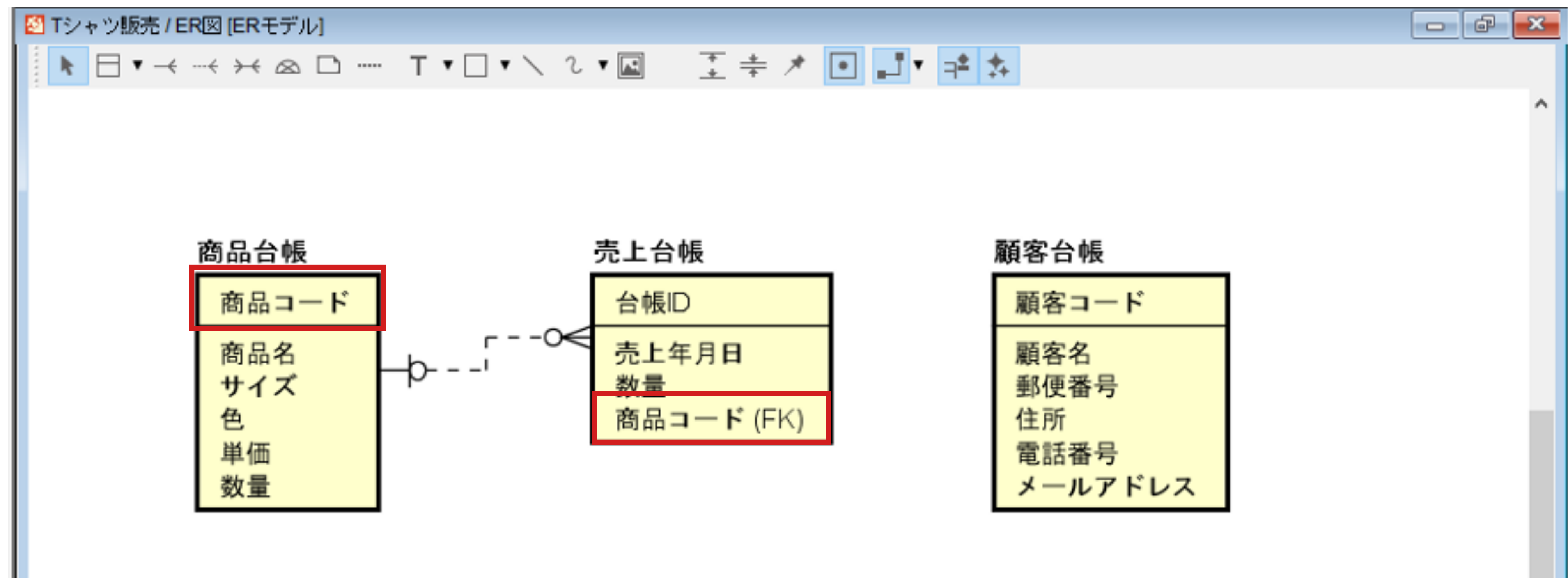
接続先



ER図の作成

リレーション

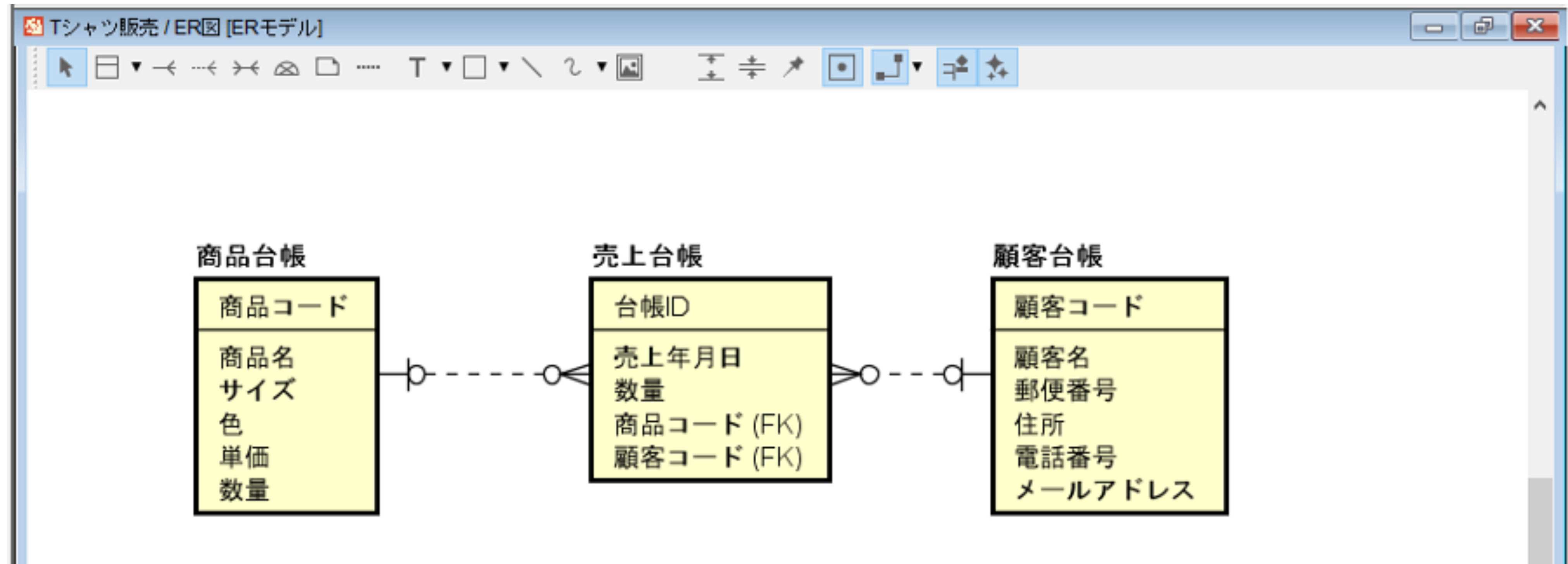
接続元の主キーが、接続先の属性に現れていることを確認



ER図の作成

リレーション

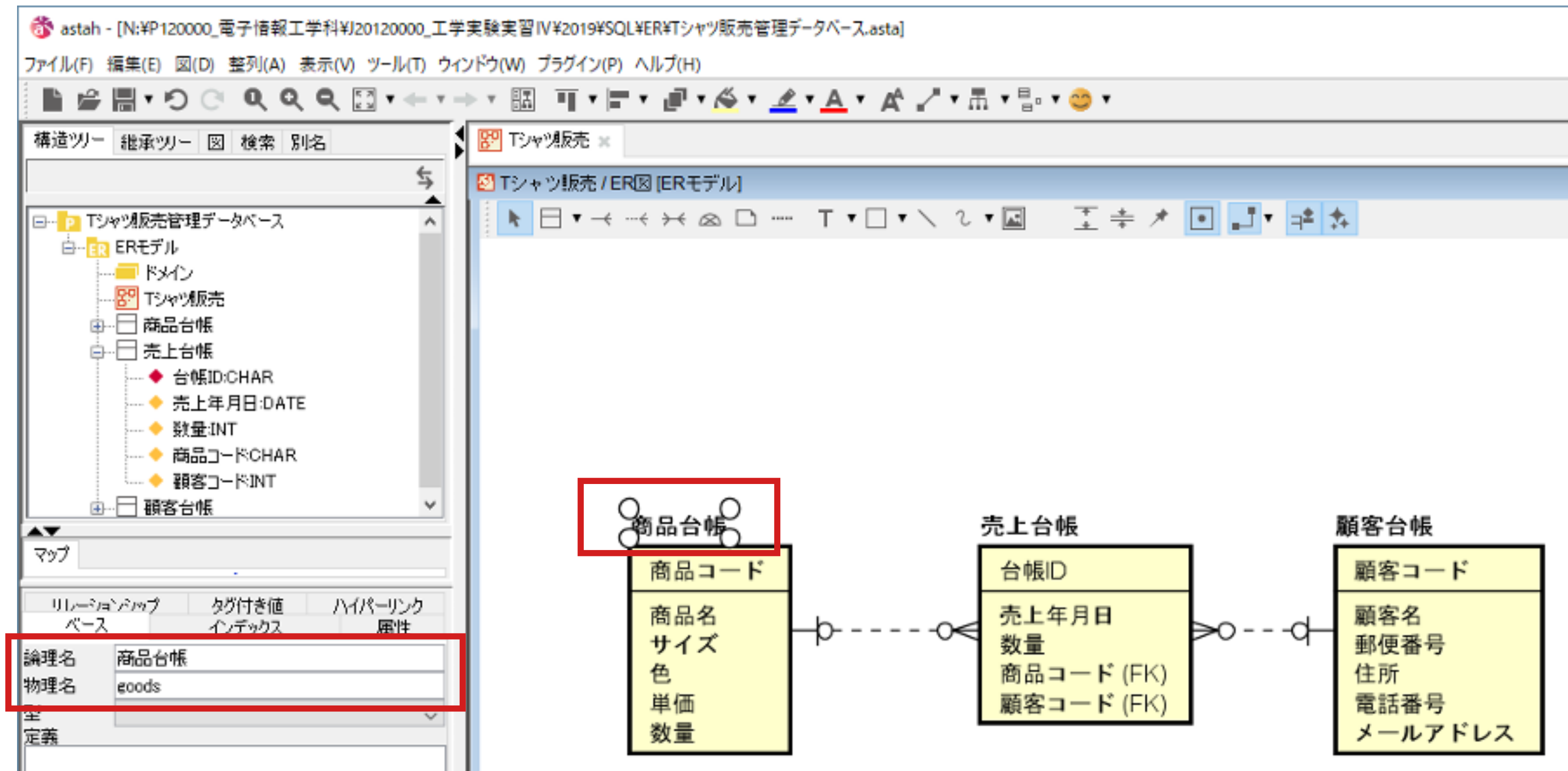
他の接続も関係付け



ER図の作成

物理名: テーブル名

テーブル名をクリックして、プロパティの「物理名」に入力



ER図の作成

物理名: 主キー名

主キーをクリックして、左側の設定で物理名やデータ型を定義する

The screenshot displays the astah software interface for creating an ER diagram. The main window shows an ER diagram with three entities: 商品台帳 (Product Ledger), 売上台帳 (Sales Ledger), and 顧客台帳 (Customer Ledger). The 商品台帳 entity has attributes: 商品コード (Product Code), 商品名 (Product Name), サイズ (Size), 色 (Color), 単価 (Unit Price), and 数量 (Quantity). The 売上台帳 entity has attributes: 台帳ID (Ledger ID), 売上年月日 (Sales Date), 数量 (Quantity), 商品コード (FK) (Product Code (FK)), and 顧客コード (FK) (Customer Code (FK)). The 顧客台帳 entity has attributes: 顧客コード (Customer Code), 顧客名 (Customer Name), 郵便番号 (Postal Code), 住所 (Address), 電話番号 (Phone Number), and メールアドレス (Email Address). The 商品コード attribute in 商品台帳 is highlighted as the primary key. The left sidebar shows the project structure, and the bottom panel shows the properties of the selected primary key.

astah - [N:\P120000_電子情報工学科\20120000_工学実験実習IV\2019\SQL\ER\Tシャツ販売管理データベース.asta] (*)

ファイル(F) 編集(E) 図(D) 整理(A) 表示(V) ツール(T) ウィンドウ(W) プラグイン(P) ヘルプ(H)

構造ツリー 継承ツリー 図 検索 別名

構造ツリー

- 数量:INT
- 売上台帳
 - 台帳ID:CHAR
 - 売上年月日:DATE
 - 数量:INT
 - 商品コード:CHAR
 - 顧客コード:INT
- 顧客台帳
 - 顧客コード:INT
 - 顧客名:CHAR
 - 郵便番号:VARCHAR
 - 住所:VARCHAR
 - 電話番号:CHAR
 - メールアドレス:VARCHAR

マップ

ベース タグ付き値 ハイパーリンク

論理名 商品コード

物理名 code

主キー ☒

NOT NULL ☒

初期値

データ型 CHAR

長さ/精度 32

商品台帳

- 商品コード
- 商品名
- サイズ
- 色
- 単価
- 数量

売上台帳

- 台帳ID
- 売上年月日
- 数量
- 商品コード (FK)
- 顧客コード (FK)

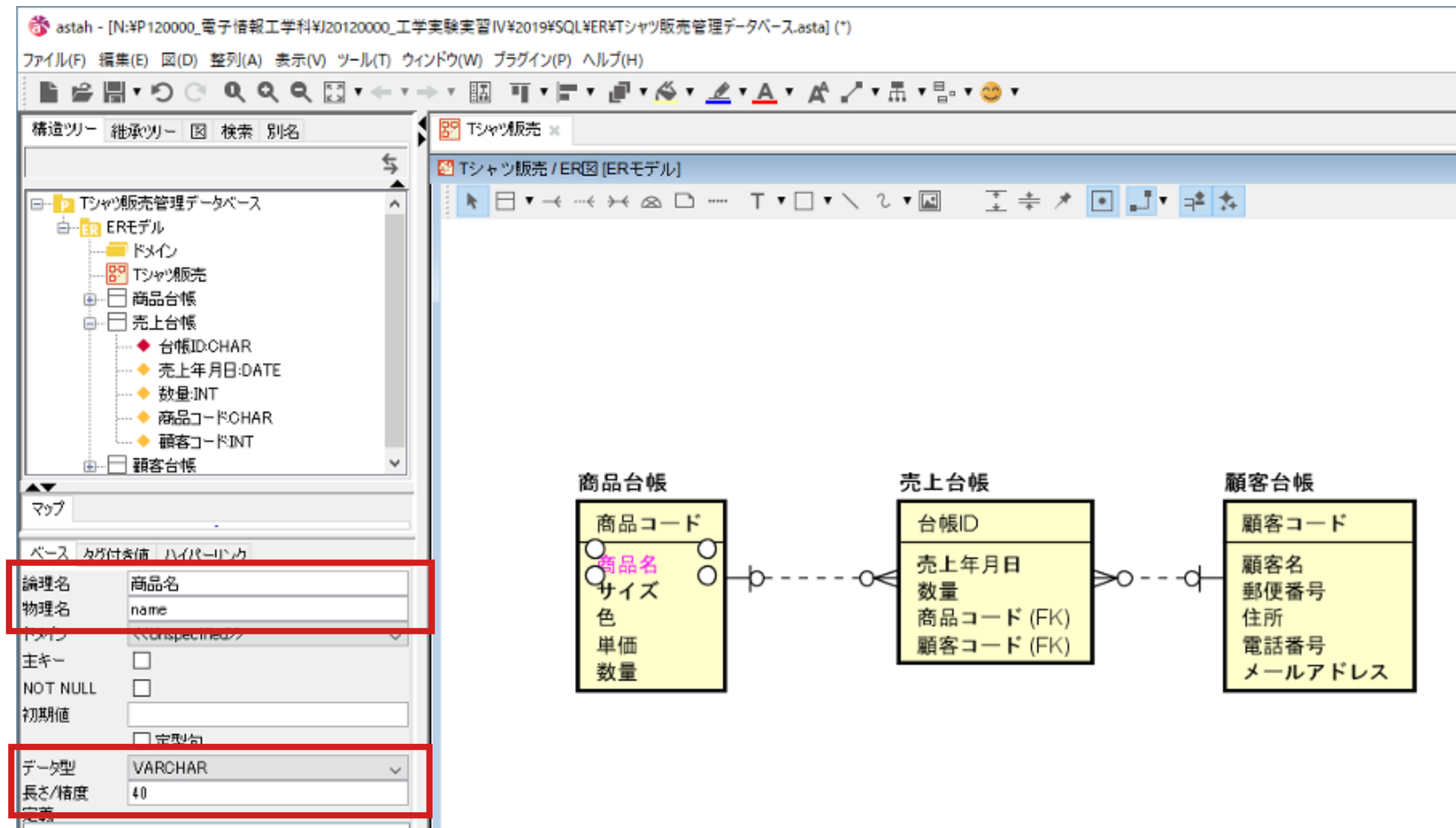
顧客台帳

- 顧客コード
- 顧客名
- 郵便番号
- 住所
- 電話番号
- メールアドレス

ER図の作成

物理名: 属性名

属性をクリックして、物理名やデータ型を定義する

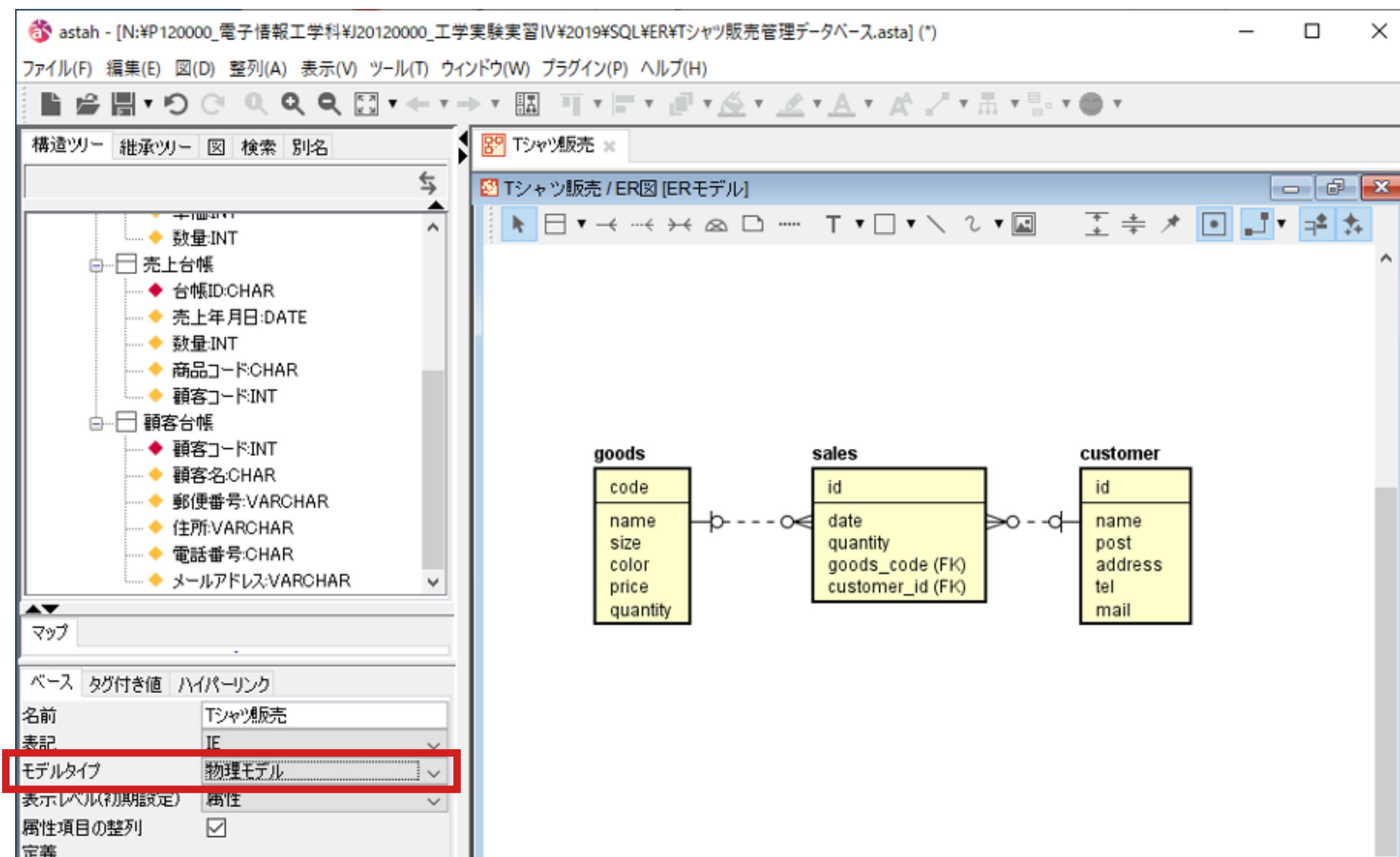


ER図の作成

物理名: 物理モデルを表示

プロパティ → ベース → 「モデルタイプ」を「物理モデル」に切り替え

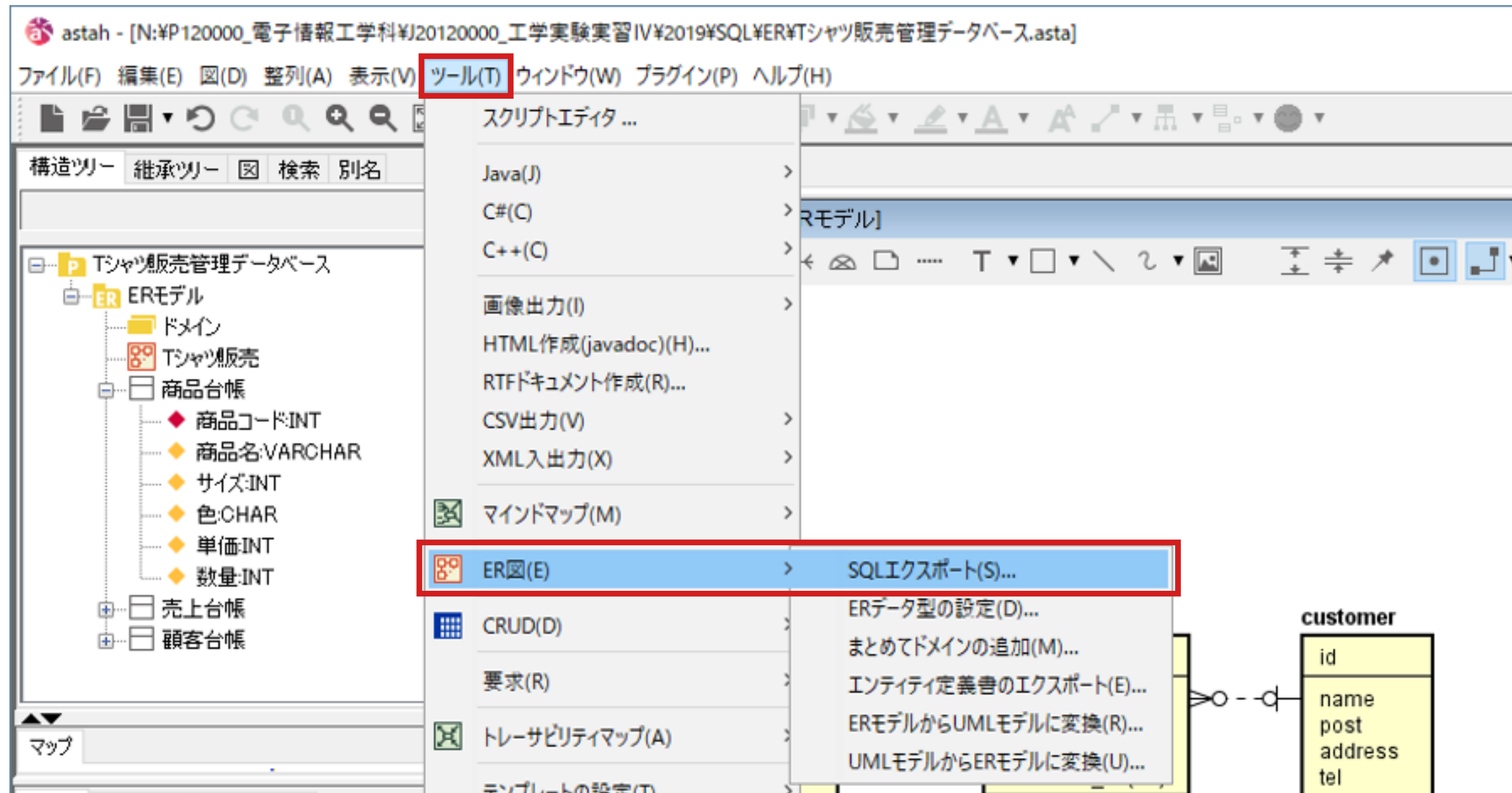
「（論理名）」のように表示されると物理名が設定できていない



ER図の作成

SQL出力

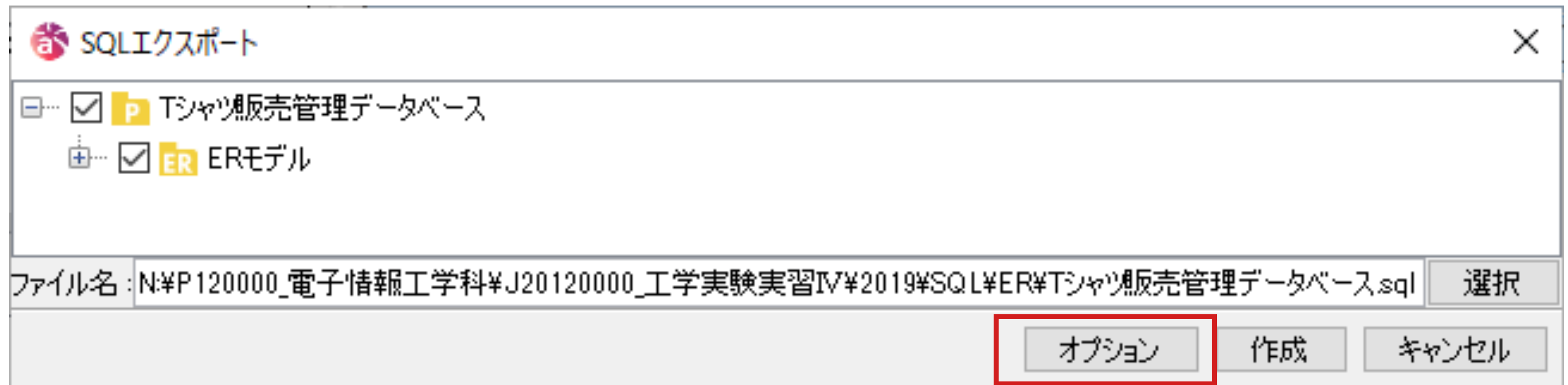
ツール → ER図 → SQLエクスポート



ER図の作成

SQL出力

オプションを選択



ER図の作成

SQL出力

CREATE TABLE文のみ使用するにチェック

SQLエクスポートオプションの設定

出力するモデルタイプ: 物理モデル

☐ カラム名を引用符文字列で囲む

☒ CREATE TABLE文のみを使用する

☐ DROP TABLE文を出力する

☒ 外部キーの出力

DROP TABLE: DROP TABLE文にCASCADE CONSTRAINTSを出力する

一意インデックス: CREATE UNIQUE INDEX文を出力する

☒ 非一意インデックス

☐ セパレータ文字列: :

コメント: エンティティ: なし 属性: なし

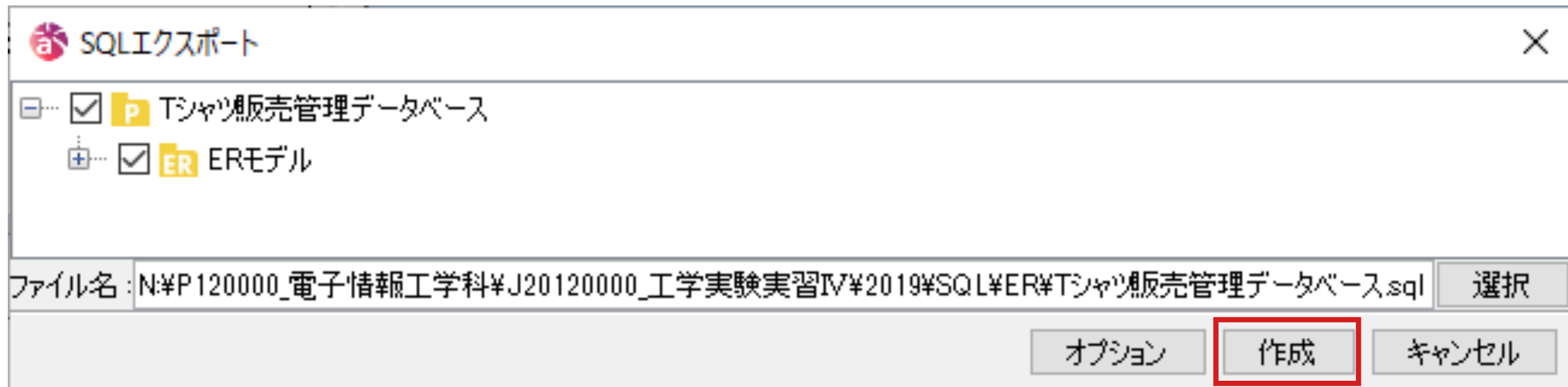
ファイル出力時の文字コード: UTF-8

了解 キャンセル

ER図の作成

SQL出力

作成をクリック



5章

データベースの設計

フィールドの洗い出し



データベースの設計

フィールドの洗い出し

非正規形のテーブル

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
				電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
				材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

データベースの設計

フィールドの洗い出し

第1正規化

- レコードをまたいで定義されている値がないようにわかる
- 1レコードごとに分離

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
22421	4	田中 次郎	情報エレクトロニクス系	電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
22310	4	中村 花子	機械ロボティクス系	材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

データベースの設計

フィールドの洗い出し

第2正規化

- 主となるカラムと従となるカラムを分離

主キー

学籍番号	学年	名前	系	科目	担当者	点数
22421	4	田中 次郎	情報エレクトロニクス系	データベース	中村 剛	90
22421	4	田中 次郎	情報エレクトロニクス系	電子回路	和田 知子	100
22310	4	中村 花子	機械ロボティクス系	制御工学	江田 純也	87
22310	4	中村 花子	機械ロボティクス系	材料力学	藤枝 明彦	68
22315	4	成田 翼	機械ロボティクス系	材料力学	藤枝 明彦	89
21504	3	木村 剛	都市デザイン系	都市計画	内川 早苗	89

主キー

データベースの設計

フィールドの洗い出し

第2正規化

- 候補キー
 - 主キーの候補となるキー
 - レコードを一意に特定するための属性
 - レコードを一意に特定するための属性の組み合わせ
 - 例だと学籍番号、科目

データベースの設計

フィールドの洗い出し

第2正規化

- 部分関数従属
 - 「一部の候補キーに対して」 関数従属する属性
 - 例だと、学籍番号が決まると、学年、名前、系が決まる
 - 学年、名前、系のカラムは候補キーの1つである「学籍番号」に 関数従属しているため「部分関数従属」しているという

データベースの設計

フィールドの洗い出し

第2正規化

- 主となるカラムと従となるカラムを分離

学生に関するデータ

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

科目に関するデータ

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
材料力学	藤枝 明彦
都市計画	内川 早苗

点数に関するデータ

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

データベースの設計

フィールドの洗い出し

第2正規化

- 重複箇所を削除

学生に関するデータ

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

科目に関するデータ

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
材料力学	藤枝 明彦
都市計画	内川 早苗

成績に関するデータ

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

データベースの設計

フィールドの洗い出し

第2正規化

- 重複箇所を削除

学生に関するデータ

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

科目に関するデータ

科目	担当者
データベース	中村 剛
電子回路	和田 知子
制御工学	江田 純也
材料力学	藤枝 明彦
都市計画	内川 早苗

点数に関するデータ

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

データベースの設計

フィールドの洗い出し

第3正規化

- 第2正規化で分離できていない従属している部分を抜き出す



データベースの設計

フィールドの洗い出し

テーブル実装

- 主キーを与える

学生に関するテーブル

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系

成績に関するテーブル

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89

系に関するテーブル

系ID	系
1	情報エレクトロニクス系
2	機械ロボティクス系
3	都市デザイン系

科目に関するテーブル

科目ID	科目	担当者
10001	データベース	中村 剛
10002	電子回路	和田 知子
10003	制御工学	江田 純也
10004	材料力学	藤枝 明彦
10005	都市計画	内川 早苗

データベースの設計

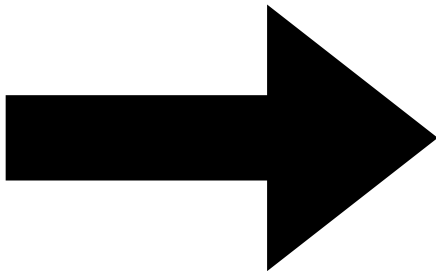
フィールドの洗い出し

テーブル実装

- 外部キーを参照させる

学生に関するテーブル

学籍番号	学年	名前	系
22421	4	田中 次郎	情報エレクトロニクス系
22310	4	中村 花子	機械ロボティクス系
22315	4	成田 翼	機械ロボティクス系
21504	3	木村 剛	都市デザイン系



学籍番号	学年	名前	系ID
22421	4	田中 次郎	1
22310	4	中村 花子	2
22315	4	成田 翼	2
21504	3	木村 剛	3

参照されるテーブル

系ID	系
1	情報エレクトロニクス系
2	機械ロボティクス系
3	都市デザイン系

データベースの設計

フィールドの洗い出し

テーブル実装

- 外部キーを参照させる

成績に関するテーブル

学籍番号	科目	点数
22421	データベース	90
22421	電子回路	100
22310	制御工学	87
22310	材料力学	68
22315	材料力学	89
21504	都市計画	89



参照されるテーブル

科目ID	科目	担当者
10001	データベース	中村 剛
10002	電子回路	和田 知子
10003	制御工学	江田 純也
10004	材料力学	藤枝 明彦
10005	都市計画	内川 早苗

学籍番号	科目ID	点数
22421	10001	90
22421	10002	100
22310	10003	87
22310	10004	68
22315	10004	89
21504	10005	89

データベースの設計

フィールドの洗い出し

テーブル実装

- ・ カラムを定義する

student

論理名	物理名	データ型	オプション
学籍番号	id	INT	NOT NULL, PRIMARY KEY
学年	grade	INT	
名前	name	CHAR (32)	
系	course_id	INT	FOREIGN KEY (course.id)

course

論理名	物理名	データ型	オプション
系ID	id	INT	NOT NULL, PRIMARY KEY
系	course	CHAR (64)	

データベースの設計

フィールドの洗い出し

テーブル実装

- ・ カラムを定義する

score			
論理名	物理名	データ型	オプション
学籍番号	id	INT	NOT NULL, PRIMARY KEY
科目	subject_id	INT	NOT NULL, PRIMARY KEY FOREIGN KEY (subject.id)
点数	score	INT	

subject			
論理名	物理名	データ型	オプション
科目ID	id	INT	NOT NULL, PRIMARY KEY
科目	subject	VARCHAR (128)	
担当者	teacher	CHAR (32)	

6章

SQL

データベース作成

```
> CREATE DATABASE database_name;
```

- ・「company」というデータベースを作成するとき

```
> CREATE DATABASE company;
```

SQL

データベース作成

- ・ 今あるデータベースを一覧で確認

```
> SHOW DATABASES;
```

```
> SHOW DATABASES;
+-----+
| Database           |
+-----+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+-----+
```

SQL

データベース作成

- 指定したデータベースを操作対象にする
 - データベースを作成しただけでは操作対象になっていない

```
> USE database_name;
```

- 「company」というデータベースを操作対象するとき

```
> USE company;
```

SQL

テーブル作成

```
> CREATE TABLE table_name (col_name1 date_type1,  
col_name2 date_type2, . . . ) table_option
```

- 「member」というテーブルを作成するとき

```
> CREATE TABLE member (  
id int NOT NULL,  
name VARCHAR (64) ,  
place VARCHAR (64) ,  
age INT  
);
```

SQL

テーブル作成

- 対象とするデータベースに含まれるテーブル一覧の表示

```
> SHOW TABLES;
```

```
> SHOW TABLES;
```

```
+-----+  
| Tables_in_company |  
+-----+  
| member             |  
+-----+
```

SQL

テーブル作成

- ・ テーブルの定義を確認

```
> DESC table_name;
```

```
> DESC member;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	varchar(64)	YES		NULL	
age	int(11)	YES		NULL	

SQL

挿入

- データをテーブルに挿入する

```
> INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

- 1件挿入する

```
> INSERT INTO member (id, name, place, age) VALUES (101, '木村太郎', '北海道', 28);
```

- 複数件挿入する

```
> INSERT INTO member (id, name, place, age) VALUES  
(102, '田中花子', '宮城', 38),  
(103, '中村裕子', '宮城', 41);
```

SQL

挿入

- ・ カラムの順番を、登録されている順番とするとき
 - ・ (col1, col2, ...) が不要

```
> INSERT INTO member VALUES (104, '中村康太', '青森', 41);
```

実行結果

```
> SELECT * FROM member;
+-----+-----+-----+-----+
| id    | name          | place    | age    |
+-----+-----+-----+-----+
| 101   | 木村太郎      | 北海道   | 28     |
| 102   | 田中花子      | 宮城     | 38     |
| 103   | 中村裕子      | 宮城     | 41     |
| 104   | 中村康太      | 青森     | 41     |
+-----+-----+-----+-----+
```

SQL

選択

- データを抽出する

```
> SELECT col1, col2, ... FROM table_name WHERE condition
```

- 全てを表示する場合 (表示する項目、順番を指定しない)

```
> SELECT * FROM member; ← 「*」を指定
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

SQL

選択

- 表示させたいものがある場合は具体的に指定する

```
> SELECT id,name FROM member;
```

id	name
101	木村太郎
102	田中花子
103	中村裕子
104	中村康太

SQL

選択

- ・ 複数テーブルを扱う時、カラム名が重複する場合はテーブル名を明示する

```
> SELECT id,member.name,member.age FROM member;
```

id	name	age
101	木村太郎	28
102	田中花子	38
103	中村裕子	41
104	中村康太	41

SQL

選択

- 表示するカラム名を指定したいとき

```
> SELECT id,member.name,member.age AS toshi FROM member;
```

id	name	toshi
101	木村太郎	28
102	田中花子	38
103	中村裕子	41
104	中村康太	41

SQL

選択

- ・ 順番を指定したい時

```
> SELECT * FROM member ORDER BY age;  
+-----+-----+-----+-----+  
| id  | name          | place      | age  |  
+-----+-----+-----+-----+  
| 101 | 木村太郎      | 北海道     | 28   |  
| 102 | 田中花子      | 宮城        | 38   |  
| 103 | 中村裕子      | 宮城        | 41   |  
| 104 | 中村康太      | 青森        | 41   |  
+-----+-----+-----+-----+
```

- ・ 昇順は「ASC」（デフォルト）、降順は「DESC」

```
> SELECT * FROM member ORDER BY age ASC;
```

```
> SELECT * FROM member ORDER BY age DESC;
```

SQL

選択

- ・ テーブル結合時の重複結果の除外

```
> SELECT id,name,place,age FROM member;  
+-----+-----+-----+-----+  
| id  | name      | place  | age  |  
+-----+-----+-----+-----+  
| 104 | 中村康太 | 青森   | 41   |  
| 102 | 田中花子 | 宮城   | 38   |  
| 102 | 田中花子 | 宮城   | 38   |  
+-----+-----+-----+-----+
```

```
> SELECT DISTINCT id,name,place,age FROM member;  
+-----+-----+-----+-----+  
| id  | name      | place  | age  |  
+-----+-----+-----+-----+  
| 104 | 中村康太 | 青森   | 41   |  
| 102 | 田中花子 | 宮城   | 38   |  
+-----+-----+-----+-----+
```

SQL

選択

- 条件を指定したい時の演算子

演算子	例	説明
=	age = 20	ageが20である
!=	age != 20	ageが20でない
<	age < 20	ageが20未満
<=	age <= 20	ageが20以下
>	age > 20	ageが20よりも大きい
>=	age >= 20	ageが20以上

SQL

選択

- 条件を指定したい時の演算子

演算子	例	説明
AND	age = 41 AND place='宮城'	ageが41かつplaceが宮城
OR	age = 41 OR place='宮城'	ageが41またはplaceが宮城
BETWEEN AND	age BETWEEN 20 AND 30	age が20から30の間
LIKE	age LIKE 41	ageが41

- ワイルドカード

記号	意味	例	説明
%	0文字以上の文字列	name LIKE '藤田%'	nameが藤田から始まるすべて
_	任意の1文字	age LIKE '4_'	ageが4から始まる2桁の文字

SQL

選択

- 条件を指定したい時の演算子

```
> SELECT id,name,place,age FROM member WHERE age=41;
```

id	name	place	age
103	中村裕子	宮城	41
104	中村康太	青森	41

```
> SELECT id,name,place,age FROM member WHERE age=41 AND place='宮城';
```

id	name	place	age
103	中村裕子	宮城	41

SQL

選択

- 条件を指定したい時の演算子

```
> SELECT * FROM member WHERE age>30;
+-----+-----+-----+-----+
| id  | name          | place  | age  |
+-----+-----+-----+-----+
| 102 | 田中花子      | 宮城   | 38   |
| 103 | 中村裕子      | 宮城   | 41   |
| 104 | 中村康太      | 青森   | 41   |
+-----+-----+-----+-----+
```

SQL

選択

- ・ワイルドカード

```
> SELECT * FROM member where name LIKE "中村%";
```

id	name	place	age
103	中村裕子	宮城	41
104	中村康太	青森	41

```
> SELECT * FROM member WHERE age LIKE '_8';
```

id	name	place	age
101	木村太郎	北海道	28
102	田中花子	宮城	38

SQL

選択

- 最大値

```
> SELECT MAX(age) FROM member;  
+-----+  
| MAX(age) |  
+-----+  
|         41 |  
+-----+
```

- 平均値

```
> SELECT AVG(age) FROM member;  
+-----+  
| AVG(age) |  
+-----+  
|  37.0000 |  
+-----+
```

SQL

選択

- 合計

```
> SELECT SUM(age) FROM member;  
+-----+  
| SUM(age) |  
+-----+  
|      148 |  
+-----+
```

- 行の数を数える (NULL含む)

```
> SELECT COUNT(*) FROM member;  
+-----+  
| COUNT(*) |  
+-----+  
|         4 |  
+-----+
```

SQL

削除

- データベース削除

```
> DROP DATABASE database_name;
```

- テーブル削除

```
> DROP TABLE table_name;
```

SQL

削除

- データ削除

```
> DELETE FROM table_name WHERE condition;
```

```
> DELETE FROM member WHERE id=104;
> SELECT * FROM member;
+-----+-----+-----+-----+
| id    | name          | place    | age    |
+-----+-----+-----+-----+
| 101   | 木村太郎      | 北海道   | 28     |
| 102   | 田中花子      | 宮城     | 38     |
| 103   | 中村裕子      | 宮城     | 41     |
+-----+-----+-----+-----+
```

SQL

削除

- データ削除 (指定したテーブルのすべてのデータ)

```
> DELETE FROM table_name;
```

SQL

更新

- **すでに登録されているレコードの更新**
 - 条件を正しく指定しないと、当てはまるすべてのレコードが更新されるので注意

```
> UPDATE table_name SET col=val WHERE condition;
```

```
> UPDATE member SET place='岩手' WHERE id='101';
```

```
> SELECT * FROM member;
```

id	name	place	age
101	木村太郎	岩手	28
102	田中花子	宮城	38
103	中村裕子	宮城	41
104	中村康太	青森	41

SQL

設定変更

- データ型変更

```
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   |     | NULL    |       |
| name  | varchar(64)   | YES  |     | NULL    |       |
| place | varchar(64) | YES  |     | NULL    |       |
| age   | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

```
> ALTER TABLE member MODIFY place char(32);
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   |     | NULL    |       |
| name  | varchar(64)   | YES  |     | NULL    |       |
| place | char(32)     | YES  |     | NULL    |       |
| age   | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

SQL

設定変更

- カラム追加

```
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   |     | NULL    |       |
| name  | varchar(64)   | YES  |     | NULL    |       |
| place | char(32)      | YES  |     | NULL    |       |
| age   | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

```
> ALTER TABLE member add birthday date;
> DESC member;
+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   |     | NULL    |       |
| name       | varchar(64)   | YES  |     | NULL    |       |
| place      | char(32)      | YES  |     | NULL    |       |
| age        | int(11)       | YES  |     | NULL    |       |
| birthday   | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

SQL

設定変更

- ・ カラム名

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

> ALTER TABLE member CHANGE name namae varchar(64);

> DESC member;

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
namae	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	
birthday	date	YES		NULL	

SQL

設定変更

- カラム削除

```
> DESC member;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   |     | NULL    |       |
| namae      | varchar(64)   | YES  |     | NULL    |       |
| place      | char(32)      | YES  |     | NULL    |       |
| age        | int(11)       | YES  |     | NULL    |       |
| birthday   | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

```
> ALTER TABLE member DROP birthday;
> DESC member;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   |     | NULL    |       |
| namae  | varchar(64)   | YES  |     | NULL    |       |
| place  | char(32)      | YES  |     | NULL    |       |
| age    | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

SQL

設定変更

- 主キー設定 (単独カラム)

```
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   |     | NULL    |       |
| name   | varchar(64)   | YES  |     | NULL    |       |
| place  | char(32)      | YES  |     | NULL    |       |
| age    | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

```
> ALTER TABLE member ADD PRIMARY KEY (id);
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   | PRI | NULL    |       |
| name   | varchar(64)   | YES  |     | NULL    |       |
| place  | char(32)      | YES  |     | NULL    |       |
| age    | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

SQL

設定変更

- 主キー設定 (複数カラム)

```
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   |     | NULL    |       |
| name   | varchar(64)   | YES  |     | NULL    |       |
| place  | char(32)      | YES  |     | NULL    |       |
| age    | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

```
> ALTER TABLE member ADD PRIMARY KEY (id,name);
> DESC member;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   | PRI | NULL    |       |
| name   | varchar(64)   | NO   | PRI | NULL    |       |
| place  | char(32)      | YES  |     | NULL    |       |
| age    | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
```

SQL

設定変更

- 主キー削除

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

> DROP INDEX `PRIMARY` ON member;

> DESC member;

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		NULL	
name	varchar(64)	YES		NULL	
place	char(32)	YES		NULL	
age	int(11)	YES		NULL	

SQL

JOIN

- ・ テーブルの関係をデータの抽出に反映させるための命令

科目(subject)	教員ID(teacher_id)
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

教員ID(id)	教員(name)
401	押田 京一
402	荒井 善昭
403	楡井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

SQL

JOIN

- まずはテーブルを作成してみよう

```
> CREATE DATABASE nnctj;
> USE nnctj;
> CREATE TABLE subject(subject VARCHAR (32), teacher_id INT);
> CREATE TABLE teacher(id INT, name VARCHAR(32));
> INSERT INTO subject (subject, teacher_id) VALUES
("電磁気学", "404"),
("電子回路", "402"),
("シミュレーション", "404"),
("ネットワーク基礎", "407"),
("オブジェクト指向", "409"),
("集積回路設計", "408"),
("計算機科学史", "405"),
("フーリエ解析", "601"),
("画像処理", "401"),
("ベクトル解析", "603");
```

SQL

JOIN

- まずはテーブルを作成してみよう

```
> INSERT INTO teacher (id, name) VALUES  
("401", "押田 京一"),  
("402", "荒井 善昭"),  
("403", "楡井 雅巳"),  
("404", "西村 治"),  
("405", "藤澤 義範"),  
("406", "大矢 健一"),  
("407", "伊藤 祥一"),  
("408", "芦田 和毅"),  
("409", "藤田 悠");
```

SQL

JOIN

- テーブルの確認

```
> SELECT * FROM subject;
+-----+-----+
| subject          | teacher_id |
+-----+-----+
| 電磁気学          |          404 |
| 電子回路          |          402 |
| シミュレーション  |          404 |
| ネットワーク基礎  |          407 |
| オブジェクト指向  |          409 |
| 集積回路設計      |          408 |
| 計算機科学史      |          405 |
| フーリエ解析      |          601 |
| 画像処理          |          401 |
| ベクトル解析      |          603 |
+-----+-----+
```

SQL

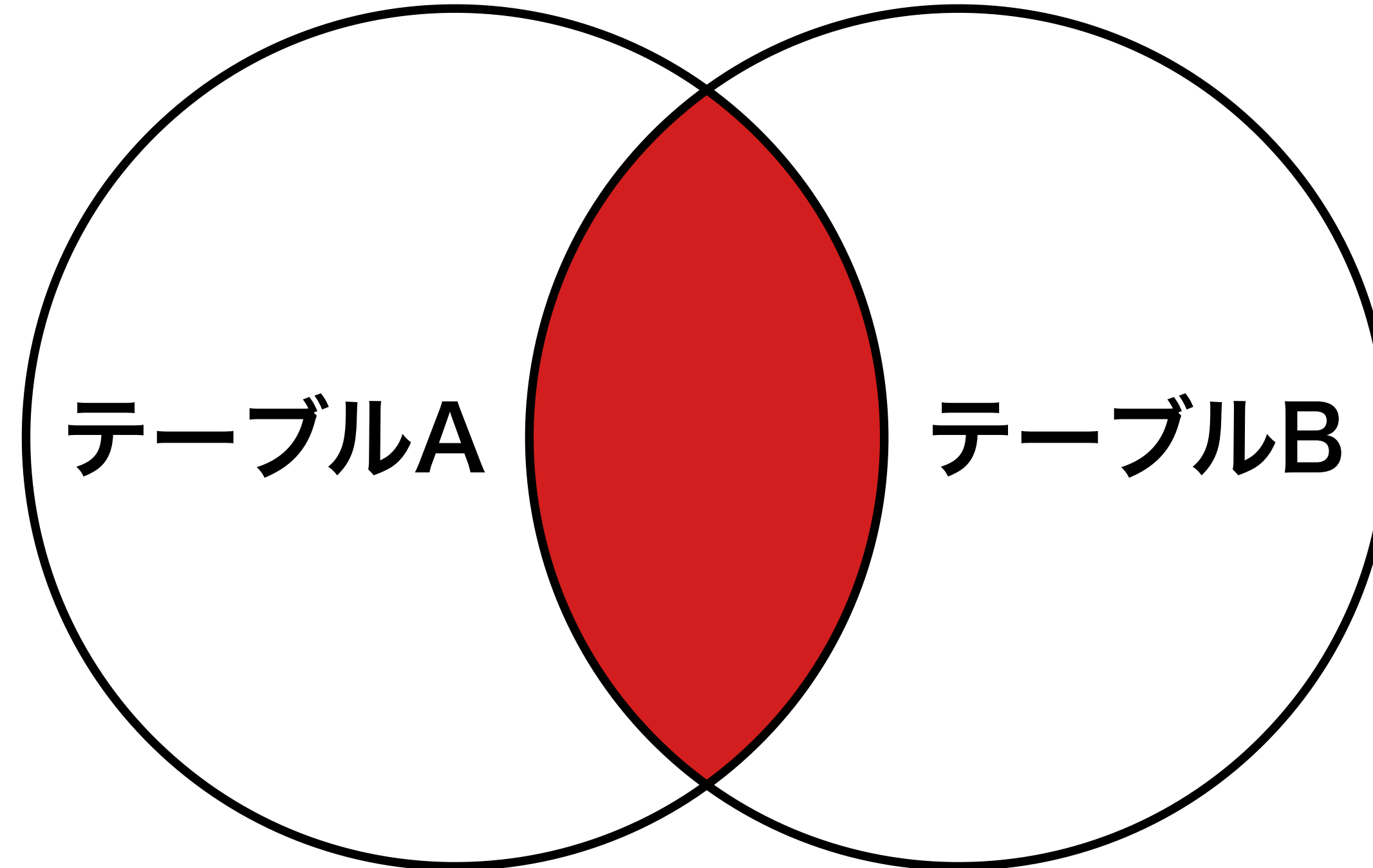
JOIN

- テーブルの確認

```
> SELECT * FROM teacher;
+-----+-----+
| id    | name          |
+-----+-----+
| 401   | 押田 京一     |
| 402   | 荒井 善昭     |
| 403   | 楡井 雅巳     |
| 404   | 西村 治       |
| 405   | 藤澤 義範     |
| 406   | 大矢 健一     |
| 407   | 伊藤 祥一     |
| 408   | 芦田 和毅     |
| 409   | 藤田 悠       |
+-----+-----+
```

SQL JOIN

- 内部結合 (INNER JOIN)



テーブルAとB両方にあるデータをまとめる

SQL

JOIN

- 内部結合 (INNER JOIN)

```
> table1 INNER JOIN table2 ON condition
```

科目(subject)	教員ID(teacher_id)
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

教員ID(id)	教員(name)
401	押田 京一
402	荒井 善昭
403	榆井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

SQL

JOIN

- 内部結合 (INNER JOIN)

```
> SELECT * FROM subject INNER JOIN teacher ON subject.teacher_id = teacher.id;
```

subject	teacher_id	id	name
画像処理	401	401	押田 京一
電子回路	402	402	荒井 善昭
電磁気学	404	404	西村 治
シミュレーション	404	404	西村 治
計算機科学史	405	405	藤澤 義範
ネットワーク基礎	407	407	伊藤 祥一
集積回路設計	408	408	芦田 和毅
オブジェクト指向	409	409	藤田 悠

- つながりのない「フーリエ解析」、「ベクトル解析」は表示されていない

SQL

JOIN

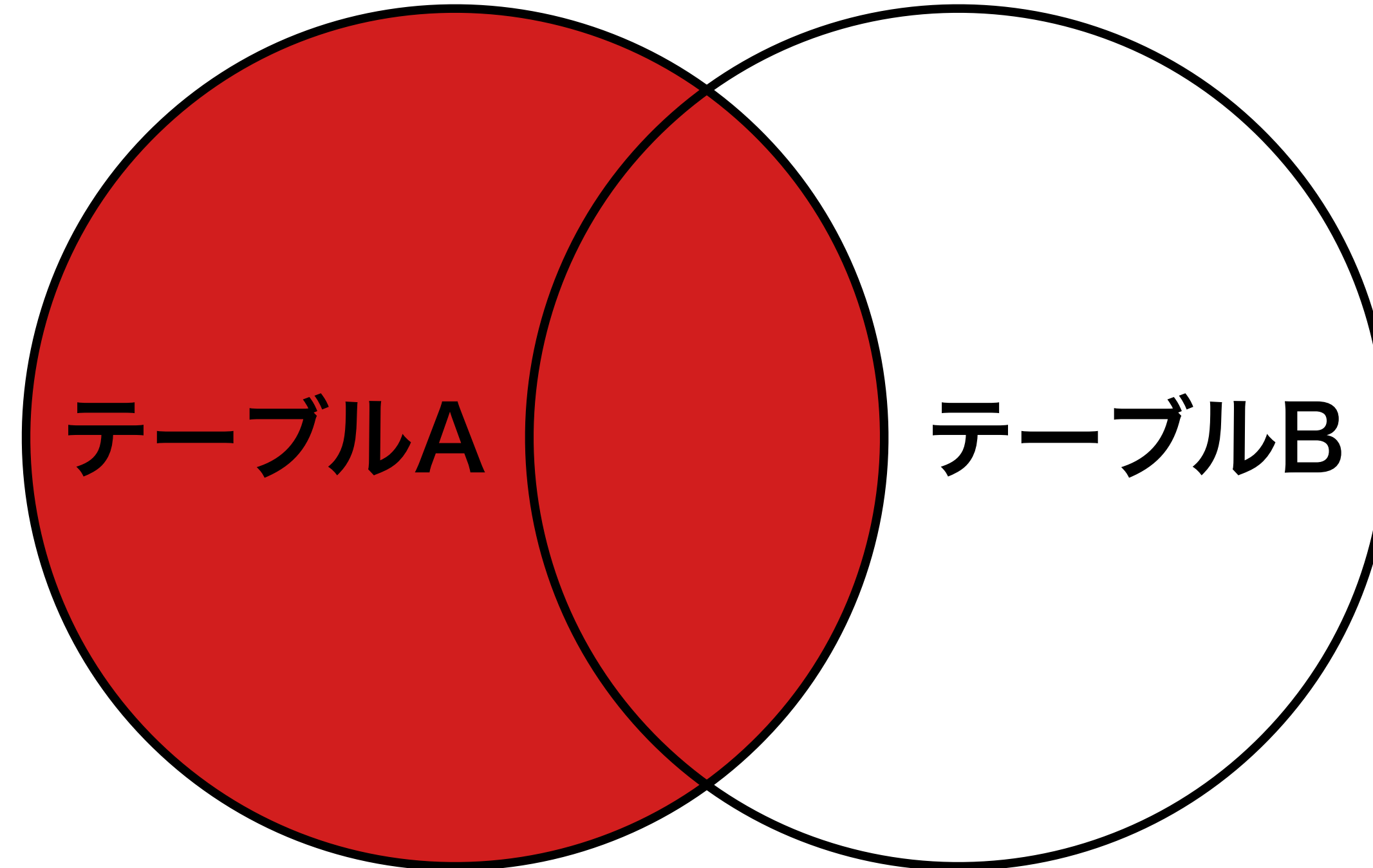
- 内部結合 (INNER JOIN)

```
> SELECT subject.subject,teacher.id,teacher.name FROM subject
INNER JOIN teacher ON subject.teacher_id = teacher.id;
```

subject	id	name
画像処理	401	押田 京一
電子回路	402	荒井 善昭
電磁気学	404	西村 治
シミュレーション	404	西村 治
計算機科学史	405	藤澤 義範
ネットワーク基礎	407	伊藤 祥一
集積回路設計	408	芦田 和毅
オブジェクト指向	409	藤田 悠

SQL JOIN

- 外部結合 (OUTER JOIN): 左外部結合



テーブルAに存在するデータとテーブル間で一致するデータをまとめる

SQL

JOIN

- 外部結合 (OUTER JOIN): 左外部結合

```
> table1 LEFT OUTER JOIN table2 ON condition
```

科目(subject)	教員ID(teacher_id)
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

左側を主として結合



教員ID(id)	教員(name)
401	押田 京一
402	荒井 善昭
403	楡井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

SQL

JOIN

- 外部結合 (OUTER JOIN): 左外部結合

> SELECT * FROM subject LEFT OUTER JOIN teacher
ON subject.teacher_id = teacher.id;

subject	teacher_id	id	name
画像処理	401	401	押田 京一
電子回路	402	402	荒井 善昭
電磁気学	404	404	西村 治
シミュレーション	404	404	西村 治
計算機科学史	405	405	藤澤 義範
ネットワーク基礎	407	407	伊藤 祥一
集積回路設計	408	408	芦田 和毅
オブジェクト指向	409	409	藤田 悠
フーリエ解析	601	NULL	NULL
ベクトル解析	603	NULL	NULL

SQL

JOIN

- 外部結合 (OUTER JOIN): 右外部結合

```
> table1 RIGHT OUTER JOIN table2 ON condition
```

科目(subject)	教員ID(teacher_id)
電磁気学	404
電子回路	402
シミュレーション	404
ネットワーク基礎	407
オブジェクト指向	409
集積回路設計	408
計算機科学史	405
フーリエ解析	601
画像処理	401
ベクトル解析	603

右側を主として結合



教員ID(id)	教員(name)
401	押田 京一
402	荒井 善昭
403	楡井 雅巳
404	西村 治
405	藤澤 義範
406	大矢 健一
407	伊藤 祥一
408	芦田 和毅
409	藤田 悠

SQL

JOIN

- 外部結合 (OUTER JOIN): 右外部結合

> SELECT * FROM subject RIGHT OUTER JOIN teacher
ON subject.teacher_id = teacher.id;

subject	teacher_id	id	name
電磁気学	404	404	西村 治
電子回路	402	402	荒井 善昭
シミュレーション	404	404	西村 治
ネットワーク基礎	407	407	伊藤 祥一
オブジェクト指向	409	409	藤田 悠
集積回路設計	408	408	芦田 和毅
計算機科学史	405	405	藤澤 義範
画像処理	401	401	押田 京一
NULL	NULL	403	楡井 雅巳
NULL	NULL	406	大矢 健一

7章

ファイル ディレクトリ設定

- ・ ファイル出力時、標準では以下のようなエラーが出る

```
> use tshirts_sales_db;  
> SELECT * FROM sales into outfile "test.txt";  
ERROR 1290 (HY000): The MySQL server is running with the --secure-file-priv option so it  
cannot execute this statement
```

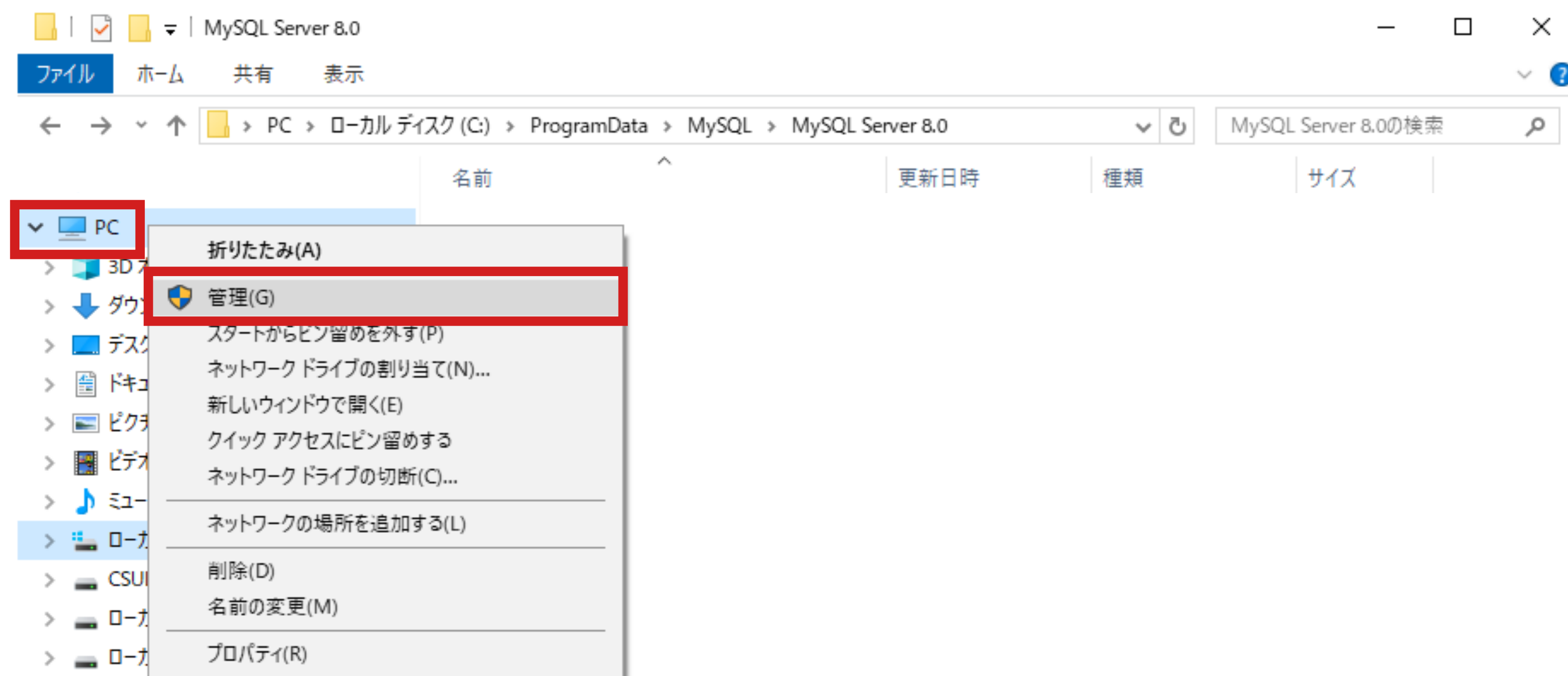
- ・ 「C:\ProgramData\MySQL\MySQL Server 8.4\my.ini」を
以下のように変更

157	# This variable is used to limit the effect of data import and export operations, such as
158	# those performed by the LOAD DATA and SELECT ... INTO OUTFILE statements and the
159	# LOAD_FILE() function. These operations are permitted only to users who have the FILE privilege.
160	#secure-file-priv="C:/ProgramData/MySQL/MySQL Server 8.4/Uploads" ← これをコメントアウト
161	secure-file-priv="" ← これを追加

変更できたらMySQLを再起動

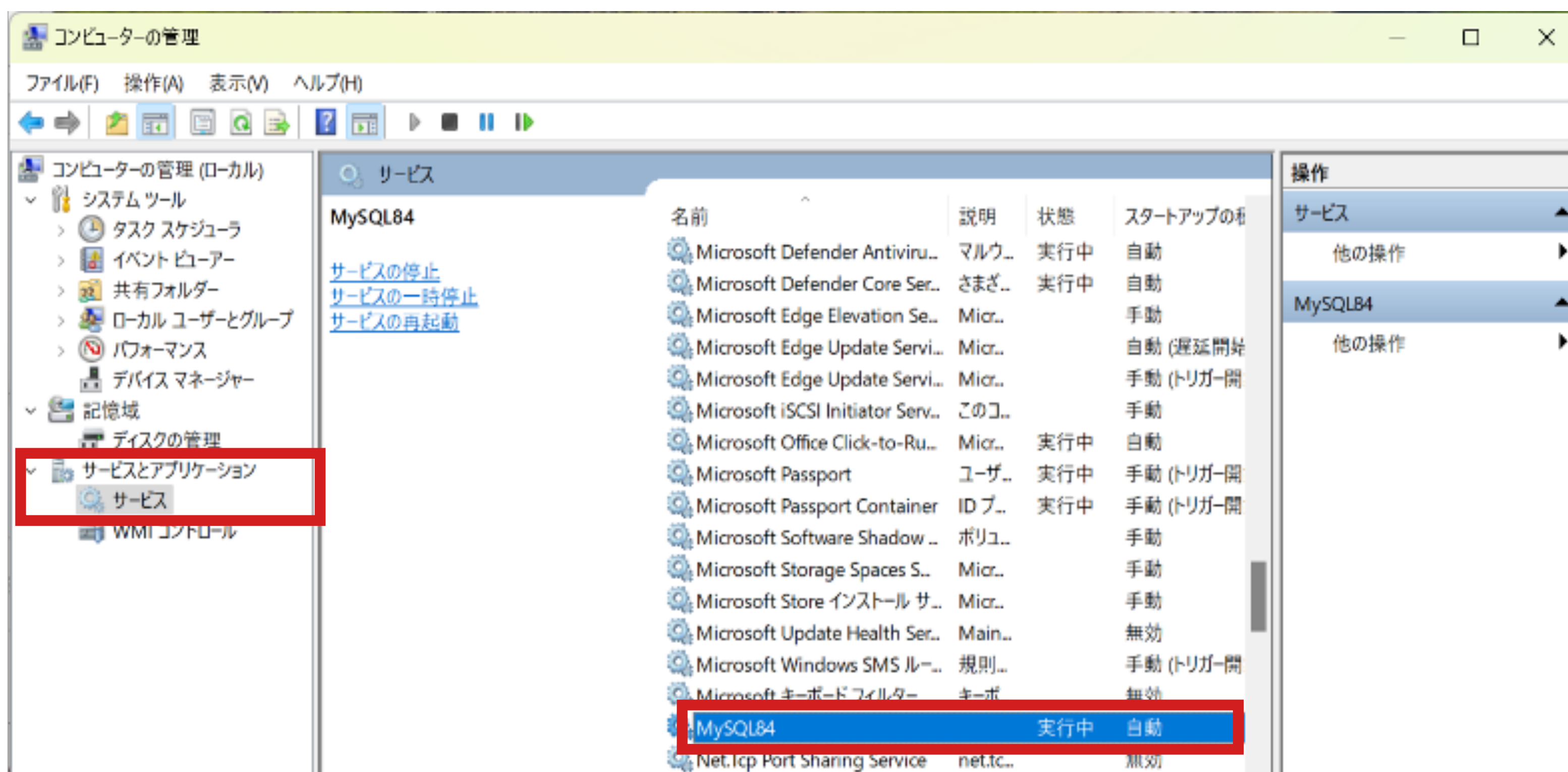
ファイル ディレクトリ設定

- ・ エクスプローラー → 「PC」 を右クリック
- ・ その他のオプション → 管理 → 「コンピュータの管理」が開く



ファイル ディレクトリ設定

- ・ サービスとアプリケーション → サービス
- ・ 「MySQL84」を右クリック → 再起動



ファイル ディレクトリ設定

- ・ Cドライブ直下 (c:\) にtmpフォルダを作成する
- ・ 以下のSQL文を実行する

```
> select * from sales into outfile 'c:/tmp/test.txt';
```

ファイル

ファイルからのINSERT

- ・タブ区切り

実行前

```
> use company;
> select * from member;
+-----+-----+-----+-----+
| id  | name          | place  | age  |
+-----+-----+-----+-----+
| 101 | 木村太郎      | 岩手   | 28   |
| 102 | 田中花子      | 宮城   | 38   |
| 103 | 中村裕子      | 宮城   | 41   |
| 104 | 中村康太      | 青森   | 41   |
+-----+-----+-----+-----+
```

ファイル

ファイルからのINSERT

- 以下のテキストファイルを作成する
 - ファイル名: member.txt
 - 文字コード: UTF-8
 - 区切り文字: タブ
 - 改行コード: LF

105	伊藤加奈子	福島	32
106	長谷川修次	秋田	45

ファイル

ファイルからのINSERT

- 以下のSQL文を実行

```
LOAD DATA INFILE 'c:/tmp/member.txt' INTO TABLE member;
```

実行後

```
> select * from member;
+-----+-----+-----+-----+
| id    | name                | place  | age  |
+-----+-----+-----+-----+
| 101   | 木村太郎            | 岩手   | 28   |
| 102   | 田中花子            | 宮城   | 38   |
| 103   | 中村裕子            | 宮城   | 41   |
| 104   | 中村康太            | 青森   | 41   |
| 105   | 伊藤加奈子          | 福島   | 32   |
| 106   | 長谷川修次          | 秋田   | 45   |
+-----+-----+-----+-----+
```

ファイル

ファイルからのINSERT

- 以下のテキストファイルを作成する
 - ファイル名: member2.txt
 - 文字コード: UTF-8
 - 区切り文字: , (カンマ)
 - 改行コード: LF

107,田村浩二,青森,24

108,上村優子,岩手,56

ファイル

ファイルからのINSERT

- 以下のSQL文を実行

```
LOAD DATA INFILE 'c:/tmp/member2.txt' INTO TABLE member fields terminated by ',';
```

```
> select * from member;
+-----+-----+-----+-----+
| id    | name          | place  | age  |
+-----+-----+-----+-----+
| 101   | 木村太郎      | 岩手   | 28   |
| 102   | 田中花子      | 宮城   | 38   |
| 103   | 中村裕子      | 宮城   | 41   |
| 104   | 中村康太      | 青森   | 41   |
| 105   | 伊藤加奈子    | 福島   | 32   |
| 106   | 長谷川修次    | 秋田   | 45   |
| 107   | 田村浩二      | 青森   | 24   |
| 108   | 上村優子      | 岩手   | 56   |
+-----+-----+-----+-----+
```

ファイル

コマンド読み込み

- ・ データベース作成、SQL操作などの命令が書かれたテキストファイル
- ・ SQL文として入力する
- ・ 例えば以下のようなテキストファイル (sql.txt) を作成

```
CREATE DATABASE sql12345;  
USE sql12345;  
CREATE TABLE books(  
  id INT NOT NULL PRIMARY KEY,  
  name varchar(128),  
  author char(64)  
);  
INSERT INTO books (id, name, author) VALUES  
  ("10001", "ライ麦畑でつかまえて", "J・D・サリンジャー"),  
  ("10002", "モモ", "ミヒヤエル・エンデ"),  
  ("10003", "二年間の休暇", "ジュール ヴェルヌ")  
;  
SELECT id, name, author FROM books;
```

ファイル

コマンド読み込み

- 実行

```
> source c:/tmp/sql.txt
Query OK, 1 row affected (0.01 sec)

Database changed
Query OK, 0 rows affected (0.03 sec)

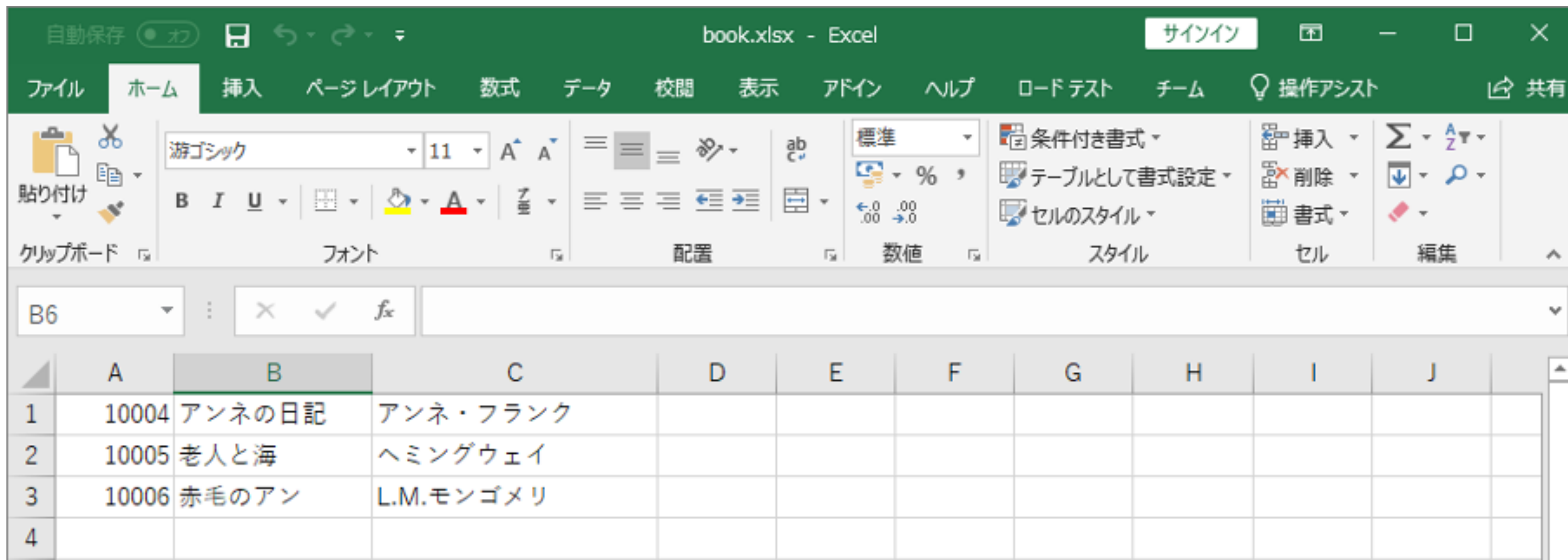
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

id	name	author
10001	ライ麦畑でつかまえて	J・D・サリンジャー
10002	モモ	ミヒヤエル・エンデ
10003	二年間の休暇	ジュール ヴェルヌ

3 rows in set (0.00 sec)

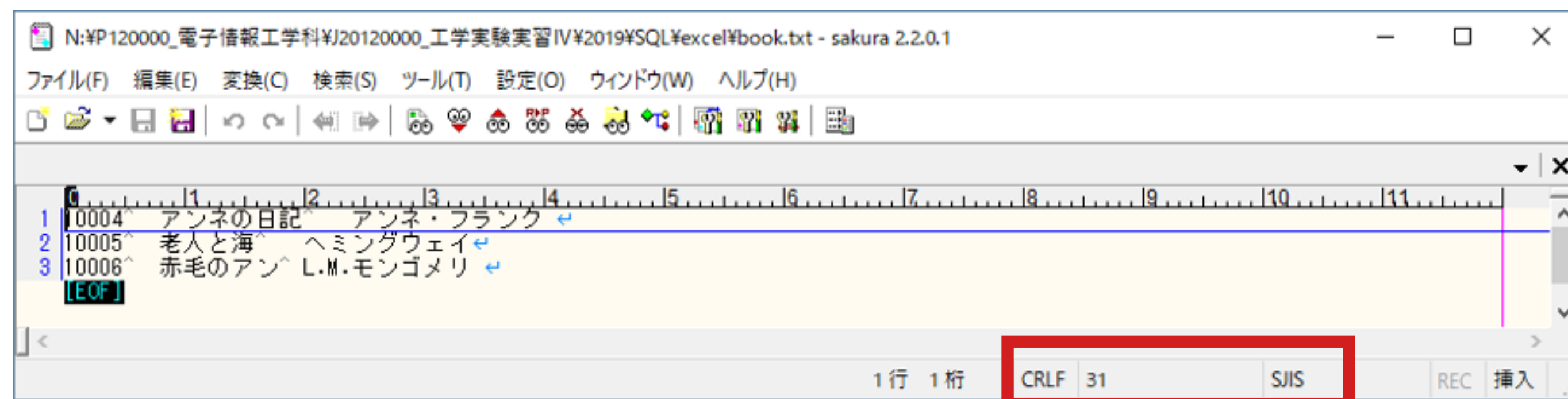
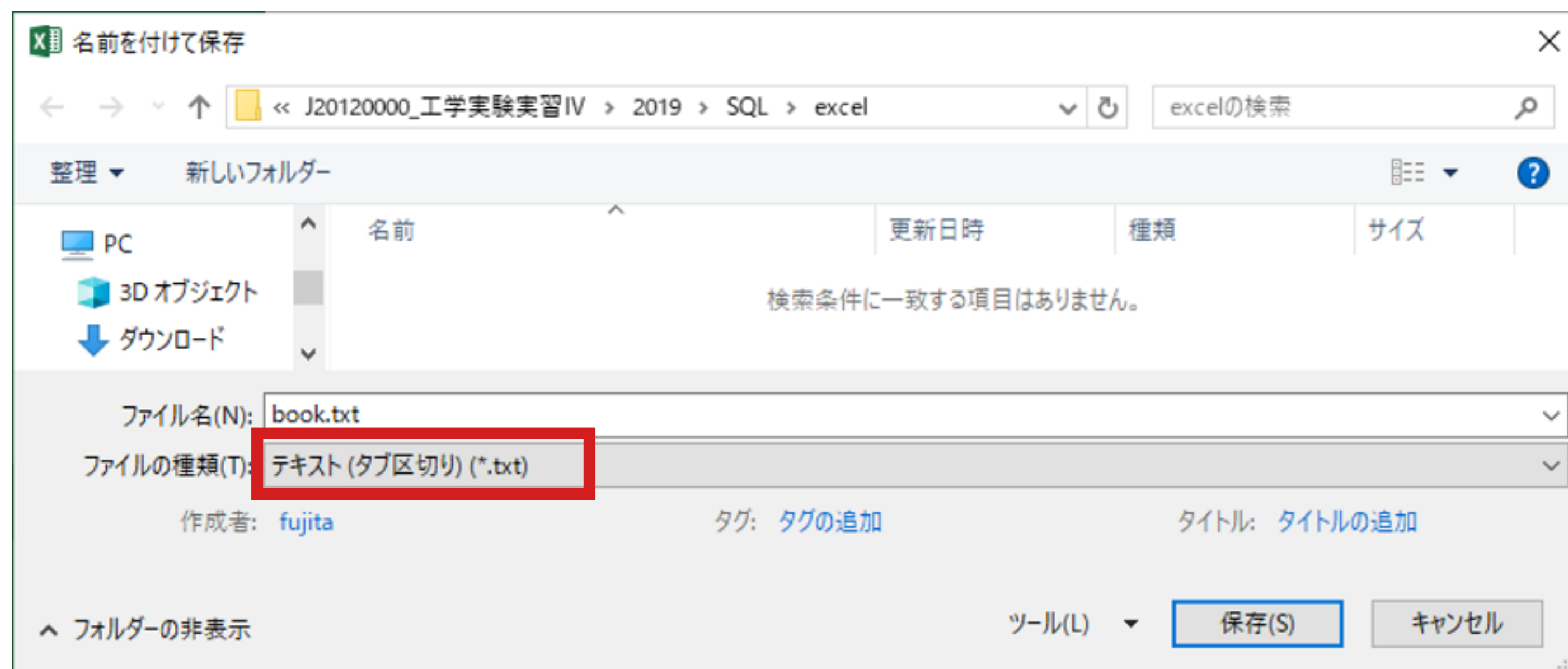
Excelのデータ出力

- 以下のようなExcelファイルがあるとする



ファイル Excelのデータを出力

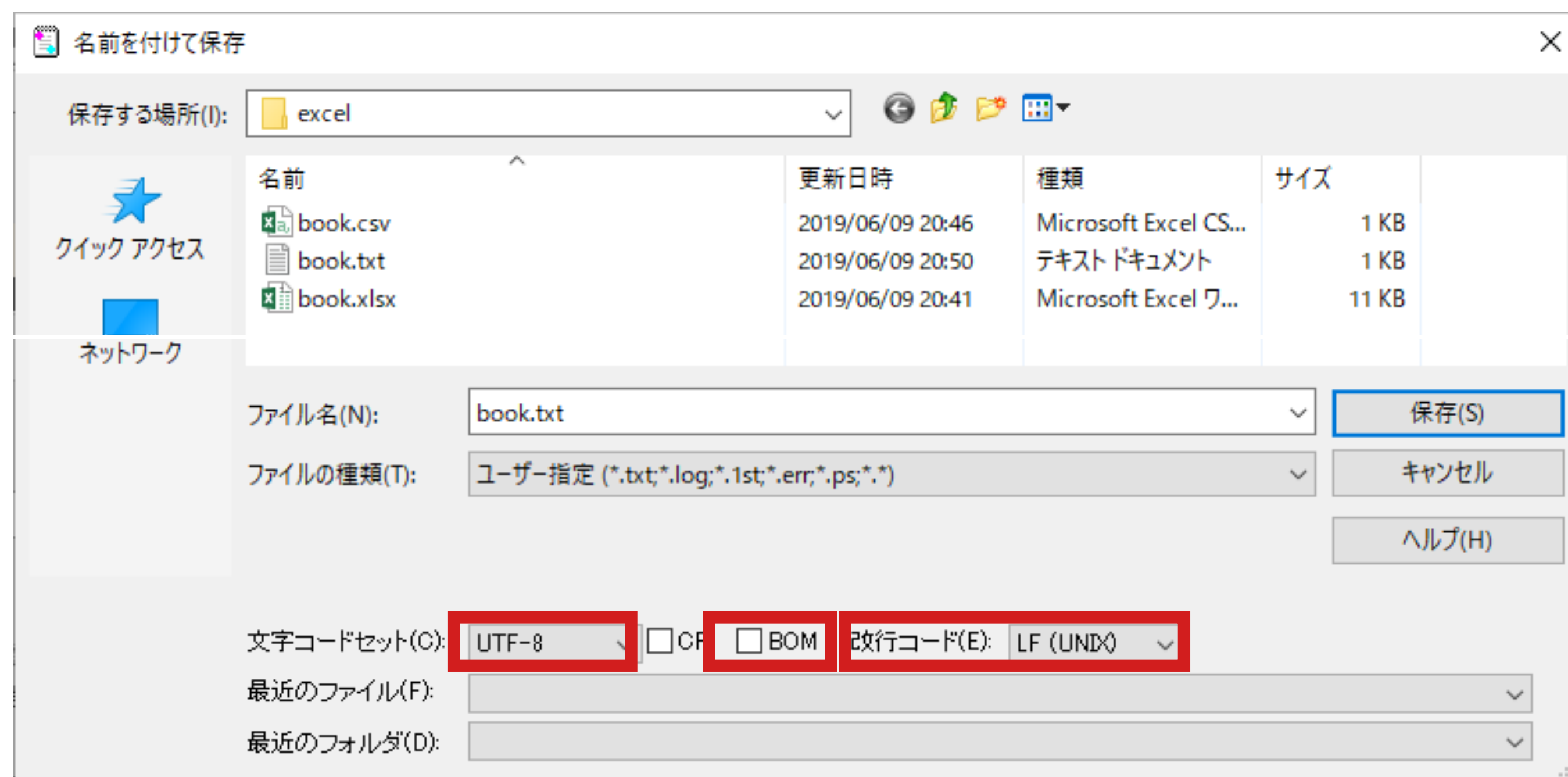
- 名前を付けて保存 → 「ファイルの種類」で「テキスト (タブ区切り)」を選択



ファイル

Excelのデータを出力

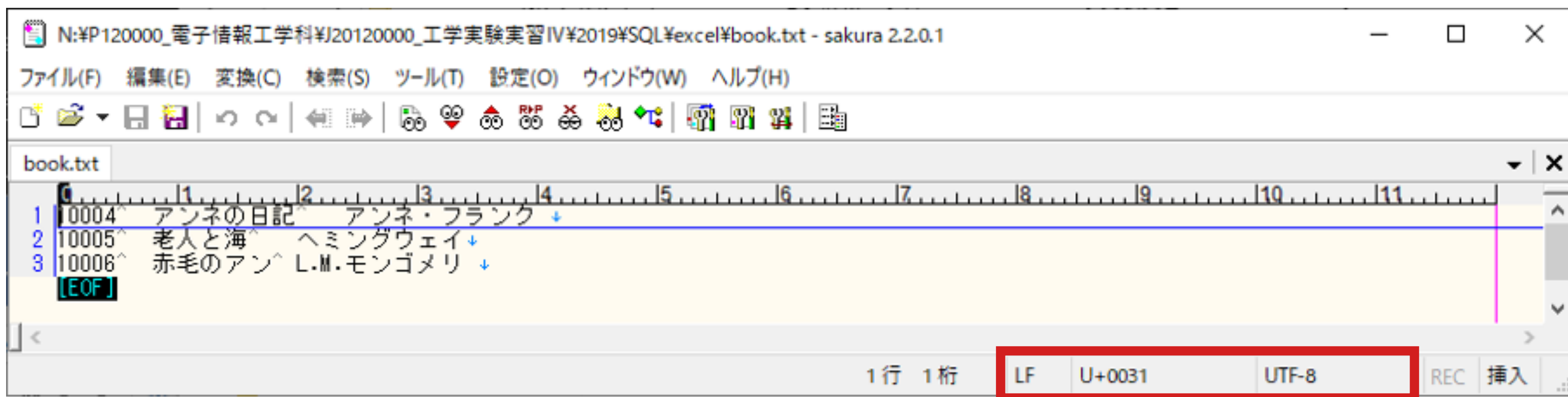
- ・ テキストエディタから以下の設定で名前を付けて保存
 - ・ 文字コード: UTF-8
 - ・ BOM: なし、オフ
 - ・ 改行コード: LF



ファイル

Excelのデータを出力

- ・ テキストエディタから以下の設定で名前を付けて保存
 - ・ 文字コード: UTF-8
 - ・ BOM: なし、オフ
 - ・ 改行コード: LF



ファイル

Excelのデータを出力

- 出力したファイルをレコードとして読み込む

```
> LOAD DATA INFILE 'c:/tmp/book.txt' INTO TABLE books;
```

```
> select * from books;
```

id	name	author
10001	ライ麦畑でつかまえて	J・D・サリンジャー
10002	モモ	ミヒヤエル・エンデ
10003	二年間の休暇	ジュール ヴェルヌ
10004	アンネの日記	アンネ・フランク
10005	老人と海	ヘミングウェイ
10006	赤毛のアン	L.M. モンゴメリ

9章

課題

課題

課題1

- 以下のテーブルを第3正規化までせよ
 - 第1正規化から第3正規化までの過程も示す
- 第3正規化した結果をER図で表す

学籍番号	学生名	年齢	性別	系	校舎	科目名	点数	判定	教員コード	教員名	入力日
26401	山川 一郎	18	男	都市デザイン系	C棟	国語	65	可	G002	長松 次郎	2025-06-10
						物理	42	不可	G001	長松 遥	2025-06-11
						現代社会	79	良	G005	高田 真由	2025-06-14
26205	谷泉 太郎	19	男	情報エレクトロニクス系	B棟	物理	76	良	G001	長松 遥	2025-06-11
						電気工学	66	可	E012	三浦 裕	2025-06-12
						信号処理	90	優	E012	三浦 裕	2025-06-12
26132	海土 花子	20	女	機械ロボティクス系	A棟	物理	87	優	G001	長松 遥	2025-06-10
						機械力学	78	良	M301	上条 雄三	2025-06-11
						計測工学	100	優	M553	西岡 剛	2025-06-12

課題

課題1

- 第3正規化では外部キー（他のテーブルと関連づけるカラム）の設定まで行う
- レポートにはWordの表やExcelの表を示す
- ER図はastahから出力できる
 - ツール → 画像出力

課題

課題1: ヒント

- 点数と判定は以下の左の表のように対応させる
- 判定のテーブルは以下の右の表のようになっていると考える
 - このテーブルは他のテーブルとのリレーションを持たない
- 最終的に8個のテーブルになる

点数	判定
0～59	不可
60～69	可
70～79	良
80～100	優

id	上限	下限	判定
1	59	0	不可
2	?	?	可
3	?	?	良
4	?	?	優

課題

課題2

- ・ 課題1で第3正規化したテーブルを以下の表に示す設定を用いて作成せよ
- ・ 定義したデータベース、テーブルを生成するSQL文、データを挿入するSQL文を定義する
 - ・ テーブルを生成するSQL文は、課題1で作成したER図からSQLエクスポートで取得する
- ・ テーブルを実装した結果として、SELECT文を実行し、各テーブルに登録した内容を表示せよ

項目	カラム（論理名）	カラム（物理名）	型	必須／非必須
学籍番号	学籍番号	code	CHAR(5)	必須
学生名	名前	name	VARCHAR(40)	必須
年齢	年齢	age	INT	非必須
性別	性別	gender	CHAR(4)	非必須
系	系	course	VARCHAR(40)	必須
校舎	校舎	place	VARCHAR(40)	必須
科目名	科目名	subject	VARCHAR(40)	必須
教員コード	教員コード	teacher_id	CHAR(4)	必須
教員名	名前	name	VARCHAR(40)	必須
点数	点数	score	INT	必須
判定	判定	result	VARCHAR(2)	必須
入力日	入力日	date	DATE	必須

課題

課題3

- ・ 課題2で登録したデータ (第3正規化したテーブル) に対し以下の問いに答えよ
 - ・ レポートには、SQL文も実行結果も両方報告すること
- 1. 第1正規化したときの表を表示するSQLを定義せよ
- 2. いずれかの科目で判定が優の学生の学籍番号、氏名、系を問い合わせるためのSQL文を実行、実行結果を示せ
 - ・ ただし、条件文は「80点以上」とせず、「判定が優である」とすること
- 3. 担当が「高田」または「三浦」の科目を履修している学生の、学生名、科目名、担当、判定を問い合わせるSQL文を実行し、実行結果を示せ
 - ・ ただし表示するレコードは「高田」または「三浦」が担当している科目のもののみでよい
- 4. 科目名に「学」が含まれる科目について、履修している学生名、科目名、点数を問い合わせるSQL文を実行、実行結果を示せ
- 5. 担当が「三浦」の科目の平均点を問い合わせるSQL文を実行し、実行結果を示せ

レポートに書くこと

1. 表紙

- ・タイトル、所属、学籍番号、組番号、名前、実験日を必ず記入

2. 目的

3. 原理

4. 実験環境

- ・ハードウェア (PCスペック)、ソフトウェア (バージョンなど)

5. 課題

- ・どのような課題か？何をしたのか？
- ・課題に応じて、表や画像、SQL文、実行結果を示す

6. 考察

- ・今回の実験によって得られた、考察を述べる

評価のポイント

- 期限内提出: 10%
- 実験実施態度と完了報告: 20%
- レポートの内容: 70%
 - レポートは100点満点で評価

評価のポイント

1. レポート全般: 30点

- ・ レポートの体裁 (10)
 - ・ 表紙、章立てができているか、誤字脱字はないか
- ・ SQL、データベースについて十分説明している (10)
- ・ 考察が十分書けている (10)

2. 課題1: 30点

- ・ 第1正規化: 10点
- ・ 第2正規化: 10点
- ・ 第3正規化: 10点

3. 課題2: 15点

- ・ データベース・テーブルを生成するSQL文: 5点
- ・ 各テーブルにデータを挿入するSQL文: 5点
- ・ 各テーブルに登録した内容を表示するSQL文とその実行結果: 5点

4. 課題3: 25点 (5点 x 5問)

- ・ 1)～5)を実現するSQL文とその実行結果: 各5点

提出方法

Teamsの「課題」へ提出

提出ファイル

- レポートのPDF
- SQL文の書かれたファイル
 - .sqlがうまく作成できない場合はテキストファイルでも可
 - データベース、テーブル作成、データ挿入、課題1～3が1ファイルに含まれていること

提出ファイル名

- レポートのPDF: 組番号_氏名.pdf (例: 493_萩原隆義.pdf)
- SQLファイル: 組番号_氏名.sql または .txt (例: 493_萩原隆義.sql)